

Designing Musical Networks

From Linear Algebra to Music Composition

Carmine-Emanuele Cella

May 30, 2026

draft — not for circulation

Draft.

This is a working draft of *Designing Musical Networks*. Not for circulation. Comments welcome.

Colophon.

Typeset with [KOMA-Script](#) and $\text{\LaTeX}$ using the [kaobook](#) class. Code companion: `weft`, a header-only C++17 neural network library available alongside this book.

For those who hear mathematics in music
and music in mathematics.

– the author

Preface

Contents

Contents

vii

FOUNDATION 1

1	Signals as vectors	3
1.1	Two notes	3
1.2	Vector spaces	3
1.3	Norm: the size of a signal	4
1.4	Inner product: global similarity	4
1.5	Convolution: local similarity	5
1.6	Basis and projection	6
1.7	Analysis and synthesis as matrices	7
1.8	From basis to representation	7
1.9	What we have so far, and what comes next	8
2	A-priori representations	11
2.1	The question and three answers	11
2.2	The Discrete Fourier Transform	12
2.3	The Discrete Wavelet Transform	13
2.4	Between transforms: the time-frequency trade	15
2.5	The Scattering Transform	15
2.6	Closing thoughts	17
3	Learned representations	19
3.1	When the basis comes from data	19
3.2	Principal Component Analysis	19
3.3	K-means and prototypes	21
3.4	Matching pursuit and sparse dictionaries	21
3.5	Neural networks	23
3.6	Convolutional neural networks	24
3.7	The unified picture	25

TECHNOLOGY 27

4	Why C++?	29
4.1	What the book is for	29
4.2	Why frameworks hide what we want to teach	29
4.3	What C++ forces you to do	30
4.4	Why C++ specifically, instead of Python without frameworks?	30
4.5	Honest counterarguments	31
4.6	When you would NOT use C++	32
4.7	The point	32

5	Matrix: the foundation	33
5.1	The conceptual leap: from neuron to matrix	33
5.1.1	A neuron is a dot product	33
5.1.2	A layer is a matrix	33
5.1.3	Batching: why we need matrix $\times$ matrix	34
5.2	Conventions used in weft	34
5.3	Walking through Matrix.h	34
5.3.1	Why templated on the scalar type	34
5.3.2	Why a flat <code>std::vector</code> , not vector-of-vectors	35
5.3.3	Why the i-k-j loop ordering for matmul	35
5.3.4	Why + quietly broadcasts a column vector	35
5.3.5	Why apply is templated on the function	35
5.3.6	Why hadamard is a free function	36
5.4	Seeing it work: the forward pass falls out	36
5.5	What's next	36
6	Backpropagation, and the Dense layer	39
6.1	Backprop is the chain rule, organized	39
6.2	Each layer is a self-contained gradient router	39
6.3	The intuition: perturbation propagation	40
6.4	Deriving Dense's backward pass	40
6.5	The transpose symmetry — and a free sanity check	41
6.6	Implementation notes	41
6.6.1	Caching	41
6.6.2	The Layer base class	42
6.6.3	Initialisation	42
6.6.4	Where the <code>1/batch_size</code> lives (decision worth flagging)	42
6.7	Testing — and why the numerical gradient check matters more than anything	42
6.8	What's next	43
7	One weight, one step (SGD by hand)	45
7.1	The setup	45
7.2	Step 1 — Forward pass (the projection)	45
7.3	Step 2 — Turning “off by 0.9” into a single number: the loss	46
7.4	Step 3 — The “error signal” $\partial L / \partial z$	46
7.5	Step 4 — Backpropagating to w_1 via the chain rule	46
7.6	Step 5 — The SGD update	47
7.7	Step 6 — Did it actually help?	47
7.8	Where is “squared error” in the SGD update?	48
7.9	How this scales up to the matrix formulas	48
7.10	See it run	49
8	Non-linearities (ReLU and Softmax)	51
8.1	Why nonlinearities are not optional	51
8.2	ReLU — as simple as it gets	51
8.2.1	Backward	52
8.2.2	Why it works so well	52
8.3	Softmax — turning scores into probabilities	52
8.3.1	3a. The numerical stability trick (log-sum-exp shift)	53
8.3.2	3b. The Jacobian — the first non-trivial backward	53
8.3.3	3c. Backward from the Jacobian	54

8.3.4	3d. The Softmax + cross-entropy shortcut (preview)	54
8.4	Other activations (briefly)	54
8.5	What's next	55
9	The Jacobian (the unifying object behind every backward pass)	57
9.1	What a Jacobian is	57
9.1.1	Scalar function of a scalar	57
9.1.2	Scalar function of a vector	57
9.1.3	Vector function of a vector	57
9.1.4	The geometric picture	58
9.2	Backprop is "multiply by the transposed Jacobian"	58
9.3	The Jacobians of weft's three layers	58
9.3.1	Dense: J is W	59
9.3.2	ReLU: J is diagonal	59
9.3.3	Softmax: J is dense, but structured	59
9.4	A concrete worked example	59
9.5	Never materialise a Jacobian you can avoid	60
9.6	Summary	61
10	Cross-entropy loss (and the Softmax+CE shortcut)	63
10.1	Why not squared error?	63
10.2	Cross-entropy: definition and interpretation	63
10.3	The batch form	64
10.4	The standalone gradient	64
10.5	The Softmax + CE shortcut (the beautiful part)	65
10.6	Why we still implement them separately in weft	65
10.7	What we built	66
10.8	What's next	66
11	Network: layers composed	67
11.1	Forward = function composition	67
11.2	Backward = composition, reversed	67
11.3	Ownership: <code>unique_ptr<Layer<T>></code>	68
11.4	The <code>add<L>(args...)</code> template helper	68
11.5	Loss stays outside Network	68
11.6	The full-pipeline numerical gradient check	69
11.7	What we built	70
11.8	What's next: IRIS	70
12	IRIS: the first end-to-end classifier	71
12.1	The standard ML pipeline	71
12.2	Why standardise features?	71
12.3	The "fit on train only" rule	72
12.4	The training loop pattern	72
12.5	Accuracy as the eval metric	73
12.6	What the example actually shows	74
12.7	The seed-variance warning	74
12.8	What's next	75
13	Optimizers (SGD and Adam)	77
13.1	Why plain SGD struggles	77
13.2	Momentum — accelerate in consistent directions	77

13.3	RMSProp — per-parameter step sizes	78
13.4	Adam = momentum + RMSProp + bias correction	78
13.4.1	What Adam intuitively does	78
13.4.2	Typical hyperparameters	79
13.5	The Optimizer abstraction	79
13.5.1	In the training loop	80
13.6	The empirical comparison	80
13.7	Practical recommendations	81
13.8	What’s next	81
14	Train/eval mode and Dropout	83
14.1	Why a mode flag exists at all	83
14.2	Dropout — what and why	84
14.2.1	Why it works	84
14.2.2	Common rates	84
14.2.3	Where to place it	85
14.3	The code	85
14.3.1	Testing dropout	86
14.4	Looking ahead	86
15	MNIST classifier	87
15.1	The data pipeline	87
15.2	The architecture	87
15.3	The optimiser choice	88
15.4	The training loop in detail	88
15.5	Train vs test accuracy: reading the gap	89
15.6	Running it	89
15.7	What’s missing (worth knowing)	90
16	The FFT	91
16.1	The DFT	91
16.2	Why we don’t compute the DFT directly	91
16.3	The Cooley-Tukey trick	91
16.4	From recursive to iterative	92
16.5	The inverse	92
16.6	What we’ll actually use it for	93
17	Loading audio and extracting features	95
17.1	The WAV format and our reader	95
17.1.1	Endianness	95
17.1.2	What we actually accept	96
17.1.3	Stereo to mono	96
17.2	The framing pipeline	96
17.2.1	Why a window function?	97
17.2.2	Why energy-weighted averaging?	97
17.3	Log-magnitude spectrum	97
17.4	MFCC: Mel-Frequency Cepstral Coefficients	98
17.4.1	The mel scale	98
17.4.2	The mel filterbank	99
17.4.3	Why the DCT at the end?	99
17.5	Averaged vs per-frame output	99
17.6	Which feature to choose?	100

17.7	What's not in here (yet)	100
18	Musical instrument classifier (SOL)	103
18.1	Why two tools	103
18.2	Two extraction modes	103
18.2.1	Averaged (one feature per file)	103
18.2.2	Per-frame (one feature per non-silent frame)	104
18.3	The file-leakage problem (and the fix)	104
18.4	Per-file aggregation at eval time	105
18.5	Confusion matrix	105
18.6	The cache format (v2)	105
18.7	What to expect from the four combinations	106
18.8	What's still not in here	106
19	Autoencoders	109
19.1	MSE loss	109
19.2	Sigmoid activation	110
19.3	What an autoencoder is	110
19.4	Why two Networks instead of one stack	111
19.5	Demo 1 — reconstruction	111
19.6	Demo 2 — latent interpolation	112
19.7	The catch, and what's next	112
20	Variational autoencoders	113
20.1	Encode a distribution, not a point	113
20.2	The reparameterisation trick	113
20.3	The KL divergence term	114
20.4	The total loss, and the balance	115
20.5	Latent dimension: quality vs. the manifold picture	115
20.6	The demos	116
20.7	What this sets up for audio	116
21	Visualising weights	119
21.1	Weights are images	119
21.2	The linear classifier: ten templates	119
21.3	Why a diverging colormap	119
21.4	Reaching the weights	120
21.5	Deeper networks	120
22	The audio VAE: training, serialization, and remapping	121
22.1	Serialization — train once, apply many times	121
22.2	The log-magnitude pipeline	121
22.3	Training (train_audio_vae)	122
22.4	Remapping (vae_remap) — projection onto the manifold	122
22.5	What to expect from the remap	123
22.6	Generating (vae_generate): interpolation and sampling	123
22.7	Bigger picture	124
23	Making training practical	125
23.1	Where the time goes	125
23.2	Compute the same work faster	125
23.3	Do less work	126

23.4	Make long runs safe	126
23.5	An optional bigger hammer: BLAS	126
23.6	The hammer we still leave in the drawer: GPU	127
24	From matrices to tensors	129
24.1	What a tensor is	129
24.2	NCHW: the order of the axes	129
24.3	One class or two?	130
24.4	The case for Path A (unified Tensor)	130
24.5	The case for Path B (what we have)	131
24.6	Why production libraries chose Path A	132
24.7	The principle, and a look ahead	132
25	Convolutional networks	133
25.1	Why dense MLPs are wrong for images	133
25.2	The convolution operator	134
25.3	Padding and stride	134
25.4	im2col: making convolution fast on a CPU	135
25.5	Pooling	136
25.6	The conv-to-dense bridge	136
25.7	What the library provides	137
25.8	The CIFAR-10 classifier	138
25.9	The payoff: visualising what the first layer learned	139
25.10	What's next	139
26	A convolutional VAE for audio	141
26.1	Patch shape	141
26.2	Architecture	142
26.3	The frequency-axis problem	143
26.4	Training	144
26.5	Three applications	144
26.6	What this is and is not	144
27	Attention	147
27.1	Queries, keys, values	147
27.2	The scaling	148
27.3	Backward	148
27.4	Why three inputs, why not a Layer	149
27.5	Multi-head attention	149
27.6	Two implementation traps we hit	150
27.7	What's not here yet	150
28	Positional encoding, LayerNorm, and the transformer block	153
28.1	Why position matters	153
28.2	Sinusoidal positional encoding	153
28.3	LayerNorm	154
28.4	The transformer encoder block	155
28.5	What we have	156
28.6	What's next	156

FOUNDATION

Signals as vectors

1

The fool doth think he is wise, but the wise man knows himself to be a fool.

William Shakespeare, *As You Like It*

1.1 Two notes

Listen to a sustained violin note and a sustained flute note, played at the same pitch. They have the same fundamental frequency. They sound nothing alike. Now play them together. They blend into something that is neither violin nor flute — a third instrument that exists only while they are sounding together.

The question *why do they sound different* can be answered at two levels. At the musical level the answer is timbre: the way each instrument's overtones reinforce or attenuate the others, the attack transient, the bow noise or the breath noise, the small fluctuations in pitch and amplitude. At the mathematical level the answer is that the two recordings, considered as long sequences of numbers, point in different directions in a very high-dimensional space. When we add them, we are doing geometry: the result is a third point in the same space, neither of the originals but related to both.

This book lives in the gap between those two answers. The technical chapters (Part II) build software that operates on sounds as high-dimensional vectors. The creative chapters (Part III) reason about timbre, style, and composition in those terms. This first part is about establishing the geometry — the structure of the space that makes the technical work possible and the creative work coherent.

The central claim is simple, and it will reappear in many guises: *representing a signal means projecting it onto a set of reference vectors*. The Fourier transform projects onto sinusoids. The wavelet transform projects onto localized oscillations. Principal component analysis projects onto data-derived directions of maximum variance. A neural network projects onto weight vectors learned from data, then applies a nonlinearity and projects again. The mathematics underlying all four is the same: an inner product, repeated. What differs is the choice of reference vectors and whether they are designed by hand or learned from data.

1.2 Vector spaces

To think of a signal as a point in space, we need to say precisely what “space” means. A *vector space* is a set V on which two operations are defined — addition of elements, and multiplication by scalars — which satisfy a list of axioms.¹ The elements of V are called *vectors*. When the

This is the setup. The math we develop in this chapter — vector spaces, inner products, projections — is the language in which the difference between a violin and a flute, and the meaning of “blend,” can be made precise.

Notation. Vectors are denoted by bold lowercase letters $\mathbf{x}$, $\mathbf{y}$. Matrices by bold uppercase, $\mathbf{W}$, $\mathbf{B}$. The n -th component of vector $\mathbf{x}$ is written $\mathbf{x}[n]$. The inner product of two vectors is $\langle \mathbf{x}, \mathbf{y} \rangle$, and convolution is $\mathbf{x} * \mathbf{y}$. Appendix ?? reviews matrix conventions.

1: The full axiom list appears in Appendix ??; for our purposes, what matters is that they let us add signals and scale them without leaving the space.

scalar field is the real numbers $\mathbb{R}$, we say V is a real vector space; when it is the complex numbers $\mathbb{C}$, a complex vector space.

A discrete-time signal of length N is naturally an element of $\mathbb{R}^N$ (or $\mathbb{C}^N$, if we allow complex samples). The n -th component is the n -th sample. Adding two signals samplewise is vector addition. Multiplying every sample by a constant is scalar multiplication. The zero vector is the silent signal.

In this book we work exclusively in finite dimensions. The infinite-dimensional theory of Hilbert spaces is beautiful and important, but everything we need can be done in $\mathbb{R}^N$ or $\mathbb{C}^N$, and the finite case is what the computer manipulates anyway.

1.3 Norm: the size of a signal

To say one signal is louder than another, or that one is close to another, we need a notion of size and of distance. A *norm* on a vector space V is a function $\|\cdot\| : V \rightarrow \mathbb{R}$ satisfying (i) $\|\mathbf{x}\| \geq 0$ with equality only at $\mathbf{x} = \mathbf{0}$, (ii) $\|\alpha\mathbf{x}\| = |\alpha|\|\mathbf{x}\|$ for any scalar α , and (iii) the triangle inequality $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$.

The norm we will use almost everywhere is the *Euclidean* or L^2 norm,

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{n=0}^{N-1} |\mathbf{x}[n]|^2}. \quad (1.1)$$

The square of this quantity, $\|\mathbf{x}\|_2^2$, has a physical interpretation when $\mathbf{x}$ is an audio signal: it is the *energy* of the signal. We will often work with energy directly rather than with the norm, because it avoids the square root.

A vector space equipped with a norm is called a *Banach space*. Banach spaces are what we need to talk about size and convergence. We need one more piece of structure — an inner product — to talk about angles and projections. That promotes a Banach space to a Hilbert space.

Other norms exist and matter. The L^1 norm $\sum_n |\mathbf{x}[n]|$ is used in sparse coding because it prefers solutions with many zeros. The L^∞ norm $\max_n |\mathbf{x}[n]|$ is the peak amplitude. We will meet L^1 again in Part III when we discuss matching pursuit.

1.4 Inner product: global similarity

The inner product is the single most important operation in this book. For two real-valued signals $\mathbf{x}$ and $\mathbf{y}$ of length N ,

$$\langle \mathbf{x}, \mathbf{y} \rangle = \sum_{n=0}^{N-1} \mathbf{x}[n] \mathbf{y}[n]. \quad (1.2)$$

For complex-valued signals, the second factor is conjugated: $\langle \mathbf{x}, \mathbf{y} \rangle = \sum_n \mathbf{x}[n] \overline{\mathbf{y}[n]}$.

Geometrically, the inner product measures *similarity*. The clean statement is the following. Two vectors with the same magnitude have a large inner product when they point in the same direction, zero when they are perpendicular, and a large negative value when they point in opposite directions.² When two vectors are identical, their inner product is the

2: The relation $\langle \mathbf{x}, \mathbf{y} \rangle = \|\mathbf{x}\| \|\mathbf{y}\| \cos \theta$, where θ is the angle between them, makes this precise. When $\theta = 0$ the cosine is 1; when $\theta = \pi/2$ it is zero; when $\theta = \pi$ it is -1 .

square of their norm: $\langle \mathbf{x}, \mathbf{x} \rangle = \|\mathbf{x}\|^2$. When they are orthogonal ($\langle \mathbf{x}, \mathbf{y} \rangle = 0$), they share nothing along each other's direction.

The inner product is *global*. Every sample of $\mathbf{x}$ is multiplied with every corresponding sample of $\mathbf{y}$, and the products are summed into a single number. No spatial or temporal locality is preserved — the contribution of a sample at the beginning of the signal is indistinguishable, in the final sum, from the contribution of a sample at the end. This is sometimes what we want (when comparing the overall frequency content of two signals, say) and sometimes not (when looking for a specific pattern at an unknown time). When we want locality, we use convolution instead, which is the next section's subject.

A vector space equipped with an inner product (and the norm derived from it via $\|\mathbf{x}\| = \sqrt{\langle \mathbf{x}, \mathbf{x} \rangle}$) is called a *Hilbert space*. Hilbert spaces are the natural setting for analysis, synthesis, and learning. Every space we work with in this book will be a Hilbert space.

1.5 Convolution: local similarity

A signal is rarely interesting in its entirety. When we listen, we attend to local features — the attack of a note, a particular harmonic fluctuation, a pause. The mathematical operation that picks out local features is the *convolution*.

Given a signal $\mathbf{x}$ and a short pattern $\mathbf{h}$ (called the *kernel* or *filter*) of length M , the convolution $\mathbf{x} * \mathbf{h}$ is the sequence

$$(\mathbf{x} * \mathbf{h})[n] = \sum_{m=0}^{M-1} \mathbf{x}[n+m] \mathbf{h}[m]. \quad (1.3)$$

For each output position n , the kernel is aligned with the signal starting at sample n , and the inner product is taken over just those M samples. This is the local version of the inner product from the previous section: instead of computing one number that summarises the entire signal, we compute many numbers, each summarising a small neighbourhood.

Convolution sits at the centre of both signal processing and modern machine learning. In signal processing it underlies filtering, reverberation, pattern detection, and edge detection. In machine learning it is the basis of convolutional neural networks, which we build in Part II. In both fields, convolution is doing the same thing: *repeated inner products, slid along the signal*.

The kernel $\mathbf{h}$ is itself a signal — a short one, typically. Choosing the kernel is choosing what local feature to look for. A kernel that resembles a quarter-period of a sinusoid will respond strongly where the signal looks like that fragment of sinusoid; this is the seed of Fourier analysis. A kernel that resembles an edge will respond at sharp transitions; this is the seed of the Sobel filter and of the first layer of an image conv-net. Different kernels reveal different features; the right kernel to use depends on what we are trying to discover.

There is a deep theorem connecting convolution to Fourier analysis, namely that convolution in the time domain is equivalent to pointwise

A small example. If $\mathbf{h} = (1, -1)$, then $(\mathbf{x} * \mathbf{h})[n] = \mathbf{x}[n+1] - \mathbf{x}[n]$, the discrete derivative. If $\mathbf{h} = (1, 1)/2$, we get a two-point smoother. Choosing $\mathbf{h}$ chooses what features the convolution highlights. This is precisely what a convolutional neural network learns to do: its kernels are exactly such patterns, but discovered from data rather than designed.

multiplication in the frequency domain. We will state and prove it in Chapter 2 when we have set up the Fourier transform; Appendix ?? contains a fuller proof. The practical implication is that convolution can be computed quickly via the FFT, which is why convolutional networks scale.

1.6 Basis and projection

We have inner products (global similarity) and convolutions (local similarity). The next step is to use them to *decompose* a signal into pieces.

Given a set of vectors $\{\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_K\}$ in a vector space V , the *span* of the set is the collection of all linear combinations $\sum_k \alpha_k \mathbf{b}_k$ that can be formed from them. If every vector in V can be written this way — if the span of the set is all of V — and if no vector in the set is a linear combination of the others, the set is called a *basis* for V .

A basis is the minimal set of building blocks for the space. In $\mathbb{R}^N$ there are always exactly N vectors in any basis; we can pick the “natural” basis $\mathbf{e}_n$ (the n -th unit vector, equal to one at position n and zero elsewhere) or any other set of N linearly independent vectors.

The point of having a basis is that any vector $\mathbf{x} \in V$ can be written uniquely as

$$\mathbf{x} = \sum_{k=1}^N c_k \mathbf{b}_k. \quad (1.4)$$

The numbers c_k are the *coordinates* of $\mathbf{x}$ in the basis $\{\mathbf{b}_k\}$. They are what change when we change basis; the underlying vector $\mathbf{x}$ does not.

When the basis is *orthonormal* — meaning the basis vectors are mutually perpendicular and each has unit norm — there is a beautifully simple formula for the coordinates:

$$c_k = \langle \mathbf{x}, \mathbf{b}_k \rangle. \quad (1.5)$$

The coordinate c_k is just the inner product of $\mathbf{x}$ with the k -th basis vector. Combining this with (1.4),

$$\mathbf{x} = \sum_{k=1}^N \langle \mathbf{x}, \mathbf{b}_k \rangle \mathbf{b}_k. \quad (1.6)$$

This is the single most important formula in classical signal processing. It says: to reconstruct $\mathbf{x}$, compute its inner product with each basis vector (the *analysis* step), then build $\mathbf{x}$ back up by combining the basis vectors weighted by those inner products (the *synthesis* step). In an orthonormal basis, analysis and synthesis are exact inverses of one another.

We will see many examples of (1.6) in disguise. The Fourier transform is this formula with $\mathbf{b}_k$ a sinusoid of frequency k . The Discrete Wavelet Transform is this formula with $\mathbf{b}_k$ a scaled, shifted copy of a mother wavelet. Principal component analysis is this formula with $\mathbf{b}_k$ the k -th eigenvector of the data covariance. A neural network with linear activations is this formula with $\mathbf{b}_k$ a learned weight vector.

When the basis is not orthonormal, equation (1.5) still makes sense as the *projection* of $\mathbf{x}$ onto the direction of $\mathbf{b}_k$, but the projections no longer give exact coordinates — the basis vectors interfere. An orthonormal basis decouples them. This is why so many useful representations (Fourier, the standard basis, principal components) are designed to be orthonormal.

The choice of basis is the design of the representation. When we pick sinusoids, we have decided that frequency is the property we want to expose. When we pick wavelets, we have decided that local time-frequency structure matters. When we let a network learn its basis, we have decided to let the data decide. Each choice has consequences: it determines what the representation makes obvious, what it hides, and what kinds of operations are easy versus hard to express in it. Part II of this book builds the tools to make such choices; Part III explores their compositional consequences.

1.7 Analysis and synthesis as matrices

Equations (1.5) and (1.6) both become cleaner when written in matrix form. Let $\mathbf{B}$ be the matrix whose k -th row is the basis vector $\mathbf{b}_k^\top$. Then computing all the inner products $\langle \mathbf{x}, \mathbf{b}_k \rangle$ at once is the same as multiplying $\mathbf{x}$ by $\mathbf{B}$:

$$\mathbf{c} = \mathbf{B} \mathbf{x} \quad (\text{analysis}). \quad (1.7)$$

The vector $\mathbf{c}$ contains all the coordinates c_k . Equivalently written, $\mathbf{c} = \Phi(\mathbf{x})$ where Φ is the *analysis operator*.

For the synthesis direction, if the basis is orthonormal, the inverse of $\mathbf{B}$ is its transpose (or, in the complex case, its Hermitian adjoint $\mathbf{B}^H$). Multiplying (1.7) on the left by $\mathbf{B}^H$ recovers $\mathbf{x}$:

$$\mathbf{x} = \mathbf{B}^H \mathbf{c} = \mathbf{B}^H \mathbf{B} \mathbf{x} \quad (\text{synthesis}). \quad (1.8)$$

For non-orthonormal but still invertible $\mathbf{B}$, the synthesis uses $\mathbf{B}^{-1}$ rather than $\mathbf{B}^H$.

So the entire analysis-synthesis cycle is just a pair of matrix-vector multiplications. This is the bridge to the rest of the book. The Fourier transform is exactly this with one specific choice of $\mathbf{B}$ (a matrix of complex sinusoids). The DWT is this with a different specific choice (a matrix of wavelets at multiple scales). A linear neural-network layer is this with $\mathbf{B}$ *learned from data* instead of designed in advance.

1.8 From basis to representation

The basis idea generalises in three important ways, and the generalisations are what make modern representations work.

Relaxing linear independence. In a classical basis, we insist that no basis vector is a linear combination of the others. This is what makes the representation *unique*: every $\mathbf{x}$ has exactly one set of coordinates. But uniqueness is not always what we want. Sometimes we want a redundant set of vectors — more than N of them in an N -dimensional space — because the redundancy can encode information about the signal that a minimal basis cannot. Such sets are called *frames* or *dictionaries*. Matching pursuit, which we will meet in Part III, is a method for choosing a sparse subset of a dictionary to represent a signal.

Allowing dimensionality change. In a classical basis, the matrix $\mathbf{B}$ is square: N basis vectors in an N -dimensional space. If we allow $\mathbf{B}$ to be rectangular — say $K \times N$ with $K < N$ — we get a *projection onto a subspace*. The output has fewer dimensions than the input. This is dimensionality reduction, and it is what PCA, the encoder of an autoencoder, and the early layers of a classifier all do. The inverse direction — N to K with $K > N$ — is dimensionality expansion, which is what the decoder of an autoencoder does, and what generative models do when they turn a small latent vector into a full image or audio file.

Nonlinearity. A pure projection $\mathbf{x} \mapsto \mathbf{B}\mathbf{x}$ is a linear operation; composing two linear operations gives another linear operation, so *stacking* pure projections gives nothing new — any depth of linear layers collapses to a single equivalent layer. To get a deep representation that is more expressive than a shallow one, we need to insert a nonlinearity between the projections. In modern neural networks, this is the ReLU function $\max(0, \cdot)$, the sigmoid σ , or any of a small zoo of others. In the Scattering Transform (Chapter 2), it is the absolute value $|\cdot|$ that plays this role.

Putting all three together — redundant dictionaries, varying dimensionality, nonlinear stages — gives the basic shape of a modern representation:

$$\mathbf{z}^{(k)} = f_k(\mathbf{B}_k \mathbf{z}^{(k-1)} + \mathbf{b}_k), \quad k = 1, 2, \dots, L, \quad (1.9)$$

with $\mathbf{z}^{(0)} = \mathbf{x}$ the input, $\mathbf{B}_k$ the projection matrix of the k -th layer, $\mathbf{b}_k$ a bias vector (allowing the layer to be *affine* rather than purely linear, which lets the representation work with non-zero-centred data), and f_k the nonlinearity.

Equation (1.9) is the structural backbone of essentially every neural network in this book. Chapter 3 discusses how the $\mathbf{B}_k$ and $\mathbf{b}_k$ are *learned* from data by gradient descent on a loss function. In Part II we will build the code that does this.

Encoders and decoders. When the layers reduce dimensionality ($\mathbf{B}_k$ progressively narrower) and end in a small latent vector, we call the whole stack an *encoder*. Reverse it — layers that progressively expand dimensionality from a latent vector back to a full signal — and we have a *decoder*. An autoencoder is an encoder followed by a decoder, trained so that the output reconstructs the input.

1.9 What we have so far, and what comes next

We have set up the conceptual frame:

- Signals are vectors in a finite-dimensional Hilbert space.
- The inner product measures global similarity between vectors.
- Convolution is the inner product applied locally, kernel-by-kernel.
- Representations are projections onto a chosen basis (or dictionary), and reconstruction is the reverse operation.
- The choice of basis is the design of the representation; it determines what is made obvious and what is hidden.
- Layered representations stack projections separated by nonlinearities, allowing them to express far more than a single linear projection can.

The next two chapters give specific instances. Chapter 2 treats *a-priori* representations — those whose basis is designed in advance — with the Fourier transform, the wavelet transform, and the Scattering Transform as our three main examples. Chapter 3 treats *learned* representations —

those whose basis is derived from data — with principal component analysis, k -means clustering, matching pursuit, and neural networks. Chapter ?? closes Part I by working a single concrete problem — musical style transfer — through each of these representations in turn, showing how the choice changes what is possible. This sets up the technical work of Part II, where we build the code, and the compositional work of Part III, where we use it.

Music is the pleasure the human mind
experiences from counting without being
aware that it is counting.

Gottfried Wilhelm Leibniz

2.1 The question and three answers

A sustained violin note arriving at our ears is a pressure wave: a long sequence of small fluctuations around atmospheric pressure, sampled by the eardrum at something like 40 000 times per second. As a mathematical object it is a vector in $\mathbb{R}^N$ with N very large. As a listening experience, it is a single sustained tone with a particular timbre, attack, and decay.

The gap between those two descriptions is what a *representation* is supposed to close. We want a transformation Φ such that $\Phi(\mathbf{x})$ exposes the structure we care about — pitch, brightness, attack, the texture of the bowing — and ideally hides what we don't. The previous chapter argued that any reasonable Φ looks like an inner product with a chosen set of reference vectors. This chapter looks at three specific choices, each historically and practically important. All three are *a-priori* in the sense that the reference vectors are designed mathematically, not learned from data. The next chapter will turn to representations whose reference vectors come from data.

The three a-priori representations we consider are:

- ▶ The *Discrete Fourier Transform* (Section 2.2), whose reference vectors are pure sinusoids. Fourier exposes frequency content with maximum sharpness and is the workhorse of classical signal processing.
- ▶ The *Discrete Wavelet Transform* (Section 2.3), whose reference vectors are localized oscillations — a single *mother wavelet*, scaled and translated to cover the time-frequency plane. Wavelets give us frequency information with time localisation, at the cost of some spectral precision.
- ▶ The *Scattering Transform* (Section 2.5), a multi-layer cascade of wavelet convolutions and absolute-value nonlinearities. It is the bridge between classical signal processing and modern convolutional neural networks: structurally identical to a CNN, but with all the filters fixed by mathematics rather than learned from data.

The progression Fourier $\rightarrow$ Wavelet $\rightarrow$ Scattering will turn out to add, one step at a time, three things: *locality* (wavelets are localised, sinusoids are not), *depth* (Scattering stacks two or three wavelet transforms), and *nonlinearity* (Scattering inserts an absolute value between the layers). Once we have these three, a fourth step — replacing the predefined filters with *learned* ones — gets us a convolutional network. That fourth step is the subject of the next chapter.

2.2 The Discrete Fourier Transform

The Discrete Fourier Transform represents a length- N signal as a sum of N complex sinusoids. For each frequency index $k \in \{0, 1, \dots, N-1\}$,

$$\hat{\mathbf{x}}[k] = \sum_{n=0}^{N-1} \mathbf{x}[n] e^{-i2\pi kn/N}. \quad (2.1)$$

The inverse transform reconstructs $\mathbf{x}$ from $\hat{\mathbf{x}}$ via the same formula with the opposite sign in the exponent and a $1/N$ normalisation:

$$\mathbf{x}[n] = \frac{1}{N} \sum_{k=0}^{N-1} \hat{\mathbf{x}}[k] e^{i2\pi kn/N}. \quad (2.2)$$

The DFT in (2.1) would naively cost $O(N^2)$ operations to evaluate. The *Fast Fourier Transform* reduces this to $O(N \log N)$ by exploiting the recursive structure of the sum. The FFT is what makes Fourier analysis practical for audio; we use it extensively in Part II.

The DFT as projection. Compare (2.1) with the inner-product formula (1.2) from the previous chapter. The DFT is just the inner product of $\mathbf{x}$ with the k -th complex sinusoid:

$$\hat{\mathbf{x}}[k] = \langle \mathbf{x}, \mathbf{f}_k \rangle, \quad \mathbf{f}_k[n] = e^{i2\pi kn/N}. \quad (2.3)$$

Each $\mathbf{f}_k$ is one of our reference vectors: a discrete sinusoid that completes exactly k full cycles over the length of the signal. The set $\{\mathbf{f}_k\}_{k=0}^{N-1}$ is an *orthogonal basis* for $\mathbb{C}^N$, and in fact an orthonormal one after the $1/N$ scaling in (2.2). This is why the inverse transform exists in closed form: an orthogonal basis is its own decoder.

In matrix form, letting $\mathbf{F}$ be the $N \times N$ matrix with $\mathbf{F}_{kn} = e^{-i2\pi kn/N}$,

$$\hat{\mathbf{x}} = \mathbf{F} \mathbf{x}, \quad \mathbf{x} = \frac{1}{N} \mathbf{F}^H \hat{\mathbf{x}}. \quad (2.4)$$

The Fourier transform is, exactly, a particular choice of $\mathbf{B}$ in the framework of the previous chapter.

What the DFT exposes. Because each basis vector $\mathbf{f}_k$ is a pure tone of frequency k , the coefficient $\hat{\mathbf{x}}[k]$ is large precisely when $\mathbf{x}$ contains energy at that frequency. The magnitude $|\hat{\mathbf{x}}[k]|$ is the amount of frequency k in the signal; the argument $\arg \hat{\mathbf{x}}[k]$ is its phase. For musical signals — a sustained pitched tone — the DFT magnitude shows a peak at the fundamental and at each harmonic, telling us immediately how loud each harmonic is. This is the basis of pitch detection, spectral envelopes, and timbre analysis.

What the DFT hides. Two properties of (2.3) matter. First, the sum runs over the *entire* signal: every sample contributes to every coefficient. A short click at $n = 100$ contributes to $\hat{\mathbf{x}}[k]$ for every k . The DFT has no notion of “when” — it gives a global summary of frequency content, averaged over all of time. Second, the sinusoids $\mathbf{f}_k$ extend unattenuated across the whole signal length; they are global functions masquerading as basis vectors.

This is fine for stationary signals (an organ note holding constant for seconds at a time) but disastrous for transient signals (a snare hit, a

consonant in speech, the attack of a piano note). For such signals we want *locality*: a representation that tells us not just what frequencies are present but *when*.

Translation invariance and frequency sensitivity. The DFT has one beautiful invariance and one painful sensitivity.

If we shift $\mathbf{x}$ in time by Δ , the DFT magnitudes are unchanged — only the phases change:

$$\mathbf{x}[n - \Delta] \longleftrightarrow \hat{\mathbf{x}}[k] e^{-i2\pi k\Delta/N}. \quad (2.5)$$

This is what we want: a violin note arriving slightly later sounds the same. The magnitude spectrum is *translation-invariant*.

But if we shift $\mathbf{x}$ in *frequency* by even a small amount — say, the violin is slightly mistuned, or our sample rate drifts — the DFT coefficients can move around dramatically. A pure tone at frequency $k + 0.5$ smears across many bins because it doesn't match any single basis vector. This is the well-known *leakage* phenomenon, and it's the fundamental cost of using strictly periodic basis vectors on signals that are not strictly periodic.

Convolution becomes multiplication. The Fourier transform's most useful algebraic property, far beyond its role as a representation, is that convolution in the time domain corresponds to pointwise multiplication in the frequency domain:

$$\mathbf{y} = \mathbf{x} * \mathbf{h} \iff \hat{\mathbf{y}}[k] = \hat{\mathbf{x}}[k] \cdot \hat{\mathbf{h}}[k]. \quad (2.6)$$

This is the *convolution theorem*; we sketch a proof in Appendix ???. The practical consequence is that a convolution that would naively cost $O(NM)$ to evaluate directly costs $O(N \log N)$ via three FFTs (forward, multiply, inverse). For large kernels, the FFT-based convolution is by far the faster method.

In the other direction, multiplication of two signals corresponds to *convolution* of their spectra. This is how the Short-Time Fourier Transform works (next), why amplitude modulation in audio creates sidebands, and why the spectral analysis of a windowed signal shows the window's spectrum convolved with the signal's spectrum.

A continuous version of the leakage problem is the *Heisenberg uncertainty principle for signals*: a representation cannot have arbitrarily sharp resolution in both time and frequency. The DFT chooses maximum frequency resolution (sharp basis vectors in frequency) and accepts zero time resolution (global basis vectors in time). Wavelets choose a different trade-off; the scattering transform yet another.

2.3 The Discrete Wavelet Transform

The DFT gives us frequency at the cost of time. The Wavelet Transform restores time by giving up some frequency.

Mother wavelets and their family. Pick a *mother wavelet* ψ : a short, oscillatory, zero-mean function of finite energy. Generate a family of basis vectors by scaling and translating it:

$$\psi_{a,b}[n] = \frac{1}{\sqrt{|a|}} \psi\left(\frac{n-b}{a}\right). \quad (2.7)$$

The scale parameter a stretches or compresses the wavelet: a large a produces a wide, low-frequency wavelet; a small a produces a narrow, high-frequency one. The translation parameter b shifts the wavelet along the time axis. The normalisation $1/\sqrt{|a|}$ keeps the energy constant across scales.

The Discrete Wavelet Transform of a signal $\mathbf{x}$ is the array of inner products of $\mathbf{x}$ with this family:

$$\mathbf{w}[a, b] = \langle \mathbf{x}, \psi_{a,b} \rangle. \quad (2.8)$$

The coefficient $\mathbf{w}[a, b]$ tells us how much of the signal looks like a wavelet of scale a centred at time b . Large coefficients mark places in time-frequency where the signal has structure at that particular scale.

Common mother wavelets include the *Haar* (a square wave: simplest, discontinuous), *Daubechies* families (compactly supported, orthogonal, smoother), the *Mexican hat* (a Gaussian's second derivative), and the *Morlet* (a complex sinusoid times a Gaussian, much used in time-frequency audio analysis). The choice of mother wavelet matters in practice; for this book the Morlet is the most relevant.

The time-frequency plane. Where the DFT gives one coefficient per frequency, the DWT gives one coefficient per *location in time-frequency*. Plot a DWT as a heatmap with time on the horizontal axis and scale on the vertical axis, and you see a picture: which frequencies are active when. This is exactly the *spectrogram-like* visualisation that audio engineers use to look at sounds. In Part II we build a related construction — the Short-Time Fourier Transform — which gives a similar picture using DFTs of short windowed segments. Wavelets are the more principled relative, with proper mathematical foundations rather than ad-hoc windowing.

What the wavelet representation costs. We gain time localisation, but we pay for it. Two specific costs.

First, the representation is no longer translation-invariant in magnitude. Shifting $\mathbf{x}$ in time *does* change which wavelet coefficients are large, because the coefficients are indexed by both scale and position. A wavelet representation is *translation-equivariant*: the pattern of coefficients shifts in lockstep with the signal, but the shifted pattern is not the same pattern.

Second, the frequency resolution at any single scale is lower than the DFT's. Each wavelet covers a range of frequencies rather than a single one; the trade is wider time coverage in exchange for narrower frequency coverage. This is unavoidable: tight in time forces broad in frequency, and vice versa.

Stability to frequency drift. In exchange, the DWT is much more stable than the DFT to small drifts in frequency. A pure tone at frequency $f_0 + \delta$ produces wavelet coefficients close to those of a pure tone at f_0 , because each wavelet's frequency response is already broad. This makes wavelets the better tool for analysing real-world signals whose frequencies wobble or evolve — vibrato, glissando, the slowly drifting pitch of a struck bell.

Matrix form. Just as with the DFT, the DWT has a matrix representation. Let Ψ be the matrix whose rows are the wavelets $\psi_{a,b}$ at every (a, b) pair in our chosen discretisation. Then

$$\mathbf{w} = \Psi \mathbf{x}, \quad \mathbf{x} = \Psi^H \mathbf{w} \quad (\text{if the wavelets form an orthonormal basis}). \quad (2.9)$$

We have the same analysis-synthesis structure as the DFT, with a different choice of basis. The framework is invariant; the basis is the design choice.

2.4 Between transforms: the time-frequency trade

Before we get to the Scattering Transform, it is worth pausing on the relationship between the DFT and the DWT, because it makes the design space visible.

The DFT picks one extreme of the time-frequency trade: maximally sharp in frequency, maximally blurry in time. Every basis vector spans the whole signal, so we know exactly which frequencies are present but nothing about when.

The Short-Time Fourier Transform (STFT), which we build in Part II, picks a middle point: we cut the signal into short overlapping windows, take the DFT of each, and stack the results. Each window has fixed time width (call it W), so every frequency bin has the same time resolution W and the same frequency resolution $\sim 1/W$. Audio engineers reach for the STFT constantly; it is the basis of the spectrogram.

The DWT picks a different middle point, one where the time-frequency resolution varies with scale. High frequencies (small a) get fine time resolution and coarse frequency resolution — a clear “the transient happened here” answer. Low frequencies (large a) get coarse time resolution and fine frequency resolution — a clear “the pitch is f_0 ” answer over a longer window. This is exactly the right behaviour for natural signals: short events tend to be broadband, long events tend to be narrowband.

The progression is from rigid (DFT) to adaptive (DWT) to multi-layered (Scattering, next). Each step gives up some elegance — the DFT has a closed-form inverse; the DWT requires careful choice of mother wavelet; the Scattering Transform has multiple stages and a fixed but non-trivial schedule of operations — in exchange for better behaviour on real signals.

The DWT’s variable resolution is sometimes called *constant-Q analysis*: each band has a fixed ratio of frequency width to centre frequency. This matches the way the human auditory system processes sound, where critical bands also have approximately constant Q . It is not a coincidence that wavelet-like decompositions appear in the cochlea.

2.5 The Scattering Transform

The Scattering Transform, introduced by Mallat in the early 2010s (*Group invariant scattering*, Communications on Pure and Applied Mathematics, 2012), builds on the DWT in two crucial ways: it adds a nonlinearity (the absolute value) between successive wavelet decompositions, and it stacks several of them. The resulting structure is, mathematically and conceptually, a convolutional neural network with all the filters fixed by hand.

One layer. We start with a signal $\mathbf{x}$ and a family of wavelets $\{\psi_{\lambda_1}\}$ indexed by scale (we collapse the notation $\psi_{a,b}$ into ψ_{λ} where λ ranges over both scale and shift). The first-layer scattering coefficients are

$$S_1\mathbf{x} = |\mathbf{x} * \psi_{\lambda_1}|. \quad (2.10)$$

Compute the wavelet transform of $\mathbf{x}$, take the absolute value of each coefficient. This is the only operation; nothing learned, nothing adjustable.

Two layers. The output $S_1\mathbf{x}$ is itself a signal — in fact, a collection of signals, one per scale. We can apply the same operation to each of them, using a second family of wavelets $\{\psi_{\lambda_2}\}$:

$$S_2\mathbf{x} = ||\mathbf{x} * \psi_{\lambda_1}| * \psi_{\lambda_2}|. \quad (2.11)$$

This is a wavelet transform applied to the magnitude of a wavelet transform. The result captures *interactions* between frequency bands: how does activity at scale λ_1 vary over time? How does that variation itself look at scale λ_2 ?

The transform can be extended to three or more layers by repeating the recipe. In practice, two or three layers suffice; later layers tend to capture diminishing amounts of energy.

Why the absolute value. Without the nonlinearity between the two wavelet transforms, equation (2.11) would collapse into a single wavelet transform (because the composition of two linear operators is a linear operator). The absolute value prevents this collapse: it is the *nonlinearity* that makes a deep representation more expressive than a shallow one.

The choice of absolute value, specifically, is not arbitrary. It is the simplest nonlinearity that is *stable* (Lipschitz: small changes in input produce only small changes in output) and that removes the phase information (which is highly oscillatory in time and would otherwise destabilise the second layer). Other nonlinearities work too, but the absolute value has the cleanest theoretical properties.

Stability and invariance. The scattering transform inherits two key properties from its construction:

- It is *stable to deformations*: if $\mathbf{x}$ is replaced by a slightly time-warped version $\mathbf{x} \circ \tau$ where τ is a smooth deformation, the scattering coefficients $S\mathbf{x}$ change by an amount controlled by the smoothness of τ , not by the size of the deformation itself. This is the right notion of robustness for natural signals, which are constantly subject to small distortions (recording-room acoustics, microphone characteristics, breath fluctuations).
- It is *translation-invariant at large scale*: when the final layer is averaged over a window much wider than the signal's features, the result depends on $\mathbf{x}$'s content but not on where in time that content occurs. This is what we want for classification (a violin is a violin whether it arrives at sample 1000 or 2000).

Why this matters. Bruna and Mallat showed in 2013 (*Invariant scattering convolution networks*, IEEE TPAMI) that a two-layer scattering transform achieves near-state-of-the-art accuracy on some image and audio classification tasks, *without any learning*. The features it produces are good enough that a simple linear classifier on top reaches accuracy that

The architecture. Look at the structure of the scattering transform from a distance. Each layer convolves the input with a bank of filters, applies a nonlinearity, and passes the result to the next layer. Strip the words “wavelet” and “absolute value” and replace them with “learned filter” and “ReLU” and you have, exactly, a convolutional neural network. Wavelet convolution becomes $\mathbf{W} * \mathbf{x}$; the absolute value becomes ReLU; the cascade of layers is the same.

This is the bridge from classical signal processing to modern deep learning. The scattering transform is the *minimum* deep network: filters chosen for theoretical optimality (translation invariance, locality, stability), nonlinearity chosen for stability. A CNN *replaces* the principled-but-rigid wavelets with filters that the data adjusts, trading mathematical guarantees for flexibility. Some problems do not benefit from that trade — and for those, scattering is competitive. Most problems do — and for those, we need learning.

Hierarchical features. The scattering transform also makes explicit a structural point about *depth*. The first-layer coefficients $|\mathbf{x} * \psi_{\lambda_1}|$ are very similar to a spectrogram: they tell us about frequency content over time. The second-layer coefficients capture how that frequency content is itself modulated. Energy that fluctuates rapidly at the first layer becomes visible as a clear peak at a particular λ_2 in the second layer. This is the modulation spectrum, central in audio analysis.

We will see the same hierarchical structure in CNNs: first layer detects edges; second layer detects combinations of edges; third layer detects parts; eventually the network detects objects. The CNN’s hierarchy is learned; the scattering transform’s is mathematical. They are, structurally, the same hierarchy.

2.6 Closing thoughts

We have walked through three a-priori representations: the DFT (maximum frequency resolution, no time resolution), the DWT (a flexible time-frequency trade-off), and the Scattering Transform (multi-layer, locally invariant, deformation-stable). Each is a specific choice of $\Phi(\mathbf{x}) = f(\mathbf{B}\mathbf{x})$ in the framework of the previous chapter, with $\mathbf{B}$ a fixed matrix designed by mathematicians and f either the identity, the magnitude $|\cdot|$, or a composition of those.

The three transforms answer different questions about a signal. The DFT answers “what frequencies are present?” The DWT answers “what frequencies are present, and when?” The Scattering Transform answers “what frequencies are present, when, and how do they interact?” Each successive question is more demanding; each requires more structure in the representation. The progression suggests a natural fourth step: “what features are present, and we don’t even know what features we mean?” When we don’t know what kinds of patterns to look for, we let the data tell us. That is the subject of the next chapter — representations whose reference vectors are *learned*.

The progression also previews the architectural pattern we will re-encounter many times: *projection, nonlinearity, stacking*. These three together make depth meaningful. Without projection there is nothing to compose; without nonlinearity composition collapses; without stacking there is no hierarchy. The Scattering Transform demonstrates that all three can be in place without any learning at all. The point of learning, as we will see, is to let the projections themselves adapt to whatever signals we throw at the network — to choose **B** from data rather than from a mathematician's foresight.

Learned representations

3

All models are wrong, but some are useful.

George E. P. Box

3.1 When the basis comes from data

The previous chapter built three representations whose reference vectors — sinusoids for Fourier, scaled and shifted wavelets for the DWT, products of those for Scattering — were chosen by mathematicians in advance. This works beautifully when we know what property we want to expose. We want pitch, so we project onto sinusoids. We want time-frequency structure, so we project onto wavelets. The chosen reference vectors carry our prior knowledge about the signal.

But what do we do when we don't know what to look for? When we have a million labeled images of cats and dogs and want a representation that distinguishes them, we have no idea what mathematical function picks out the relevant cat-vs-dog features. When we have orchestral recordings and want a representation that captures a composer's style, we cannot write down by hand what reference vectors would expose "Mahler-ness." The features are real but they are not in any catalogue we know how to consult.

The natural move is to *let the data choose the basis*. Rather than designing the reference vectors mathematically, we look at many examples of signals and derive a set of vectors that, in some sense, captures what is common or what is distinguishing. This is what *learned representations* do. The basis is no longer in a textbook; it lives in a parameter array computed from data.

This chapter introduces four learned representations of increasing sophistication. Principal Component Analysis (Section 3.2) finds the directions of greatest variance in a dataset. *K*-means (Section 3.3) finds prototypes that group similar examples. Matching pursuit (Section 3.4) builds a sparse, interpretable approximation of a signal from a large dictionary of candidates. Neural networks (Section 3.5) learn nonlinear, multi-layered representations whose components are themselves learned-from-data projections.

In all four cases, the underlying structure is the same as in the previous chapter: project the signal onto reference vectors, possibly apply a non-linearity, possibly stack layers. The only difference is where the reference vectors come from.

The bias-variance trade-off. Predefined bases are *low-variance*: they don't depend on which data you fed them, so they don't overfit. They're also *high-bias*: if your signal doesn't decompose well into the chosen basis, no amount of clever analysis will help. Learned bases reverse the trade-off. This is the classical bias-variance picture, made concrete.

3.2 Principal Component Analysis

PCA is the simplest learned representation: a linear projection whose basis vectors are the directions of maximum variance in the data.

Given a data matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$ with one example per row (so N examples of dimension d), PCA proceeds in three steps.

1. Centre the data. Subtract the per-feature mean so that the columns of $\mathbf{X}$ have zero mean. This is essential because the geometric machinery of PCA assumes the data is centred at the origin.

2. Compute the covariance. Form

$$\mathbf{C} = \frac{1}{N} \mathbf{X}^\top \mathbf{X}. \quad (3.1)$$

The (i, j) entry of $\mathbf{C}$ is the covariance between feature i and feature j .

3. Diagonalise. Find the eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d \geq 0$ and corresponding eigenvectors $\mathbf{e}_1, \dots, \mathbf{e}_d$ of $\mathbf{C}$. These eigenvectors are the *principal components* — the directions in feature space along which the data has the most variance. The first eigenvector $\mathbf{e}_1$ is the single direction along which the data spreads most; $\mathbf{e}_2$ is the direction of next greatest spread perpendicular to $\mathbf{e}_1$; and so on.

To get a k -dimensional representation we project onto the first k eigenvectors. Let $\mathbf{E}_k \in \mathbb{R}^{d \times k}$ be the matrix whose columns are $\mathbf{e}_1, \dots, \mathbf{e}_k$. Then

$$\mathbf{Z} = \mathbf{X} \mathbf{E}_k \quad (3.2)$$

is the data expressed in the k -dimensional principal-component basis.

PCA is mathematically a singular value decomposition (SVD) of the data matrix $\mathbf{X}$ in disguise: the principal components are the right singular vectors of $\mathbf{X}$. When we work with PCA in Part II, the SVD is the practical computation.

PCA in the framework. Compare $\mathbf{Z} = \mathbf{X} \mathbf{E}_k$ with the analysis formula $\mathbf{c} = \mathbf{B} \mathbf{x}$ from the previous chapter. PCA is exactly that, with $\mathbf{B} = \mathbf{E}_k^\top$ (the principal-component matrix) and the data being analysed simultaneously rather than one example at a time. The reference vectors are the eigenvectors. The only thing that distinguishes PCA from the Fourier transform, at the level of mathematical structure, is that the basis vectors come from the data instead of from $e^{i2\pi kn/N}$.

What PCA captures. PCA is a tool for finding low-dimensional linear structure in high-dimensional data. When applied to natural images, its principal components often look like blurry blobs and gradients — the most-variance directions in the space of 32×32 RGB images. When applied to speech features, the principal components capture coarse vowel-vs-consonant structure. When applied to text embeddings, they capture broad semantic axes (topic, sentiment).

What PCA cannot capture is nonlinear structure. The data may lie on a curved manifold in feature space; PCA will fit it with a flat hyperplane and lose all the curvature. For nonlinear structure we need either to engineer nonlinear features by hand (a hard problem) or to learn them with a network (the rest of this chapter).

3.3 *K-means and prototypes*

K-means takes a different geometric view. Rather than finding directions of variance, it finds *prototypes*: a small set of representative points such that every other data point is close to one of them.

The algorithm. Given data points $\mathbf{x}_1, \dots, \mathbf{x}_N$ and a chosen number of clusters K :

1. Initialise K prototype vectors $\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K$ (often by randomly picking K of the data points).
2. *Assign* each data point $\mathbf{x}_n$ to the nearest prototype:

$$C_n = \arg \min_k \|\mathbf{x}_n - \boldsymbol{\mu}_k\|.$$

3. *Update* each prototype to the mean of the data points assigned to it:

$$\boldsymbol{\mu}_k = \frac{1}{|\{n : C_n = k\}|} \sum_{n: C_n=k} \mathbf{x}_n.$$

4. Repeat steps 2–3 until the assignments stop changing.

K-means in the framework. *K-means* looks like a different kind of object from PCA — a discrete clustering rather than a linear projection. But viewed through our framework it is still a representation by reference vectors: every signal $\mathbf{x}$ is summarised by the index $C(\mathbf{x})$ of its nearest prototype, and the prototypes $\{\boldsymbol{\mu}_k\}$ are the reference vectors. The “projection” is the assignment operation, which is *not* a linear map but a piecewise-constant function of $\mathbf{x}$. This is our first nonlinear representation, but the nonlinearity is hidden in the *arg-min*.

A useful matrix formulation: let $\mathbf{M} \in \mathbb{R}^{K \times d}$ be the matrix whose rows are the prototypes, and let $\mathbf{C} \in \mathbb{R}^{N \times K}$ be the binary assignment matrix with $C_{nk} = 1$ if example n is in cluster k and zero otherwise. Then the reconstruction $\hat{\mathbf{X}} = \mathbf{C}\mathbf{M}$ replaces each data point with its assigned prototype, and *K-means* minimises the reconstruction error $\|\mathbf{X} - \mathbf{C}\mathbf{M}\|_F^2$. This is structurally an extreme dimensionality reduction — each example is reduced to one integer in $\{1, \dots, K\}$.

K-means and PCA are sometimes presented as alternatives, but they answer different questions. PCA: “in what direction does the data spread the most?” *K-means*: “what are the typical examples?” A composer’s archive analysed with PCA might give you a few axes of stylistic variation. Analysed with *K-means*, it might give you a handful of representative pieces. Both are useful; the right tool depends on what you want to do.

3.4 Matching pursuit and sparse dictionaries

The two representations so far — PCA and *K-means* — both insist on a small basis: k eigenvectors or K prototypes. Matching pursuit takes the opposite tack: provide a *large* dictionary of candidate reference vectors, and let the analysis algorithm choose a small subset of them per signal. This is the right idea when no fixed-size basis would do justice to the variety of signals we want to represent.

Setup. Let $\mathbf{D} \in \mathbb{R}^{d \times M}$ be a *dictionary*: a matrix whose M columns $\mathbf{d}_1, \dots, \mathbf{d}_M$ are candidate reference vectors (atoms). We allow M to be much larger than d ; the dictionary is *overcomplete*. Atoms could be Gabor wavelets at many scales and frequencies, samples from a corpus of audio recordings, or anything else relevant.

The goal is to approximate a signal $\mathbf{x}$ as a sparse linear combination of atoms:

$$\mathbf{x} \approx \sum_{k=1}^K \alpha_k \mathbf{d}_{i_k}, \quad K \ll M. \quad (3.3)$$

We want to find which K atoms (the indices $i_1, \dots, i_K$) and which coefficients $\alpha_1, \dots, \alpha_K$ minimise the approximation error. Choosing the best subset of K atoms out of M is combinatorially explosive — there are $\binom{M}{K}$ subsets to consider. In practice we use a *greedy* algorithm.

The matching pursuit algorithm. At each step we pick the atom that explains the largest fraction of what remains.

1. Start with residual $\mathbf{r}_0 = \mathbf{x}$.
2. For $k = 1, 2, \dots, K$:
 - a) Find the atom most correlated with the current residual:

$$i_k = \arg \max_i |\langle \mathbf{r}_{k-1}, \mathbf{d}_i \rangle|.$$

- b) Compute its coefficient: $\alpha_k = \langle \mathbf{r}_{k-1}, \mathbf{d}_{i_k} \rangle$ (assuming dictionary atoms are unit-norm).
 - c) Subtract the projection: $\mathbf{r}_k = \mathbf{r}_{k-1} - \alpha_k \mathbf{d}_{i_k}$.
3. Stop when $\|\mathbf{r}_K\|$ is small enough, or when K reaches a budget.

The algorithm produces an approximation $\hat{\mathbf{x}} = \sum_k \alpha_k \mathbf{d}_{i_k}$ that uses only K of the M available atoms.

Why this is a representation. Matching pursuit produces, for each input signal $\mathbf{x}$, a sparse list of (atom, coefficient) pairs. This list *is* the representation. Unlike a dense Fourier spectrum or a flat PCA projection, matching pursuit returns only those components that are large; the rest are implicit zeros. The representation is naturally *interpretable*: each atom is a specific pattern, and the coefficient tells you how much of that pattern is in the signal.

Matching pursuit in the framework. Matching pursuit is a representation by reference vectors — the atoms — but the “projection” is no longer a single matrix multiplication. It is the output of a small algorithm (the greedy selection loop). The basis is also no longer a basis in the strict sense: the dictionary is redundant, and we deliberately use only a few atoms per signal.

This breaks the clean “matrix in, matrix out” picture of the previous chapter, but it gains two things in exchange. First, *sparsity*: the representation has very few non-zero components, which is exactly what we often want for compression and for interpretation. Second, *adaptivity*: each signal gets its own custom subset of atoms, matched to its content, rather than being squeezed through the same projection as every other signal.

Matching pursuit was introduced by Mallat and Zhang in 1993. Its descendants include orthogonal matching pursuit (OMP), basis pursuit (which solves a related L^1 minimisation problem), and the broader field of sparse coding and compressed sensing. When the dictionary itself is also learned from data, the algorithm is called *dictionary learning* or *sparse coding*; this borders on unsupervised neural-network methods.

Why matching pursuit matters here. We will meet matching pursuit again in Part III, where it plays the role of a learned-but-not-neural codec: a way to decompose orchestral sounds into a sequence of atom-indices that captures their timbre and structure. These atom-indices then become the input to a transformer model that learns to generate music. In this sense matching pursuit gives us the best of both worlds: an interpretable codec like Fourier, and a learned basis like a neural network.

This pattern — combine an a-priori transform with a learned model on its output — is increasingly common. It is the basis of AudioLM, MusicLM, and several other recent music-generation systems. We will build our own version in Part III.

3.5 Neural networks

We come to the modern centre of mass. A *neural network* is a representation built from many layers, each of which is a learned linear projection followed by a nonlinearity.

A single layer. The basic building block, repeated many times, is

$$\mathbf{z} = f(\mathbf{W}\mathbf{x} + \mathbf{b}). \quad (3.4)$$

Here $\mathbf{W}$ is a learned matrix of weights, $\mathbf{b}$ is a learned bias vector that allows the projection to be affine rather than purely linear, and f is a fixed nonlinearity applied to each component (most commonly ReLU, $f(x) = \max(0, x)$).

The inner product structure is right there in $\mathbf{W}\mathbf{x}$. Each row of $\mathbf{W}$ is a reference vector $\mathbf{w}_i$, and the i -th component of the output is

$$z_i = f(\langle \mathbf{w}_i, \mathbf{x} \rangle + b_i). \quad (3.5)$$

This is the inner-product analysis of the previous chapter, with two modifications: the basis vectors $\mathbf{w}_i$ are learned, and a nonlinearity f is applied to each result.

Logistic regression as a one-layer network. When the nonlinearity is the sigmoid $\sigma(x) = 1/(1 + e^{-x})$ and the output is interpreted as a probability, equation (3.4) is exactly *logistic regression* — the classical statistical model for binary classification. This is not a coincidence: a neural network is, layer for layer, a stack of generalised regressions, each producing features for the next.

Multiple layers. Stacking layers gives a deep network:

$$\mathbf{z}^{(L)} = f_L(\mathbf{W}_L f_{L-1}(\mathbf{W}_{L-1} \cdots f_1(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) \cdots + \mathbf{b}_{L-1}) + \mathbf{b}_L). \quad (3.6)$$

Without the nonlinearities f_i , this composition would collapse into a single linear transformation — the product $\mathbf{W}_L \cdots \mathbf{W}_1$ — giving us no more expressive power than a one-layer network. The nonlinearity is

The universal approximation theorem.

A sufficiently wide two-layer network with a non-polynomial nonlinearity can approximate any continuous function on a compact domain to any desired accuracy. This is a celebrated theoretical result, and it sounds powerful, but it understates the role of depth. A two-layer network may need exponentially many neurons to do what a deeper network does with polynomially many. Depth is not just possible; it is efficient.

what makes depth meaningful. We met this argument already in the Scattering Transform, where the absolute value played the same role.

How the weights are learned. This is the algorithmic heart of modern machine learning. Given a dataset of input-output pairs $\{(\mathbf{x}_n, \mathbf{y}_n)\}$, we define a *loss function* $\mathcal{L}$ that measures how badly the network's predictions match the targets. We then adjust the weights to reduce the loss by *gradient descent*:

$$\mathbf{W}_\ell \leftarrow \mathbf{W}_\ell - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}_\ell}, \quad (3.7)$$

with a learning rate η . Computing the gradients $\partial \mathcal{L} / \partial \mathbf{W}_\ell$ for every layer is done by *backpropagation*, which is the multivariate chain rule applied methodically from the output layer back to the input layer. Part II of this book derives backpropagation in detail, implements it in C++, and verifies it with numerical gradient checks.

Encoders and decoders. We can use depth-with-nonlinearity to build two complementary structures:

- ▶ An *encoder* starts with a high-dimensional input and produces a progressively lower-dimensional representation, distilling the input to a compact latent code. Each layer has fewer outputs than inputs.
- ▶ A *decoder* reverses the process: starts with a small latent vector and produces a high-dimensional output. Each layer has more outputs than inputs.

An *autoencoder* chains the two: encoder, latent bottleneck, decoder, trained so the output reconstructs the input. The latent bottleneck forces the network to find a compact representation that loses as little information as possible. When the encoder and decoder are both deep nonlinear networks, the autoencoder is a learned analogue of PCA — with the same goal (dimensionality reduction) but able to capture curved manifolds rather than just flat hyperplanes.

A *variational autoencoder* extends the autoencoder to a generative model: the bottleneck is interpreted as a probability distribution, and we can sample new outputs by sampling from the latent distribution and running them through the decoder. Part II builds both, and the audio VAE chapter of Part II uses one to morph between sounds.

3.6 Convolutional neural networks

A convolutional neural network applies the inner product locally, like the Scattering Transform of the previous chapter, but with the filters learned from data rather than designed by hand. The architectural template is exactly the same:

- ▶ Convolve the input with a bank of filters.
- ▶ Apply a nonlinearity.
- ▶ Possibly pool (spatial downsampling).
- ▶ Repeat for several layers.

- Finish with a few dense layers for classification.

What changes from the scattering picture is that the filters are no longer wavelets chosen for analytical convenience; they are free parameters trained jointly with the rest of the network. Each filter is a small inner product applied at every location in the input: the same inner product, slid across the signal.

Why share weights. The defining feature of a CNN compared to a fully-connected network is *weight sharing*: the same filter weights are used at every spatial position. This has three consequences. It reduces the number of parameters dramatically (a 3×3 filter on an image of any size has only 9 weights, not 9 weights per image location). It builds in *translation equivariance*: shifting the input shifts the output. And it encodes a prior of locality: each output value depends on a small local neighbourhood, capturing the fact that pixel relationships are usually nearby.

These three are not optional features; they are the architectural prior that makes CNNs work on images and audio while fully-connected networks struggle.

The hierarchy. In a multi-layer CNN, early layers learn low-level features (edges, colour patches, texture primitives), middle layers learn combinations (corners, simple shapes, repeating textures), and late layers learn task-specific concepts (object parts, faces, musical-instrument families). This hierarchical organisation emerges naturally from gradient descent on a classification loss; nothing in the training procedure explicitly asks for it.

The same hierarchical pattern appeared in the Scattering Transform: first-layer wavelet coefficients capture frequency content, second-layer coefficients capture modulations of that content. The CNN learns the same kind of hierarchy, but discovers what the relevant features *are* from the data rather than receiving them as wavelets. This is the gain from learning, and it is the gain that explains why CNNs dominate image and audio tasks in practice.

Empirically interpretable. Train a CNN on natural images and the first-layer filters — which are easy to visualise as small RGB patches — come out looking like oriented edge detectors and colour-opponent blobs. These are exactly the kinds of features that classical signal-processing theory tells us *should* be useful for natural images. Krizhevsky's AlexNet paper (2012) and Zeiler and Fergus's 2014 follow-up made this picture famous; we reproduce it in Part II's CIFAR-10 example.

3.7 The unified picture

We have now seen, across two chapters, a sequence of representations:

- **DFT**: linear projection onto sinusoids.
- **DWT**: linear projection onto scaled-shifted wavelets.
- **Scattering**: multi-layer wavelet cascade with absolute value as nonlinearity.
- **PCA**: linear projection onto eigenvectors of the data covariance.
- **K-means**: piecewise-constant assignment to learned prototypes.
- **Matching pursuit**: sparse selection from a learned (or designed) overcomplete dictionary.
- **Fully-connected network**: stacked nonlinear projections with learned dense weights.

- **Convolutional network:** stacked nonlinear projections with learned local weights, shared across spatial position.

Every one of these fits the template $\Phi(\mathbf{x}) = f(\mathbf{B}\mathbf{x})$, possibly with several layers stacked, possibly with the projection $\mathbf{B}\mathbf{x}$ replaced by a convolution. The two axes of variation are: (1) where the reference vectors come from (predefined or learned), and (2) whether locality is enforced (global inner products versus local convolutions). This gives a 2×2 table:

	<i>Global</i> (inner product)	<i>Local</i> (convolution)
<i>Designed basis</i>	DFT, PCA, <i>K</i> -means	DWT, Scattering
<i>Learned basis</i>	FC network, matching pursuit	CNN

The progression of this book is along the diagonal: from designed-global (Chapter 2) through designed-local and learned-global (this chapter) to learned-local (the CNN, and the conv-VAE we will build in Part II). Each step adds capability; each step also adds opacity, because learned representations are harder to interpret than designed ones.

The next chapter closes Part I by working a concrete problem — musical style transfer — through each cell of this table. We will see what *the same problem looks like* when posed in the waveform (Fourier), in the spectrum (DFT), in the wavelet domain (DWT), via sparse atoms (matching pursuit), and via a learned latent space (autoencoder). The exercise will make vivid how the choice of representation determines what is musically possible.

TECHNOLOGY

Why C++?

4

If you have any background in modern machine learning, the obvious question on opening this book is: PyTorch exists, NumPy exists, the whole field has consolidated around Python — why is this library in C++? The answer is not “C++ is better than Python.” It isn’t, in general. The answer is that **C++ is better than Python for what this book is trying to teach**. The case is worth making honestly, because if you don’t buy it you’ll spend the whole book wondering why we’re not using PyTorch, and that’s a distraction from the actual material.

4.1 What the book is for

Two goals, which point at the same language:

1. To teach how every piece of a neural network actually works — the matrix algebra, the gradients, the optimizers, the convolutions, the attention mechanism — at the level of someone who could implement each piece themselves, with a pencil if they had to.
2. To build a foundation that’s directly usable for deploying neural networks in audio applications: VST plugins, audio units, embedded sound systems, real-time DSP. All of these are written in C++, without exception.

Goal 1 rules out heavy ML frameworks. Goal 2 rules out Python entirely for the eventual application target. The intersection is “implement from scratch, in the language audio actually runs in.” That’s C++.

4.2 Why frameworks hide what we want to teach

If we wrote this library in Python with PyTorch, the entire chapter on backpropagation would be one line:

```
loss.backward()
```

That line works. You get correct gradients. But you’ve learned nothing about how those gradients were computed. The framework’s autograd has hidden the math.

The same happens at every other level:

- `nn.Linear(in, out)` hides the weight matrix, the bias, and the forward pass.
- `nn.Conv2d` hides the `im2col-and-matmul` trick that makes 2-D convolution fast.
- `optim.Adam` hides the per-parameter first and second moments, the bias correction, the update rule.
- `nn.MultiheadAttention` hides the queries, keys, values, scaling, softmax, and the multi-head projection.

PyTorch is a beautiful productivity tool — but the very things that make it good for productivity make it bad for understanding. When the goal is to *demystify* each of these pieces, hiding them defeats the purpose. Calling a black box doesn't teach what's inside the box.

4.3 What C++ forces you to do

C++ has no `autograd`, no `nn.Module`, no decorator that quietly records a computation graph. To do backprop in C++, you have to:

1. Allocate a weight matrix W and bias b , explicitly, with shapes.
2. Implement the forward pass: $Z = W * X + b$.
3. Cache the input X because the backward pass needs it.
4. When the next layer's gradient dZ arrives, compute by hand:
 - ▶ $dW = dZ * X^T$ (with batch averaging applied in the loss),
 - ▶ $db = \text{sum across the batch of } dZ$,
 - ▶ $dX = W^T * dZ$,
5. Hand dW and db to the optimizer, which uses them to update W and b .

That is the actual content of backprop. The student doesn't learn it from PyTorch; they learn it from writing it. Once you've written it for one layer, you've written it for all of them, and you understand the algorithm permanently.

The same is true of every other piece. Once you've written `im2col` yourself and seen convolution become a matrix multiply, you'll never mistake "the conv layer applies a kernel" for the full picture again. Once you've written Adam's running averages by hand, you'll know exactly what changes when someone says "we tweaked beta2."

4.4 Why C++ specifically, instead of Python without frameworks?

A fair objection: "OK, no frameworks — but couldn't you just use Python with NumPy and avoid the boilerplate of C++?" Some introductory courses do exactly that, and it works. The reasons we went the additional step to C++:

Type safety as a teaching tool. In Python, `dense.forward(x)` accepts anything that responds to multiplication. Pass the wrong shape and you get a runtime error somewhere deep in the call stack. In C++ with templates, `Dense::forward(const Matrix<float>&)` literally cannot accept a `Tensor4D<float>` — the compiler refuses. Wrong shape, wrong rank, wrong scalar type: all caught at the call site, before you even run the program. For someone learning the data flow of a neural network, that compile-time enforcement is a feature, not a bureaucracy. It tells you exactly where the boundaries between data structures are.

Memory layout becomes visible. Our `Matrix` stores numbers in a flat row-major buffer. You SEE that — it's `std::vector<T> data_` and an

indexing formula. Our `Tensor4D` stores them in NCHW order, and the index formula is right there in the code: `data[((n*C + c)*H + h)*W + w]`. When you understand WHY this layout makes 2-D spatial convolution cache-friendly, you’ve learned something real about how hardware works. In Python the layout is hidden by NumPy and you never have to think about it.

Parallelism becomes visible. Our `matmul` parallelises across rows with `std::thread`. You SEE the threads being spawned and joined. You SEE the work-threshold below which threading isn’t worth its overhead. You SEE the row-chunking that avoids synchronisation. In Python, threads are essentially useless because of the GIL, and the real parallelism happens inside NumPy or PyTorch in code you cannot see or modify.

The performance story is honest and modifiable. When this library trains a network slowly, you can profile it and see exactly where the time goes — and fix it. We did exactly this when we added the `make BLAS=1` opt-in: profiling showed `Matrix::operator*` was the bottleneck, so we optionally route it to a tuned BLAS library via an `if constexpr` dispatch. The whole story — threads, BLAS dispatch, the option to leave it off — is right there in `Matrix.h`, observable and modifiable. In Python with PyTorch, performance is whatever the framework decides; you can benchmark it, but you can’t open it up and change it.

C++ is the language audio lives in. This is a book for music applications. The eventual goal is for the things you learn here to make their way into actual audio tools — VST plugins, audio units, embedded systems, real-time DSP. **All of those are written in C++.** There is no path from a Python prototype to a JUCE plugin that doesn’t involve rewriting in C++. By starting in C++, you skip that rewrite — the language you learn the math in IS the language your final application will run in.

C++ teaches systems thinking. Beyond the math, neural networks are systems: they consume data, allocate memory, run computations in parallel, share resources, serialise to disk. C++ exposes these concerns. You make choices about ownership (`unique_ptr` for layers, raw pointers for transient optimiser state), move semantics, binary serialisation formats with magic headers, optional dependencies guarded by macros. These are real engineering concerns that Python hides almost entirely — and that you eventually need to understand if you want to build anything that runs outside a notebook.

C++ has no version churn. PyTorch’s API changes frequently; models written in 2018 don’t always run cleanly in 2024. A Python ML codebase needs maintenance just to keep running. A C++ codebase using `-std=c++17` and the standard library will compile and run identically in 2044. The numerical methods we’re teaching don’t change. The language we teach them in shouldn’t add noise around them.

4.5 Honest counterarguments

It would be dishonest not to address the costs:

- **More lines of code per layer.** A PyTorch `nn.Linear` is shorter than our `Dense.h`. We accept this — the extra lines are the lesson.

- ▶ **Slower to iterate.** Every architectural experiment requires a re-compile. We accept this too; compile time is negligible at our scale.
- ▶ **No autograd, so we derive every gradient by hand.** This is often listed as a downside of C++. For a learning project it is the entire point.
- ▶ **Less ecosystem.** PyTorch has pretrained models, datasets, model hubs, deployment tools. We have none of that. For a finished application you'd reach for those eventually; for learning, you don't need any of them.

There is also one place where C++ is genuinely worse: for the kind of large-scale empirical experimentation that drives most modern ML research (sweeping hyperparameters across hundreds of runs, comparing dozens of architectures on huge datasets), Python's REPL and notebook ecosystem are the right tools. We are not doing that kind of research in this book. We are building a small set of architectures carefully.

4.6 When you would NOT use C++

To be clear: this argument is for this project specifically, not for machine learning in general. If you are:

- ▶ prototyping a new model architecture quickly,
- ▶ working with very large datasets that need distributed training,
- ▶ using off-the-shelf pretrained models (BERT, Whisper, CLIP),
- ▶ writing research papers that need rapid iteration on ideas,
- ▶ training in the cloud on GPUs you don't own,

you should reach for Python with PyTorch. That's what PyTorch is for, and there's no shame in using the right tool for the job.

The argument here is narrower: for **learning** how neural networks work, and for **deploying** them in **audio applications**, C++ is the right choice. For research at scale on general datasets, it isn't.

4.7 The point

Frameworks make easy things easy and hard things invisible. For a learning project, you want the opposite: hard things visible, no easy magic. C++ gives you that. Every gradient in this library is one you derived. Every operation is one you wrote. Every optimisation is one you can profile and change. There is no `loss.backward()` to hide behind.

When you finish this book, you will have written a complete neural-network library from scratch in C++ — backprop, conv layers with `im2col`, attention with positional encoding, the whole arc. Then you can use PyTorch if you want to, and you will know exactly what it is doing under the hood, because you will have built each piece yourself.

Every neural network is, at its heart, a sequence of matrix operations. Before we build a layer that learns, we need a solid matrix class — and we need to understand *why* a matrix is the right abstraction, not an arbitrary choice. This note explains both: first the conceptual leap from a single neuron to a matrix, then the design decisions inside `src/Matrix.h`.

5.1 The conceptual leap: from neuron to matrix

5.1.1 A neuron is a dot product

Strip an artificial neuron of all the mystique and what's left is embarrassingly simple. It takes a vector of inputs x , has its own weight vector w and a bias scalar b , and computes:

$$z = w * x + b = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

That value z is then passed through a nonlinearity (covered in a later note). The dot product is the *entire* computation of a neuron. Everything else — depth, learning, representation — is built on top of this one operation.

5.1.2 A layer is a matrix

A layer isn't one neuron, it's many — say m neurons all looking at the same n inputs. Each neuron i has its own weight vector w_i . If we stack those weight vectors as the **rows** of a matrix W , then W has shape $(m \times n)$: row i is “the incoming weights to neuron i .”

The whole layer's output is then:

$$z = Wx + b \quad \text{shapes: } (m \times n)(n) + (m) = (m)$$

A single matrix-vector product computes *all* the neurons simultaneously.

This is why W is shaped $(\text{out} \times \text{in})$ and not the other way around. The shape isn't a convention — it follows directly from “each output neuron is a weighted sum of the inputs.”

5.1.3 Batching: why we need matrix $\times$ matrix

You rarely push one example through at a time. You push a *batch*. Take N input examples, each a column vector of length n , and lay them side by side as the columns of a matrix X of shape $(n \times N)$. Then:

$$Z = W X + b \quad \text{shapes: } (m \times n)(n \times N) + (m \times 1) = (m \times N)$$

Column c of Z is the layer output for example c . Processing the whole batch is now a *single* matrix–matrix multiply instead of N separate ones.

Why this matters: one big dense multiply uses the CPU cache vastly better than many tiny ones, and the same operation is what a GPU parallelizes trivially. **Matrix multiplication is the beating heart of the whole library**, which is why we start here.

5.2 Conventions used in weft

Two related but separate choices shape the whole library:

Math convention — examples as columns. An input batch has shape $(\text{features} \times \text{batch_size})$. A weight matrix has shape $(\text{out} \times \text{in})$. The forward pass is $Z = W * X + b$. This is the convention used in most “from scratch” textbooks (e.g. Nielsen, *Neural Networks and Deep Learning*). Frameworks like PyTorch flip it (batch is the first dimension), but the column convention reads more like the linear algebra and is friendlier for teaching.

Storage convention — row-major in memory. Internally, the numbers live in a single flat array, with element (i, j) at position $i * \text{cols} + j$. This is *independent* of the math convention above; it’s just a memory-layout choice for cache friendliness.

5.3 Walking through Matrix.h

Most of the choices in `src/Matrix.h` aren’t arbitrary. Each one encodes something about how the rest of the library will feel to use.

5.3.1 Why templated on the scalar type

```
template <typename T = float>
class Matrix { ... };
```


Neural nets normally run in `float` — half the memory, faster, plenty of precision. But when we later want to *check* that our backprop gradients are correct (a numerical gradient check — the single best way to catch backprop bugs), we'll want `double` for accuracy. Templating lets us flip between them by changing one type.

Templates also satisfy the “all code in .h” rule naturally: template definitions essentially *must* live in headers.

5.3.2 Why a flat `std::vector`, not vector-of-vectors

A `vector<vector<T>>` scatters each row somewhere random in memory, so iterating it thrashes the cache. One contiguous block of `rows*cols` floats sits in a predictable, cache-friendly line. Element access is a single multiply-add to index — which is exactly what `operator()(i, j)` compiles down to.

5.3.3 Why the i-k-j loop ordering for matmul

```
for (i) for (k) for (j)
    C(i,j) += A(i,k) * B(k,j);
```

The textbook ordering is i-j-k, computing one element of `C` at a time. That stride-walks through `B` column-wise, which is bad for cache because consecutive accesses jump rows. The i-k-j ordering instead has the innermost loop walk *contiguously* through `B`'s row `k` and `C`'s row `i` simultaneously. Same number of multiplications, much friendlier to the cache. A free win.

5.3.4 Why + quietly broadcasts a column vector

The forward pass `Z = W*X + b` has `b` of shape `(out × 1)` and `W*X` of shape `(out × batch)`. For the code to read like the math, `+` needs to add `b` to *every column* of `W*X`. So `operator+` checks: same shape → element-wise; right operand is a column vector → broadcast across columns.

Limiting broadcasting to this one case (rather than full NumPy-style broadcasting) is deliberate. More magic means more surprises; this is the only broadcast a feed-forward net actually needs.

5.3.5 Why apply is templated on the function

```
template <typename Func>
Matrix apply(Func f) const;
```

Activation functions and their derivatives are all element-wise. Templating on the callable means the compiler can inline a lambda — no `std::function` indirection, no virtual call. ReLU will literally be `M.apply([](T x){ return x > 0 ? x : 0; })`.

5.3.6 Why hadamard is a free function

`operator*` already means matrix multiplication. Overloading it for the element-wise product — which behaves completely differently — would be a trap. In backprop you'll see *both* on the same line, and they must be visually distinct:

```
auto delta = hadamard(W_next.transpose() * delta_next,
                      activation_derivative);
```

The element-wise product gets its own name precisely so it can't be confused with the `matmul`.

5.4 Seeing it work: the forward pass falls out

With just `Matrix.h`, the forward pass of a linear layer already reads like the equation:

```
#include "Matrix.h"
using weft::Matrix;

Matrix<float> W(3, 2);           // 3 output neurons, each
                                // looking at 2 inputs
W.randomizeNormal(0.f, 0.1f);

Matrix<float> X{{1, 2, 3, 4},    // feature 1 across 4 examples
               {5, 6, 7, 8}};    // feature 2 across 4 examples
                                // -> (2 x 4)

Matrix<float> b(3, 1, 0.5f);     // one bias per output neuron

Matrix<float> Z = W * X + b;     // forward pass of a Dense
                                // layer
                                // Z has shape (3 x 4): one
                                // column per example
```

The dimensions check themselves: multiply a (3×2) by a (2×4) and you get (3×4) — the inner 2s cancel. The bias (3×1) broadcasts across columns. And the line $Z = W * X + b$ is *literally* the equation we derived above.

That correspondence between the code and the math is the thing we're protecting at every step in this library.

5.5 What's next

The forward pass tells you what a layer *computes*, but not how it *learns*. The next note covers **backpropagation** — the chain rule applied systematically

— and `src/Layer.h` builds the first learnable layer (Dense) on top of `Matrix`.

Backpropagation, and the Dense layer

6

The matrix from the last note is the foundation. This note is the heart of the library: how a neural network *learns*. We derive backpropagation for a Dense layer, implement it in `src/Layer.h` and `src/Dense.h`, and verify the implementation with a *numerical gradient check* — the single most important debugging technique in deep learning.

6.1 Backprop is the chain rule, organized

A neural network is a composition of functions:

```
L = Loss( f_N( ... f_2( f_1( X ) ) ... ), target )
```

L is a single scalar — the loss. Training means finding parameters that minimise L . Gradient descent does this with the update

```
p <- p - eta * dL/dp      for every parameter p
```

So all we need is dL/dp for each weight and bias. The chain rule is the only mathematical tool we need; backpropagation is just *the chain rule applied bottom-up over the composition, organised so we reuse intermediate quantities*. There is no new calculus — only careful bookkeeping.

6.2 Each layer is a self-contained gradient router

This is the abstraction that makes the whole thing tractable. A layer plays two roles:

Forward. It takes an input X and produces an output $Y = f(X)$.

Backward. It receives the *upstream gradient* dL/dY (i.e. “how much does the loss change if my output changes?”). From that it computes two things:

- ▶ The *parameter gradients* $dL/dW, dL/db, \dots$ — stored internally for the optimiser to apply later.
- ▶ The *downstream gradient* dL/dX — returned, to become the upstream gradient of the layer below.

The crucial word is *local*. Each layer only needs to know how its own forward computation depends on its own inputs and parameters. It has no idea what’s above or below it. The global gradient assembly is just composition — the same composition that defined the network in the first place, run in reverse.

This is also why we can specify architectures in a config file: layers are self-contained units that can be chained arbitrarily.

6.3 The intuition: perturbation propagation

Skip the formulas for a second. Here is the gut-level picture for a Dense layer with $Z = W \cdot X + b$:

Pick a single weight $w[i, j]$. Nudge it by a tiny eps . The forward formula is

$$Z[i] = \sum_k W[i, k] * X[k] + b[i]$$

so only the $k = j$ term moved. **$Z[i]$ changes by $\text{eps} * X[j]$** — and no other $Z[r \neq i]$ changes at all. That $\text{eps} * X[j]$ ripples forward through whatever comes next, and ultimately changes the loss by $\text{eps} * X[j] * dL/dZ[i]$. Dividing both sides by eps :

$$dL/dW[i, j] = X[j] * dL/dZ[i]$$

Said in words:

A parameter’s gradient is “how much it influences the output” times “how much the output influences the loss.”

The first factor comes from the layer’s own structure (its forward formula). The second factor arrives from upstream. Backprop is just gluing local “how-much-I-influence-things” computations to the upstream “how-much-that-affects-the-loss” coming down from above.

6.4 Deriving Dense’s backward pass

The element-wise result above immediately becomes a matrix formula: $X[j] * dL/dZ[i]$ is the (i, j) entry of the outer product $(dL/dZ) * X^T$. For a batch, every example contributes its own outer product and they *sum together* — and that summation happens automatically when you write it as a `matmul`. Letting dZ denote dL/dZ :

$$dW = dZ * X^T \quad \text{shapes: (out x batch)(batch x in) = (out x in)}$$

For the bias. $Z[i, k] = \dots + b[i]$ for every example k in the batch, so $dZ[i, k] / db[i] = 1$. Summing the chain rule across the batch:

$$db = dZ \text{ summed across columns} \quad \text{shape: (out x 1)}$$

This is exactly what `Matrix::sumColumns()` returns — which is why we built it.

For the input gradient (sent upstream). $Z[i,k] = \text{Sum}_j W[i,j] * X[j,k]$, so $dZ[i,k] / dX[j,k] = W[i,j]$. Summing over i :

```
dL/dX[j,k] = Sum_i W[i,j] * dZ[i,k] = (W^T * dZ)[j,k]

dX = W^T * dZ          shapes: (in x out)(out x batch) = (in
  x batch)
```

So Dense's full backward pass is three lines of matrix algebra:

```
dW = dZ * X_cache.transpose(); // (out x in)
db = dZ.sumColumns();          // (out x 1)
dX = W.transpose() * dZ;        // (in x batch), returned
                                upstream
```

6.5 The transpose symmetry — and a free sanity check

Look at the pair: forward sends the signal through W , backward sends the gradient through W^T . Same matrix, transposed. This isn't a coincidence — it's a fundamental duality. A matrix W of shape $(m \times n)$ maps an n -vector to an m -vector going forward, so its dual mapping $m \rightarrow n$ going backward has to be the transpose.

The practical consequence: **whenever you're unsure about a backprop formula, check the shapes.** The dimensions tell you whether you need a transpose, which side of the product something belongs on, and so on. I've debugged more gradients with this trick than I'd like to admit.

6.6 Implementation notes

6.6.1 Caching

`forward()` saves a copy of X in `X_cache`. `backward()` needs it to compute $dW = dZ * X^T$. For a 32-example batch this copy is trivial; for larger batches we can revisit, but it's almost never the bottleneck.

6.6.2 The Layer base class

```
template <typename T = float>
class Layer {
public:
    virtual ~Layer() = default;
    virtual Matrix<T> forward(const Matrix<T>& X) = 0;
    virtual Matrix<T> backward(const Matrix<T>& dY) = 0;
    virtual void update(Optimizer<T>& /*opt*/) {} // no-op
    default
};
```

update has a default no-op so activation layers (ReLU, Softmax) that have no parameters don't need to implement anything. Dense::update overrides it to hand each parameter to the optimiser; the optimiser decides how to apply the update.

6.6.3 Initialisation

We initialise W from a Gaussian with stddev $\sqrt{2 / \text{fan_in}}$ — **He initialisation**. It keeps the variance of activations roughly constant across layers (so signals don't shrink to zero or explode), and it's the right default when paired with ReLU. We'll discuss why properly when ReLU arrives.

6.6.4 Where the `1/batch_size` lives (decision worth flagging)

When you mean-average a per-example loss, an extra $1/N$ factor needs to live somewhere. We adopt the convention that **the loss function bakes the $1/N$ into dL/dZ** , so layer backward formulas stay clean (no batch-size arithmetic inside Dense). When we add a loss class, you'll see this directly.

6.7 Testing — and why the numerical gradient check matters more than anything

Hard truth: backprop code that runs without crashing and produces “reasonable- looking” outputs is *almost always wrong on the first try*. A subtle sign error, a missing transpose, a forgotten sum across the batch — these don't crash, they just make your network not learn well, and you waste days. The cure is the **numerical gradient check**.

For every parameter p independently:

- ▶ **Analytical:** the gradient that `backward()` computed.
- ▶ **Numerical:** the actual definition of a derivative, $(L(p+\text{eps}) - L(p-\text{eps})) / (2\text{eps})$.

If they agree to ~4 decimal places, you have proof your backward matches the math. If they don't, you have a bug. There is no substitute.

In `tests/test_dense.cpp`:

1. **Forward** is verified against a tiny hand-computed example.
2. **Backward** is verified against the same example by hand.
3. **Numerical vs analytical** is run on a (3 in, 4 out, batch 5) layer for every entry of dW and db . (We use the trivial loss $L = \text{sum}(Z)$ so dL/dZ is just a matrix of ones — cleanest possible setup.)
4. **End-to-end SGD** trains a Dense layer for 50 steps on a quadratic loss and confirms the loss drops sharply — proves the whole forward/backward/ update loop is wired correctly.

The two-sided difference $(L(p+\text{eps}) - L(p-\text{eps})) / (2\text{eps})$ is preferred over the one-sided $(L(p+\text{eps}) - L(p)) / \text{eps}$: it has error $O(\text{eps}^2)$ instead of $O(\text{eps})$, so you can use a larger `eps` and get a more accurate estimate. Use `double` and a tight tolerance if you're paranoid.

6.8 What's next

A Dense layer alone can only fit linear maps. To make a network expressive we need **nonlinearities** — ReLU, then Softmax paired with **cross-entropy loss**. That's the next note, after which we'll have everything needed for our first real classifier (IRIS).

One weight, one step (SGD by hand)

7

The main backprop note (02_backprop.md) derives the matrix formulas for `Dense::backward`. This note walks through the *same arithmetic* on a single neuron with a single weight, by hand, with concrete numbers. The point is to see exactly what the matrix formulas are mass-producing.

The runnable companion is `backprop_example/backprop_example.cpp` — it does this calculation in C++ with both raw scalars and `weft::Dense` side by side and prints the numbers.

7.1 The setup

The smallest possible “network”: a single neuron with two inputs and no activation function (we’ll add activations in the next session — the linear part alone is enough to see the mechanism).

symbol	value	meaning
x_1	2.0	first input
x_2	-1.0	second input
w_1	0.5	weight on x_1
w_2	1.0	weight on x_2
b	0.1	bias
t	1.0	target output
η	0.1	learning rate

We’ll mostly follow w_1 and treat w_2 and b as frozen, then loop back at the end.

7.2 Step 1 — Forward pass (the projection)

The neuron computes a weighted sum:

```
z = w_1*x_1 + w_2*x_2 + b
  = 0.5*2 + 1.0*(-1) + 0.1
  = 1 - 1 + 0.1
  = 0.1
```

The network output is $z = 0.1$ but we wanted $t = 1.0$. The network is wrong — off by 0.9 in the “ z is too small” direction.

7.3 Step 2 — Turning “off by 0.9” into a single number: the loss

“Off by 0.9” is a feeling, not a quantity we can differentiate. We pick a loss function to turn it into a scalar. The simplest is squared error:

$$\begin{aligned} L &= 1/2(z - t)^2 \\ &= 1/2(0.1 - 1.0)^2 \\ &= 1/2*(0.81) \\ &= 0.405 \end{aligned}$$

Three things worth noting about this choice:

- ▶ The square makes L positive regardless of which side of the target we land on.
- ▶ It also penalises being very wrong more than being slightly wrong.
- ▶ The $1/2$ out front is cosmetic — it cancels with the 2 from differentiation, leaving cleaner formulas. You can drop it and rescale η ; nothing important changes.

7.4 Step 3 — The “error signal” $\partial L/\partial z$

Backprop starts by asking: *if z were nudged a tiny bit, how would L respond?* That’s dL/dz . For squared error:

$$dL/dz = z - t = 0.1 - 1.0 = -0.9$$

The *sign* tells us which way is downhill (negative $\rightarrow$ nudging z up would reduce L). The *magnitude* (0.9) tells us how steeply L responds. This single number is what’s often called the “error signal” or “delta” at the output. In a deeper network it’s exactly what gets passed backward into the previous layer.

7.5 Step 4 — Backpropagating to w_1 via the chain rule

We want dL/dw_1 . The chain rule:

$$dL/dw_1 = (dL/dz) * (dz/dw_1)$$

We already have $dL/dz = -0.9$. For dz/dw_1 , look at the forward formula $z = w_1*x_1 + w_2*x_2 + b$. If we nudge w_1 by ϵ , only the first term moves — by $\epsilon*x_1$. So $dz/dw_1 = x_1 = 2$. Therefore:

$$dL/dw_1 = (-0.9) * 2 = -1.8$$

Read it in English: a unit increase in w_1 would increase z by 2 (because $x_1 = 2$), and a unit increase in z would decrease L at a rate of 0.9. Multiplying these together: pushing w_1 up by 1 would (linearly) decrease L by 1.8. The gradient is -1.8, so to make L smaller we should make w_1 larger.

A parameter's gradient = (how much it influences the output) $\times$ (how much the output influences the loss).

That's the entire backprop machinery for one weight, in one line of arithmetic.

7.6 Step 5 — The SGD update

Plain SGD steps each weight against its gradient:

```
w_1 <- w_1 - eta * dL/dw_1
      = 0.5 - 0.1 * (-1.8)
      = 0.5 + 0.18
      = 0.68
```

The two minus signs flip — the gradient was negative, so the update is positive. We push w_1 up, which makes z bigger, which is what we want.

The learning rate η controls how big a step we take. Too small and learning crawls; too big and we overshoot and L can actually go *up*.

7.7 Step 6 — Did it actually help?

Forward pass again with the updated w_1 (others still frozen):

```
new z = 0.68*2 + 1.0*(-1) + 0.1 = 0.46
new L = 1/2(0.46 - 1.0)^2 = 0.1458
```

L dropped from 0.405 to 0.146 — about a 3 $\times$ improvement from moving one weight. If we update w_2 and b simultaneously (their gradients work out to +0.9 and -0.9, giving new values 0.91 and 0.19), the next forward pass lands at $z = 0.64$, $L = 0.065$. Closer still.

This is the training algorithm. Run it on many (input, target) pairs with many weights, and a network learns.

7.8 Where is “squared error” in the SGD update?

This is the right question to ask, because the SGD update rule $w \leftarrow w - \eta * dL/dw$ doesn’t *mention* a loss function. So where is it?

The loss is baked into the gradient. Trace where it appeared:

- ▶ **Step 2** computed $L = 0.405$ — but L itself was never used again. It was reported for our information, then discarded.
- ▶ **Step 3** computed $dL/dz = z - t$. **This is the only place the squared error lives.** That $z - t$ is the derivative of $1/2(z - t)^2$.
- ▶ **Step 4** combined that number with the chain rule. The chain rule has no idea which loss produced the -0.9 ; it just multiplies.
- ▶ **Step 5** stepped against the gradient. By this point the loss has done its job — we have a number, and SGD steps against it.

If we used absolute error $L = |z - t|$ instead, only step 3 would change (to $dL/dz = \text{sign}(z - t)$). Cross-entropy with softmax would change just that one line too. Everything else — chain rule, bias gradient, weight update — is identical.

This is why in the library we keep **losses** and **optimisers** as separate, swappable concepts. A loss class produces the initial dL/dz given the prediction and target; an optimiser applies whatever gradients arrive however it likes (SGD, momentum, Adam, ...). The two don’t need to know about each other.

7.9 How this scales up to the matrix formulas

Look at what `Dense::backward` actually does, for our one-neuron example:

$dW = dZ * X^T$ — dZ is the 1×1 matrix $[-0.9]$, X^T is the row $[2, -1]$. Their product is $[-1.8, 0.9]$ — exactly the two chain-rule products we did by hand (-1.8 for w_1 , 0.9 for w_2). The `matmul` is mass-producing the single-weight calculation for every weight at once.

$db = dZ.\text{sumColumns}()$ — for one example, this is just dZ itself: $[-0.9]$. By hand: $dL/db = dL/dz * 1 = -0.9$. Same.

$dX = W^T * dZ$ — $W = [0.5, 1.0]$, so $W^T * [-0.9] = [-0.45, -0.9]^T$. This is $dL/dx_j = w_j * dL/dz$ — “if x_j had been a bit larger, the loss would have changed at this rate.” It’s what the layer below needs to know in order to update its own weights. We don’t have a layer below here, so we ignore it. Stack two `Dense` layers and dX from the second becomes the dZ going into the first.

Every matrix line in `Dense::backward` is producing the per-weight arithmetic from this note — one for every weight in W , summed across every example in the batch. The math doesn’t get fancier when you go from one weight to a million; only the bookkeeping does, and that’s what matrices are for.

7.10 See it run

`backprop_example/backprop_example.cpp` runs this exact calculation twice — once with raw scalars (mirroring this note line by line), once with `weft::Dense` — and prints both sets of numbers so you can verify they match.

Build it from the project root with `make -C build` and run `./build/bin/examples/backprop_example`.

Non-linearities (ReLU and Softmax)

8

A network of Dense layers alone can only learn *linear* functions, no matter how deep it is. This note explains why, then builds and explains the two activations we need for our first classifier: ReLU for hidden layers and Softmax for the output of a multi-class classifier.

8.1 Why nonlinearities are not optional

Stack two Dense layers with nothing between them:

$$\begin{aligned} H &= W_1 * X + b_1 \\ Z &= W_2 * H + b_2 \end{aligned}$$

Substitute and rearrange:

$$\begin{aligned} Z &= W_2 * (W_1 * X + b_1) + b_2 \\ &= (W_2 * W_1) * X + (W_2 * b_1 + b_2) \\ &= W' * X + b' \end{aligned}$$

The two-layer “network” is mathematically identical to **one** Dense layer with $W' = W_2 * W_1$ and $b' = W_2 * b_1 + b_2$. The same algebra extends to any depth: composing affine functions yields an affine function. An affine model can only learn linear decision boundaries — straight lines or hyperplanes — so it can’t represent XOR, curves, or hierarchical features.

Any non-linear function between layers breaks this collapse. The simplest such function that works extraordinarily well in practice is **ReLU**.

8.2 ReLU — as simple as it gets

$$\text{ReLU}(z) = \max(0, z) \quad \text{element-wise}$$

Negative entries become zero; non-negative entries pass through. Shape is preserved (the operation is element-wise, so input and output have the same shape).

8.2.1 Backward

ReLU's derivative is:

```
dReLU/dz = 1   if z > 0
           = 0   if z < 0
           (undefined at z = 0; we pick 0 by convention -- exactly
            -zero values
            have measure zero in practice and the choice does not
            matter)
```

Because the operation is element-wise, the Jacobian is *diagonal* — every output depends only on the corresponding input — so backward collapses to a Hadamard product:

```
dZ = dA (*) mask,   where mask[i,j] = 1 if Z[i,j] > 0 else 0
```

In words: route the upstream gradient through wherever the input was positive, zero out everywhere else. Sometimes called a “gating” function for this reason — it gates the gradient flow based on the forward sign pattern.

8.2.2 Why it works so well

A short list of practical reasons ReLU has dominated since ~2012:

- ▶ Element-wise and dead cheap to compute and differentiate.
- ▶ No saturation in the positive direction (compare with sigmoid and tanh, which flatten out for large inputs and kill the gradient).
- ▶ Encourages sparse activations: at any given time, many units output exactly zero, which acts as an implicit form of regularisation.

A failure mode worth flagging: a unit can get stuck in the dead region (always outputting 0, always receiving 0 gradient — “dying ReLU”). Variants like Leaky ReLU and ELU address this. We’ll add Leaky ReLU later if we need it; plain ReLU is fine for IRIS.

8.3 Softmax — turning scores into probabilities

For multi-class classification, the final layer needs to output a probability distribution over K classes. Softmax does that. For each example (each column):

```
softmax(z)_i = exp(z_i) / Sum_k exp(z_k)
```

Three properties drop out immediately:

- ▶ Each output is in $(0, 1)$.
- ▶ Outputs sum to 1.
- ▶ The largest logit becomes the largest probability (monotonicity).

So we get a valid probability distribution where the network's “vote” for the most likely class is highest. Note Softmax is per-column — unlike ReLU, every output depends on every input *within an example*.

8.3.1 3a. The numerical stability trick (log-sum-exp shift)

$\exp(z)$ for $z = 1000$ overflows. The fix uses an algebraic identity that is worth seeing once:

$$\text{softmax}(z) = \text{softmax}(z - c) \quad \text{for any constant } c$$

Proof: multiply numerator and denominator of $\text{softmax}(z - c)$ by $\exp(c)$; the factors cancel and you are back at $\text{softmax}(z)$.

So we can shift all logits by *any* constant without changing the answer. The standard trick is to shift by the column max:

$$m = \max(z) \\ \text{softmax}(z)_i = \exp(z_i - m) / \sum_k \exp(z_k - m)$$

After shifting, the largest exponent is $\exp(0) = 1$ — no overflow. The rest are $\exp(\text{negative})$ in $(0, 1]$ — no overflow. Tiny terms may underflow to 0, but that is the correct answer. This shift is mandatory for any real-world Softmax; every framework does it.

8.3.2 3b. The Jacobian — the first non-trivial backward

This is the first layer where every output depends on every input, so the Jacobian is *not* diagonal. The derivation is worth doing once.

Fix one column (one example). Let $s = \text{softmax}(z)$, so $s_i = e^{z_i} / D$ with $D = \sum_k e^{z_k}$. By the quotient rule:

► **$i = j$ (diagonal entry):**

$$\begin{aligned} ds_i/dz_i &= (e^{z_i} \cdot D - e^{z_i} \cdot e^{z_i}) / D^2 \\ &= (e^{z_i}/D) * (1 - e^{z_i}/D) \\ &= s_i * (1 - s_i) \end{aligned}$$

► **$i \neq j$ (off-diagonal):**

$$\begin{aligned} ds_i/dz_j &= (0 \cdot D - e^{z_i} \cdot e^{z_j}) / D^2 \\ &= -(e^{z_i}/D) * (e^{z_j}/D) \\ &= -s_i * s_j \end{aligned}$$

Packed together with the Kronecker delta:

$$ds_i/dz_j = s_i * (\delta_{ij} - s_j)$$

8.3.3 3c. Backward from the Jacobian

Chain rule for dL/dz_j :

$$\begin{aligned}
 dL/dz_j &= \text{Sum}_i (dL/ds_i) * ds_i/dz_j \\
 &= \text{Sum}_i (dL/ds_i) * s_i * (\text{delta}_{i,j} - s_j) \\
 &= s_j * (dL/ds_j) - s_j * \text{Sum}_i s_i * (dL/ds_i) \\
 &= s_j * [(dL/ds_j) - \text{dot}] \quad \text{where dot} = \text{Sum}_i s_i * (dL/ds_i)
 \end{aligned}$$

So per column the backward pass is just two passes:

1. Compute the scalar $\text{dot} = s^T * dS$ for that column.
2. For each entry: $dZ[i,j] = S[i,j] * (dS[i,j] - \text{dot})$.

Look at the code in `src/Softmax.h` and you'll see exactly these two passes inside the column loop.

8.3.4 3d. The Softmax + cross-entropy shortcut (preview)

In classification networks, Softmax is almost always paired with **cross-entropy loss** $L = -\text{Sum}_i t_i * \log(s_i)$ where t is the one-hot target. If you compute the *combined* gradient — pushing $dL/dz = dL/ds * ds/dz$ all the way through — almost everything cancels, and you're left with:

$$dL/dz = S - \text{target}$$

That is dramatically simpler than (and numerically better-behaved than) chaining the standalone Softmax backward with a standalone cross-entropy backward. The next note covers this and the Loss class will use the shortcut.

The standalone Softmax backward we just implemented is still correct on its own — verified by the numerical gradient check in `test_softmax.cpp` — and we keep it because Softmax may also be used outside the standard classification setup (e.g. attention layers, custom losses).

8.4 Other activations (briefly)

- **Sigmoid** $\text{sigma}(z) = 1 / (1 + \exp(-z))$: maps to $(0, 1)$, derivative $\text{sigma}(1 - \text{sigma})$. Used historically and still occasionally for binary classification outputs. Suffers from saturation (vanishing gradient for large $|z|$), so usually avoided in hidden layers.
- **Tanh** $\text{tanh}(z)$: maps to $(-1, 1)$, derivative $1 - \text{tanh}^2(z)$. Zero-centred unlike sigmoid, but still saturates.
- **Leaky ReLU** $\max(\alpha z, z)$ with small α (e.g. 0.01): keeps a small non-zero gradient for negative inputs to avoid the “dying ReLU” problem.
- **GELU, Swish**: smoother variants of ReLU used in modern transformers.

We will add what we need when we need it. For IRIS, ReLU is more than enough.

8.5 What's next

We have everything for the forward pass of a real classifier: Dense $\rightarrow$ ReLU $\rightarrow$ Dense $\rightarrow$ Softmax. To train it, we need a loss function. The next note covers **cross-entropy** and the Softmax+CE combined gradient shortcut, after which we have all the pieces for the first end-to-end network on IRIS.

The Jacobian (the unifying object behind every backward pass)

9

02_backprop.md derived `Dense::backward` from a perturbation argument on a single weight. 03_activations.md built ReLU and Softmax and derived their backward formulas. This note steps back and shows that *every* backward pass we've written — and every one we'll write — is the same operation:

Backward = multiply the upstream gradient by the transposed Jacobian of the layer's forward function.

Layers differ only in what their Jacobian looks like and how cheaply we can multiply by it.

9.1 What a Jacobian is

Build up from what's familiar.

9.1.1 Scalar function of a scalar

For $f: \mathbb{R} \rightarrow \mathbb{R}$, the derivative $f'(x)$ is one number — the slope of f at x . It says: if I nudge x by ϵ , f changes by approximately $f'(x) * \epsilon$.

9.1.2 Scalar function of a vector

For $f: \mathbb{R}^n \rightarrow \mathbb{R}$, we need one partial per input, packaged as a vector — the **gradient**:

```
gradf = [ df/dx_1, df/dx_2, ..., df/dx_n ]
```

Same content as the slope, but vector-valued because there are multiple directions you can nudge.

9.1.3 Vector function of a vector

For $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ — m outputs, n inputs — each output has its own gradient. Stack them as rows and you get a matrix: the **Jacobian**.

```
      +               +
      | df_1/dx_1  df_1/dx_2  ...  df_1/dx_n |      shape: m x
n    J = | df_2/dx_1  df_2/dx_2  ...  df_2/dx_n |
      | ...          ...          ...  |
      | df_m/dx_1  df_m/dx_2  ...  df_m/dx_n |
      +               +
```

$J[i, j] = df_i/dx_j$ — “how does output i respond to a nudge in input j ?”

Row i is “the gradient of output i .” Column j is “the effect of input j on every output.”

9.1.4 The geometric picture

The Jacobian is the **local linearisation** of f at the point x . For small ϵ s:

$$f(x + \epsilon) \sim f(x) + J * \epsilon$$

This is the multi-variable generalisation of $f(x + \epsilon) \sim f(x) + f'(x) * \epsilon$. It’s exactly the perturbation intuition we used to derive Dense’s backward pass in `02b_one_weight.md` — but stated for vectors. The Jacobian answers “how does the output respond when an input is nudged?” all at once, for every (input, output) pair.

9.2 Backprop is “multiply by the transposed Jacobian”

The chain rule for a layer $f: x \rightarrow y$ reads:

$$\begin{aligned} dL/dx_j &= \text{Sum}_i (dL/dy_i) * (dy_i/dx_j) \\ &= \text{Sum}_i (dL/dy_i) * J[i, j] \end{aligned}$$

That sum is exactly the j -th entry of $J^T * dL/dy$. So in matrix form:

$$dL/dx = J^T * dL/dy$$

That single formula is the entire content of “backward” for any layer. Backprop is the chain rule; the chain rule for vector functions is matrix multiplication by the transposed Jacobian. *Every* `backward()` we will ever write — Dense, ReLU, Softmax, Conv, BatchNorm, attention — is a specialisation of this one expression.

What changes from layer to layer is (1) what the Jacobian looks like and (2) how cheaply we can multiply by it *without* materialising it.

9.3 The Jacobians of weft’s three layers

This view lets you re-read everything we’ve built as variations on $J^T * dL/dy$.

9.3.1 Dense: J is W

$y = Wx + b$, so $dy_i/dx_j = W[i, j]$. **The Jacobian is W itself.** No materialisation needed — W is already sitting in memory as a layer parameter. The chain rule gives:

$$dX = W^T * dY \quad \text{i.e.} \quad dX = J^T * dY$$

The W^T that appears in `Dense::backward` is just J^T for this particular Jacobian. The “transpose symmetry” of `02_backprop.md` is the general J^T rule, specialised to the linear case.

9.3.2 ReLU: J is diagonal

$y = \max(0, z)$ element-wise, so $dy_i/dz_j = 0$ whenever $i \neq j$. The Jacobian is **diagonal**, with:

$$J[i, i] = 1 \text{ if } z_i > 0 \text{ else } 0$$

Multiplying by a diagonal matrix is the same as element-wise multiplying by its diagonal — a Hadamard product. That is why `dZ = dA (*) mask` is correct, and why we store only the K -element mask, never the $K \times K$ matrix.

9.3.3 Softmax: J is dense, but structured

$s_i = \exp(z_i) / \sum_k \exp(z_k)$. Every output depends on every input, so:

$$J[i, j] = ds_i/dz_j = s_i * (\delta_{ij} - s_j)$$

This is the first non-diagonal Jacobian in the library. Two structural properties worth noticing:

- **It’s symmetric** ($J = J^T$): the off-diagonal $-s_i * s_j$ is symmetric in i and j , and the diagonal entries $s_i(1 - s_i)$ are well-defined. So $J^T * dS = J * dS$ — convenient.
- **Each row sums to zero**: because the outputs s_i always sum to 1, nudging any z_j rearranges the s_i without changing their total. So $\sum_i ds_i/dz_j = 0$ for any j . Useful sanity check when debugging.

9.4 A concrete worked example

Pick $z = (1, 0, -1)$. After the stability shift (subtract $\max = 1$):

$$\begin{aligned} e_{-z'} &= (e^0, e^{-1}, e^{-2}) \sim (1, 0.368, 0.135) \\ D &= 1 + 0.368 + 0.135 \sim 1.503 \\ s &\sim (0.665, 0.245, 0.090) \end{aligned}$$

Build J from $J[i,j] = s_i(\text{delta_i_j} - s_j)$:

$$J \sim \begin{array}{c|ccc|c} & + & & & + \\ + & | & 0.223 & -0.163 & -0.060 & | \\ & | & -0.163 & 0.185 & -0.022 & | \\ & | & -0.060 & -0.022 & 0.082 & | \\ + & + & & & & + \end{array}$$

(Symmetric ✓, every row sums to ~0 ✓.)

Pick an arbitrary upstream gradient $ds = (0.5, -0.3, 0.1)$ and compute the downstream $dZ = J * ds$ two ways.

Method 1 — materialise J and matmul:

$$\begin{aligned} dZ[0] &= 0.223*0.5 + (-0.163)*(-0.3) + (-0.060)*0.1 \sim 0.154 \\ dZ[1] &= -0.163*0.5 + 0.185*(-0.3) + (-0.022)*0.1 \sim -0.139 \\ dZ[2] &= -0.060*0.5 + (-0.022)*(-0.3) + 0.082*0.1 \sim -0.015 \end{aligned}$$

Method 2 — the algebraic shortcut implemented in `Softmax::backward`:

$$\begin{aligned} \text{dot} = s * ds &= 0.665*0.5 + 0.245*(-0.3) + 0.090*0.1 \sim 0.268 \\ dZ[0] &= s_0(ds_0 - \text{dot}) = 0.665*(0.5 - 0.268) \sim 0.154 \\ dZ[1] &= s_1(ds_1 - \text{dot}) = 0.245*(-0.3 - 0.268) \sim -0.139 \\ dZ[2] &= s_2(ds_2 - \text{dot}) = 0.090*(0.1 - 0.268) \sim -0.015 \end{aligned}$$

Identical results. The shortcut is precisely the algebra of $J * ds$ with the structure of J factored out — so we never have to build J.

9.5 Never materialise a Jacobian you can avoid

For K classes, Softmax's explicit Jacobian is a $K \times K$ matrix *per example*. With $K = 1000$ and a batch of 32, that's 32 million floats just to hold the Jacobians — plus a 1000×1000 matrix-vector product per example.

The shortcut formula avoids both: one dot product ($O(K)$) plus one element-wise pass ($O(K)$) per column. **$O(K)$ instead of $O(K^2)$, with zero extra allocation.**

This is the general principle of efficient backprop:

If your Jacobian has structure, exploit it. Never build a Jacobian you can avoid building.

Every layer in this library follows the rule:

- **Dense:** Jacobian *is* W — a parameter we already have.
- **ReLU:** Jacobian is diagonal — we store only the diagonal, as a mask.
- **Softmax:** Jacobian has rank-1 correction structure — one dot product per column captures the action.

- **Convolution** (later): the Jacobian is enormous but extremely sparse and shift-invariant — the action becomes another (transposed) convolution.

The skill of writing efficient layers is recognising the structure of J and reducing $J^T * dY$ to the cheapest equivalent operation.

9.6 Summary

The Jacobian is the single object underneath every backward pass — the matrix of all first-order responses of a layer’s outputs to its inputs. The chain rule for vector functions is matrix multiplication by the transposed Jacobian:

$$dL/dx = J^T * dL/dy$$

That formula, plus the principle “exploit structure, never materialise an expensive Jacobian”, is the whole story of backward in a neural-network library. Everything else is recognising what J looks like for your particular layer and rewriting $J^T * dY$ in the cheapest possible form.

Cross-entropy loss (and the Softmax+CE shortcut)

10

This note covers the loss function we'll use for every classifier in weft. It also derives the famous Softmax + cross-entropy "shortcut" — the result that the combined gradient with respect to the logits is simply $s - t$ (divided by batch size). That identity is the reason almost all classifier training code looks the way it does.

10.1 Why not squared error?

Squared error penalises numeric distance: $(\text{prediction} - \text{target})^2$. That's the right loss when the prediction and target are real-valued quantities (temperature, price, pixel intensity). For classification, our prediction is a *probability distribution* (Softmax made sure of that), and so is our target (one-hot, or smoothed). What we want is a loss that measures how *different* the predicted distribution is from the target distribution. Squared error doesn't know that the output space is the simplex.

A second reason is practical. Squared error's gradient is bounded by the size of the prediction-minus-target vector, so a confidently-wrong network gets only a moderate gradient. We want the opposite: confident wrongness should be penalised *more* aggressively, not less.

Both problems are solved by cross-entropy, which comes naturally from information theory and from a maximum-likelihood argument.

10.2 Cross-entropy: definition and interpretation

Given a target distribution t and a predicted distribution s over K classes:

$$H(t, s) = - \sum_i t_i * \log(s_i)$$

The intuition: $-\log(s_i)$ is the **surprise** of seeing class i when the model assigned it probability s_i . Predict 0.99 on what actually happens and surprise is tiny (~ 0.01); predict 0.001 and surprise is huge (~ 6.9). Cross-entropy is the expected surprise, weighted by the *true* distribution.

For a one-hot target (the typical classifier setup — true class c has $t_c = 1$, others are 0), the sum collapses:

$$H(t, s) = - \log(s_c)$$

The loss is just the negative log of whatever probability the network assigned to the correct class. Assign 99% to the truth: ~0.01 loss. Assign 1% to the truth: ~4.6 loss. Assign 0% to the truth (impossible in practice, but instructive): infinite loss.

Equivalent perspective: maximum likelihood. If your model outputs $P(\text{class} \mid \text{input}) = s_i$, then the likelihood of the data is the product of $s_{\{c_n\}}$ over examples. Maximising likelihood is equivalent (taking logs) to minimising $-\text{Sum}_n \log(s_{\{c_n\}})$, which is cross-entropy with one-hot targets. The information-theoretic and probabilistic stories give the same loss.

10.3 The batch form

Average over N examples:

$$L = (1/N) * \text{Sum}_n H_n(T_n, S_n) = -(1/N) * \text{Sum}_n \text{Sum}_i T_n[i] * \log(S_n[i])$$

Averaging (rather than summing) makes the loss magnitude independent of batch size, so learning rates don't need to be rescaled when you change the batch.

By the convention we established in `02_backprop.md`, the $1/N$ factor lives **inside the gradient returned by the loss class**, not inside the backward formulas of layers below it. This keeps layer code simple — they never see batch-size arithmetic.

10.4 The standalone gradient

Differentiating with respect to $S_n[i]$ (one entry of the predicted distribution for one example):

$$dL/dS_n[i] = -(1/N) * T_n[i] / S_n[i]$$

That's what `CrossEntropy::backward()` returns. Note the blow-up when $S_n[i]$ is tiny and $T_n[i] = 1$ — exactly the case of maximum error. The expression is correct, but it's brittle in float arithmetic.

10.5 The Softmax + CE shortcut (the beautiful part)

The Softmax Jacobian (from 03b_jacobian.md) is $dS_i/dZ_j = S_i(\delta_{i,j} - S_j)$. Chain it with the CE gradient:

```
dL/dZ_j = Sum_i (dL/dS_i) * (dS_i/dZ_j)
          = Sum_i (-T_i / S_i) * S_i * (delta_i_j - S_j)
          <- S_i cancels!
          = Sum_i (-T_i) * (delta_i_j - S_j)
          = -T_j + S_j * Sum_i T_i
          = S_j - T_j                                <- because
          Sum_i T_i = 1
```

Add the $1/N$ batch factor and you're done:

```
dL/dZ = (S - T) / N
```

The combined gradient through Softmax and cross-entropy is just “subtract target from prediction, divide by batch size.” The two derivatives, each of which individually contained pieces that can blow up, telescope to the simplest possible expression.

There are two payoffs:

- **Pedagogical:** the gradient has an obvious meaning — where your predicted probability exceeds the target, push down; where it falls short, push up. Magnitude is the discrepancy.
- **Numerical:** no S_i ever appears in a denominator in the combined form. Underflowing softmax probabilities cause no trouble.

This identity is the reason almost every classification training loop — in PyTorch, TensorFlow, JAX, here — uses a fused Softmax-cross-entropy operation internally.

10.6 Why we still implement them separately in weft

For pedagogy, we keep Softmax and CrossEntropy as separate classes, each with its own backward. The standalone formulas are correct and match the math we derived in 03_activations.md. The composition `Softmax.backward(CE.backward())` is exactly $(S - T)/N$ — that is the identity our test 4 verifies numerically. For IRIS-scale problems the numerical fragility of the standalone form never manifests. When we eventually need the fused form for stability (large-K classifiers, extreme logit values), we'll add a `SoftmaxCrossEntropy` convenience class. The mathematics is the same either way.

10.7 What we built

src/Loss.h — abstract base. Two methods: forward(predicted, target) -> scalar (caches its inputs) and backward() -> dL/d(predicted). The mean-over-batch 1/N factor lives inside the gradient by convention.

src/CrossEntropy.h — concrete loss:

- ▶ forward: computes the average cross-entropy. A tiny eps is added inside log for safety against an exact zero in predicted.
- ▶ backward: computes $-(1/N) * T / S$ elementwise (with the same eps in the denominator).

tests/test_crossentropy.cpp — five tests:

1. Forward on a hand-checkable single example: $L = -\log(0.7)$.
2. Forward batch averaging.
3. Numerical gradient check on dL/dS.
4. **The combined identity:** `Softmax.backward(CE.backward()) == (S - T)/N`, computed two ways and compared. This is mathematical proof in code.
5. Full-chain numerical gradient check: perturb each logit $Z[i,j]$, confirm $(L(Z+eps) - L(Z-eps))/(2eps)$ matches the dZ produced by chaining `Softmax.backward(CE.backward())`.

All 35 tests in the project (Matrix, Dense, ReLU, Softmax, CrossEntropy) pass.

10.8 What's next

We have every piece of a classifier:

```
Dense -> ReLU -> Dense -> Softmax -> CrossEntropy
```

The next session wires these into a small Network class that holds an ordered list of layers and runs them forward/backward as a unit, then trains it on the **IRIS** dataset — the first time we'll watch loss curves drop on real data.

Network: layers composed

11

By now, all the math in weft lives inside the individual layer and loss classes — Dense, ReLU, Softmax, CrossEntropy. This note covers the small class that composes them into a trainable model.

Network is the simplest class in the library, and it should be — its job is just to chain things together.

11.1 Forward = function composition

A neural network is literally a composition of functions:

$$f = f_N \circ f_{N-1} \circ \dots \circ f_2 \circ f_1$$

In code, that's a for loop that threads the output of one layer into the input of the next:

```
Matrix<T> A = X;
for (auto& layer : layers_)
    A = layer->forward(A);
return A;
```

That's the whole forward pass.

11.2 Backward = composition, reversed

The chain rule for composed functions is:

$$(f_N \circ \dots \circ f_1)'(X) = J_N * J_{N-1} * \dots * J_1$$

where J_k is the Jacobian of layer k . From the $J^T * dY$ view in note 03b_jacobian.md, computing dL/dX from dL/dY means *transposing* this product and applying it to the upstream gradient — which is exactly multiplying by J_1^T , then by $J_2^T, \dots$, then by J_N^T . So we walk the list in reverse:

```
Matrix<T> g = dY;
for (auto it = layers_.rbegin(); it != layers_.rend(); ++it)
    g = (*it)->backward(g);
return g;
```

The reversal isn't a convention — it's the chain rule applied to a composition, written out.

11.3 Ownership: `unique_ptr<Layer<T>>`

Each layer is heap-allocated, owned by a `std::unique_ptr<Layer<T>>`. The Network keeps a `std::vector` of those.

Why heap and pointers? Polymorphism. `Dense<T>`, `ReLU<T>`, and `Softmax<T>` are different concrete types but share the abstract base `Layer<T>`. Storing them by value in a `vector<Layer<T>>` would slice them (only the base part would be kept). Pointers preserve the dynamic type so `layer->forward(A)` dispatches to the correct override.

`unique_ptr` (rather than raw pointers) makes ownership unambiguous and automatic — when the Network is destroyed, every layer is destroyed with it. No leaks, no manual delete.

11.4 The `add<L>(args...)` template helper

```
template <template <typename> class L, typename... Args>
L<T>& add(Args&&... args) {
    auto ptr = std::make_unique<L<T>>(std::forward<Args>(args)
    ...);
    L<T>* raw = ptr.get();
    layers_.push_back(std::move(ptr));
    return *raw;
}
```

Three things this gives us at the call site:

```
net.add<Dense>(4, 16); // construct Dense<float>(4, 16), add
                       it
net.add<ReLU>();       // no args
auto& d = net.add<Dense>(16, 3); // also returns a ref to
                               the concrete layer,
d.W() = ...;              // so we can configure it
                           directly
```

The template-template parameter `template <typename> class L` lets the user pass `Dense`, `ReLU`, etc. (the *templates*, unspecialised) and weft fills in `L<T>` itself. The variadic `Args` forward to the chosen layer's constructor. Returning a reference to the concrete type (not `Layer<T>&`) lets tests and configuration code reach the layer's specific accessors without `dynamic_cast`.

11.5 Loss stays outside Network

Some libraries treat the loss as a final layer. We don't, for two reasons.

- **Type mismatch:** layers map matrices to matrices; losses map matrices to a scalar. Forcing a single forward signature would be ugly.
- **Conceptual separation:** a Network produces *predictions*. A Loss compares them to targets and produces a gradient to start backprop with. Keeping them separate makes the training loop read like the math:

```
Matrix<T> pred = net.forward(X);           // network
      produces predictions
T          L    = loss.forward(pred, target); // loss measures
      them
Matrix<T> dPred = loss.backward();          // loss starts
      the gradient
net.backward(dPred);                        // network
      finishes it
net.update(opt);                            // optimiser
      applies the step
```

This is also why PyTorch, TensorFlow, JAX, etc. all keep losses separate from the model.

11.6 The full-pipeline numerical gradient check

The test that matters most is in `tests/test_network.cpp`: build the complete classifier

```
Dense(3,5) -> ReLU -> Dense(5,4) -> Softmax -> CrossEntropy
```

with random weights and 4 examples of one-hot targets. Then for each entry $W[i, j]$ of the first Dense's weight matrix, compare:

- the analytical gradient produced by `net.backward(ce.backward())`, and
- the finite difference $(L(W + \text{eps}) - L(W - \text{eps})) / (2 \text{eps})$.

If they agree everywhere — and they do — then **every component in the library is correctly wired together and computing the right gradient**. Any bug in any layer's backward, in the Network's reverse iteration, in CrossEntropy's $-T/S$, or in Softmax's Jacobian formula would show up here.

This is the strongest correctness test we can write short of training a real model. Whenever you add a new layer to weft, the first thing to do is drop it into the middle of a network like this and re-run the numerical gradient check.

11.7 What we built

`src/Network.h` — owns layers via `unique_ptr<Layer<T>>`, provides forward, backward, update, and the `add<L>` template helper.

`tests/test_network.cpp` — three tests:

1. Forward composition matches a hand-chained sequence (sanity check).
 2. **Full-pipeline numerical gradient check** (the important one).
 3. End-to-end SGD: a small 4-feature 3-class synthetic problem trains for 200 steps and cross-entropy loss drops by more than 3×. First time the library does real learning.
-

11.8 What's next: IRIS

We have every piece needed for a real classifier:

- ▶ Matrix and the operations on it.
- ▶ Dense (forward, backward, SGD update).
- ▶ ReLU, Softmax activations.
- ▶ CrossEntropy loss.
- ▶ Network to wire them.

The next step is the IRIS dataset: load it, normalise the features, split into train/test, write a training loop, watch the loss curve, report accuracy. That goes in `examples/iris_classifier/`.

After IRIS we'll also start collecting the roadmap items the project will need as it scales:

- ▶ **Train/eval mode + Dropout**: layers gain a mode flag; Dropout drops a random fraction of activations during training, scales them up at inference (or uses inverted dropout).
- ▶ **Conv2D and MaxPool**: needs 4D tensors. Arrives with the ConvNet milestone.

IRIS: the first end-to-end classifier

12

This note walks through the IRIS example end-to-end and the new tools the library acquired to make it possible.

The example lives at `examples/iris_classifier/`. The library tools it relies on are:

- ▶ `Matrix::selectColumns(indices)` — slice a matrix by column indices.
 - ▶ `Standardizer` — fit per-feature mean/std on training data, apply to both train and test.
 - ▶ `one_hot`, `train_test_split`, `shuffled_indices`, `accuracy` — free functions in `Data.h`.
-

12.1 The standard ML pipeline

Almost every supervised-learning example follows the same six steps. IRIS is the smallest interesting instance of this pattern; MNIST, the audio classifier, and the autoencoder will follow the same skeleton.

1. **Load** the raw data into matrices (features $\times$ examples).
2. **One-hot encode** the class labels into a (`num_classes` $\times$ `N`) target matrix.
3. **Split** into train/test (random, with a fixed seed).
4. **Standardise** features — fit the scaler on TRAIN ONLY, then apply to both splits.
5. **Train** with mini-batch SGD, shuffling each epoch.
6. **Evaluate** with accuracy on the held-out test set.

The last four steps are essentially mechanical once the data is loaded. The library's `Data.h` and `Standardizer.h` cover them.

12.2 Why standardise features?

Raw IRIS features have very different scales: sepal length is in `[4.3, 7.9]`, petal width is in `[0.1, 2.5]`. A gradient update step has the same learning rate for every weight, so a weight attached to a large-scale feature receives a much larger raw gradient than one attached to a small-scale feature, simply because of the input magnitudes. This forces the optimiser to take small steps to avoid blowing up the large-scale features, and learning crawls on everything else.

Standardising — subtract per-feature mean, divide by per-feature std — puts all features on roughly the same scale (mean ~ 0 , std ~ 1). Suddenly

every weight sees comparable inputs and a single learning rate works well across all of them.

This generalises:

- ▶ **Images:** typically scale pixels to $[0, 1]$ or standardise per channel.
- ▶ **Audio:** often log-mel spectrograms then standardise.
- ▶ **Tabular:** standardise numeric features; one-hot encode categoricals.

Standardizer is the simplest case; we'll add specialised preprocessors later if needed.

12.3 The “fit on train only” rule

The single most common subtle bug in ML pipelines is computing the mean and std on the *entire* dataset (train + test together) and using those statistics to standardise both splits. This is called **data leakage**: statistics of the test set bleed into preprocessing, which lets the training process implicitly “see” test data. The test accuracy ends up optimistic and you don't notice until production.

Standardizer's API encourages the right pattern:

```
Standardizer<float> sc;
Matrix<float> X_train_std = sc.fit_transform(X_train);    //
    learn AND apply
Matrix<float> X_test_std  = sc.transform(X_test);        //
    apply (same params)
```

`fit_transform` learns mean/std from the data it's given; `transform` just applies the learned parameters. We never call `fit` on the test data. The same scaler instance is used for both, so the test inputs are mapped into the same space the training inputs live in.

12.4 The training loop pattern

Mini-batch SGD with per-epoch shuffling:

```
SGD<float> opt(learning_rate);
for (int epoch = 0; epoch < epochs; ++epoch) {
    auto idx = shuffled_indices(X_train.cols(), epoch);    // new
    order each epoch

    for (size_t start = 0; start < X_train.cols(); start +=
        batch_size) {
        size_t end = min(start + batch_size, X_train.cols());
        vector<size_t> batch_idx(idx.begin() + start, idx.begin
            () + end);
```

```

Matrix<float> X_batch = X_train.selectColumns(batch_idx)
;
Matrix<float> Y_batch = Y_train.selectColumns(batch_idx)
;

Matrix<float> S = net.forward(X_batch);
ce.forward(S, Y_batch);
net.backward(ce.backward());
net.update(opt);
}
}

```

Three things matter here:

- ▶ **Mini-batches** (rather than the full dataset or single examples). Full-batch gradient descent is too memory-hungry for big datasets and has fewer updates per epoch; pure stochastic gradient descent (one example at a time) has too noisy a gradient. Mini-batches are the middle ground — 16, 32, 64 are typical for small problems; 128–512 for larger ones.
- ▶ **Shuffle every epoch**. Without it, the network sees the same batches in the same order every time. Even though SGD is “stochastic” already (because each batch is just a noisy estimate of the true gradient), the noise from a deterministic batch order is correlated across epochs, which makes convergence slower and can settle the optimiser into bad local patterns.
- ▶ **Use the same shuffled indices seed strategy across runs** so results are reproducible. The example uses the epoch number as the seed, which gives different orders per epoch but is fully deterministic.

12.5 Accuracy as the eval metric

For classification, the natural evaluation metric is the fraction of examples where the model’s predicted class equals the true class. That’s just `argmax-per-column` of predictions vs `argmax-per-column` of one-hot targets:

```

T accuracy(predictions, targets):
    correct = 0
    for each column j:
        if argmax(predictions[:, j]) == argmax(targets[:, j]):
            correct += 1
    return correct / N

```

The implementation in `Data.h` works whether predictions is post-softmax probabilities or raw logits — only the ranking of values matters.

Loss and accuracy are not the same thing! Loss is a continuous, smooth measure used for gradient descent. Accuracy is what you actually care about. They usually move together, but it’s possible for loss to keep going down (the model becomes more confidently right where it was already right) while accuracy plateaus.

12.6 What the example actually shows

```
loaded 150 examples, 4 features, 3 classes
split: 120 train, 30 test (seed=1)
features standardised (mean ~ 0, std ~ 1 on train)

architecture: Dense(4,16) -> ReLU -> Dense(16,3) -> Softmax
loss:         cross-entropy
optimiser:    plain SGD, lr=0.1, batch_size=16
training for 100 epochs
```

epoch	train_loss	train_acc	test_loss	test_acc
0	1.2618	0.4250	1.2937	0.4000
10	0.2062	0.9500	0.1208	1.0000
20	0.1320	0.9667	0.0751	1.0000
...				
100	0.0506	0.9917	0.0611	0.9667

```
final test accuracy: 0.9667 (29/30 correct)
```

Two things to look at:

- ▶ The model's untrained baseline is ~40% accuracy. Random guessing would give ~33% (1/3 classes). The slight advantage at epoch 0 is just luck in how the random initial weights happened to split the inputs.
- ▶ The gap between final train accuracy (99.2%) and test accuracy (96.7%) is a mild form of overfitting. With 120 training examples and a network of ~80 parameters, the network has enough capacity to memorise some examples that don't generalise. For IRIS this is harmless; for MNIST it will be the central practical problem we have to solve, with techniques like Dropout, weight decay, and early stopping.

12.7 The seed-variance warning

The seed used to split train/test changes the result more than you might expect:

seed	test accuracy
0	0.9333
1	0.9667
7	0.9333
42	0.8667
123	1.0000

Same model, same training, same data — just a different 30 examples in the test set. On small test sets a single accuracy number is essentially noise. Two implications:

- ▶ Never report a single test accuracy from a tiny test set without noting the variance.
 - ▶ For real evaluation, use **k-fold cross-validation**: split the data into k disjoint folds, train k models leaving one fold out each time, and average the per-fold accuracies. Reduces variance and uses every example for both training and evaluation. We'll add a `kfold` helper in `Data.h` when we hit problems where this variance actually obscures real differences.
-

12.8 What's next

Now that we have a working classifier, the next milestone scales up: **MNIST** (handwritten digits). 60,000 training images $\times$ 784 pixels $\times$ 10 classes. Two things IRIS got away with will start to bite:

- ▶ **Capacity outstrips the data more aggressively**, so we'll need regularisation. $\rightarrow$ **Dropout** (and a training/eval mode flag on `Layer`).
- ▶ **Plain SGD will be slower** than Adam on a problem this size. Both optimisers are already available; the practical win shows up on MNIST.

We can skip k-fold cross-validation: with 10,000 test examples, a single accuracy number is no longer noisy.

After MNIST: orchestral instrument sounds, then the autoencoder, then ConvNets.

So far every layer's update method has been plain stochastic gradient descent baked into the layer itself. Adam (and friends) need per-parameter state, so we need a cleaner separation. This note covers the refactor, the two ideas behind Adam, and why it usually wins.

13.1 Why plain SGD struggles

Plain SGD updates $\text{param} \leftarrow \text{param} - \text{lr} * g$. Two structural weaknesses:

One learning rate for every parameter. Gradient magnitudes can vary by orders of magnitude across parameters of the same network — early layers vs. late layers, weights vs. biases, parameters that interact with sparse vs. dense features. A single learning rate has to be small enough not to blow up on the largest gradients, which means everything else moves too slowly.

No memory between steps. Each step uses only the current minibatch's gradient. A consistent signal pointing one way step after step gets the same treatment as a noisy oscillation; SGD treats every gradient as authoritative even if it's mostly variance.

Adam fixes each with a different idea, then combines them.

13.2 Momentum — accelerate in consistent directions

Track an exponential moving average of the gradient:

$$m_t = \text{beta_1} * m_{t-1} + (1 - \text{beta_1}) * g_t \quad \text{beta_1} \sim 0.9$$

m is the recent average gradient direction. Steps in consistent directions accumulate; oscillating directions cancel and m stays small.

Then update with m_t instead of g_t . Conceptually it gives the optimiser inertia — in a long valley you accelerate, on a noisy plateau you don't get yanked around by individual minibatches.

13.3 RMSProp — per-parameter step sizes

Track an exponential moving average of *squared* gradients, per parameter:

```
v_t = beta_2 * v_{t-1} + (1 - beta_2) * g_t^2    beta_2 ~ 0.999
```

Then scale each parameter's update by $1 / \text{sqrtv}_t$. Parameters with consistently *large* gradients get *small* steps. Parameters with consistently *small* gradients get *large* steps. The optimiser self-calibrates the effective learning rate per parameter.

13.4 Adam = momentum + RMSProp + bias correction

Combine them:

```
m_t = beta_1 * m_{t-1} + (1 - beta_1) * g_t      // 1
    st moment
v_t = beta_2 * v_{t-1} + (1 - beta_2) * g_t^2      // 2
    nd moment
param <- param - lr * m_t / (sqrtv_t + eps)
```

There's one subtlety. m and v start at zero, so for the first few steps they're biased toward zero — $m_1 = (1 - \beta_1) * g_1$ is much smaller than g_1 . Without correction, early steps would be tiny.

The fix is a **bias correction**:

```
m^_t = m_t / (1 - beta_1^t)
v^_t = v_t / (1 - beta_2^t)
param <- param - lr * m^_t / (sqrtv^_t + eps)
```

At $t = 1$, the divisors $(1 - \beta)$ cancel the $(1 - \beta)$ factor in the EMA, so the first-step update is $\text{lr} * \text{sign}(g)$ regardless of gradient magnitude. As t grows, $\beta^t \rightarrow 0$ and the correction factors approach 1 — corrections fade out and the formula becomes the plain EMA. Beautifully clean: a single algebraic factor that smoothly transitions from “bias-correct hard early on” to “trust the EMA later.”

13.4.1 What Adam intuitively does

A concrete way to see why Adam is well-behaved: m / sqrtv has magnitude roughly 1 whenever the gradient direction is consistent (signal), regardless of the gradient's absolute scale. When the gradient is noisy (random direction step to step), m averages to roughly 0 while v stays large, so m / sqrtv shrinks. So **Adam effectively translates “how consistent is this direction?” into a step size**:

- Consistent direction, any magnitude $\rightarrow$ step size $\approx \text{lr}$
- Inconsistent direction $\rightarrow$ step size shrinks toward 0
- Decoupled from raw gradient magnitude

This is why Adam works well across a huge range of problems without careful learning-rate tuning per parameter.

13.4.2 Typical hyperparameters

param	value	what it controls
lr	1e-3	base step size; sometimes 3e-4 for deep nets
beta_1	0.9	EMA decay for the 1st moment (momentum)
beta_2	0.999	EMA decay for the 2nd moment (adaptive lr)
eps	1e-8	numerical guard inside the sqrt

These defaults work surprisingly often. The most-tuned knob is lr.

13.5 The Optimizer abstraction

Adam needs per-parameter state — m , v , and t — and the same optimiser instance has to own that state for *all* of a network's parameters. So update logic doesn't live inside individual layers; it lives in an Optimizer.

```
class Optimizer<T> {
    virtual void step(Matrix<T>& param, const Matrix<T>& grad) =
        0;
};
```

Stateful optimisers key their state on `¶m` (the parameter's address), with lazy allocation on first contact:

```
class Adam : public Optimizer<T> {
    struct State { Matrix<T> m, v; int t = 0; };
    std::unordered_map<Matrix<T>*, State> state_;

    void step(Matrix<T>& W, const Matrix<T>& dW) override {
        State& s = state_[&W];
        if (s.t == 0) { /* lazy init m and v to W's shape */ }
        s.t += 1;
        /* update m, v, then apply bias-corrected step to W */
    }
};
```

A layer's update just hands its parameters to the optimiser, one at a time:

```
void Dense::update(Optimizer<T>& opt) override {
    opt.step(W_, dW_);
    opt.step(b_, db_);
}
```

The layer encapsulates *what* its parameters are; the optimiser decides *how* they get updated. Parameter-less layers (activations) inherit a no-op default. `Network::update` forwards the optimiser to every layer.

13.5.1 In the training loop

```
SGD<float> opt(0.1f); // or Adam<float> opt(1e-3f);
for (int epoch = 0; epoch < epochs; ++epoch) {
    for (auto batch : batches) {
        net.forward(X_batch);
        ce.forward(...);
        net.backward(ce.backward());
        net.update(opt);
    }
}
```

Switching optimisers is a one-line change in user code.

13.6 The empirical comparison

The optimiser test in `tests/test_optimizer.cpp` minimises a poorly-conditioned quadratic:

$$f(x, y) = \frac{1}{2} x^2 + 50 y^2 \quad \text{grad} f = (x, 100 y)$$

The gradient in y is $100\times$ larger than in x . After 100 steps from $(5, 5)$:

optimiser	final loss
SGD (lr = 0.01)	1.67
Adam (lr = 0.1)	0.08

$22\times$ lower loss. SGD's single learning rate has to be conservative enough not to oscillate on y , which leaves x essentially frozen. Adam adapts both axes independently and handles them at once.

On IRIS the difference is less dramatic — the problem is well-scaled and small enough that SGD does fine — but Adam reaches lower training loss faster (epoch 100 train loss: SGD 0.05, Adam 0.02). Both hit $\sim 96.7\%$ test accuracy. The real payoff for Adam shows up on larger, less well-behaved problems; MNIST will start to show the difference more, and a ConvNet on ImageNet-style data would show it dramatically.

13.7 Practical recommendations

- ▶ **Start with Adam at $\text{lr} = 1\text{e-}3$.** Default $\beta_1/\beta_2/\epsilon$. Works almost always for an initial trained model.
 - ▶ **Switch to SGD with a tuned schedule for the last few epochs** if you're squeezing out the final 0.5% of accuracy. Plain SGD + decay often generalises a touch better at the end of training (a real empirical finding, see e.g. "SWA" — Stochastic Weight Averaging). We're not at this level of polish yet.
 - ▶ **The lr for Adam is much less sensitive than for SGD** because the adaptive part normalises gradient magnitudes. The reciprocal-of-this insight is also true: if Adam is going wrong, the learning rate is almost always the first thing to halve.
-

13.8 What's next

The pieces we're missing for MNIST:

- ▶ **Train/eval mode flag** on Layer, so layers like Dropout can behave differently between training and inference.
- ▶ **Dropout layer**: zero out a fraction of activations during training, pass through during inference.
- ▶ **MNIST data loader** for the IDX binary format.

Then we train the MNIST classifier, which is where Adam over SGD will start mattering visibly.

Train/eval mode and Dropout

Two changes that go together. We added a training/inference mode flag to Layer, and we built Dropout — the first layer that actually uses it.

14.1 Why a mode flag exists at all

Most layers compute the same thing during training and during inference. Dense is $WX + b$ either way. ReLU is $\max(0, X)$ either way. The loss function only exists during training, so it doesn't need to care.

But a small family of layers genuinely behaves differently in the two phases:

- ▶ **Dropout** randomly zeroes activations during training, and is the identity at inference time.
- ▶ **Batch normalisation** uses the current batch's mean and variance during training, but uses *running averages* accumulated over training during inference.
- ▶ Some data-augmentation layers (random crops, random noise) similarly only fire during training.

Rather than have each of these layers carry their own flag and force callers to wire it up, the flag lives in Layer and propagates from Network:

```
class Layer<T> {  
public:  
    virtual void set_training(bool t) { training_ = t; }  
    bool training() const { return training_; }  
private:  
    bool training_ = true;  
};
```

Network exposes `train()` and `eval()` that fan the flag out across all layers. The default is training mode, because that's how every freshly-constructed network is used — you only ever flip to eval explicitly, on a validation or test set:

```
net.train();  
for (epoch = 0; ...) { ... training loop ... }  
  
net.eval();  
auto preds = net.forward(X_test);
```

The shape of this API is borrowed from PyTorch (`model.train()`, `model.eval()`) deliberately. Muscle memory transfers.

14.2 Dropout — what and why

Dropout is a regularisation technique introduced by Srivastava et al. (2014). The mechanism is brutally simple:

During training, for each activation, with probability p set it to zero; otherwise scale it by $1/(1-p)$. During inference, do nothing.

The “scale it by $1/(1-p)$ ” trick is called **inverted dropout**, and it’s the reason inference can be a no-op. The expected value of the output equals the input:

$$E[y] = p * 0 + (1-p) * x / (1-p) = x$$

so on average the network sees activations with the same mean during training and during inference. Without the scaling we’d have to scale *down* at inference time instead (multiply activations by $1-p$), which is awkward to remember and easy to forget. Pushing the math into the training path keeps inference clean.

14.2.1 Why it works

The intuition I find most compelling is **implicit ensembling**.

Each training forward pass with a different random mask is effectively training a different sub-network — a randomly chosen subset of the neurons. Over many batches we’re really training an enormous family of sub-networks that share weights. At inference time the full network (no dropout) behaves approximately like an average over all those sub-networks, and ensemble averages are reliably better than any single member.

A second framing that’s sometimes useful: dropout prevents **co-adaptation**. Without dropout, two neurons in the same layer can settle into a brittle “I will detect X *only when my neighbour detects Y*” arrangement. Dropout breaks these dependencies — your neighbour might not be there next batch — and forces each neuron to be individually useful.

Both stories point at the same outcome: a network that overfits less and generalises better, especially when the model has many more parameters than the data really needs.

14.2.2 Common rates

- ▶ 0.5 is the classic Srivastava setting, used on fully-connected layers in big networks.
- ▶ 0.2-0.3 is more common in modern practice.
- ▶ 0.1 or less for layers close to the input.
- ▶ ConvNets typically dropout less than fully-connected nets (and use spatial dropout variants instead — out of scope here).

14.2.3 Where to place it

After a Dense layer, typically after its activation. The pattern is

```
Dense -> ReLU -> Dropout -> Dense -> ReLU -> Dropout -> Dense ->
    Softmax
    ^
    the                               no dropout on
                                     classification
    head
```

You don't put dropout on the output of a softmax classifier — you'd be zeroing class probabilities, which is nonsense. You also rarely put dropout right before the final Dense.

14.3 The code

```
template <typename T = float>
class Dropout : public Layer<T> {
public:
    explicit Dropout(T rate, unsigned seed = std::random_device
    {}())
        : rate_(rate), gen_(seed) {}

    Matrix<T> forward(const Matrix<T>& X) override {
        if (!this->training() || rate_ == T(0))
            return X;

        const T scale = T(1) / (T(1) - rate_);
        std::bernoulli_distribution keep(double(T(1) - rate_));
        mask_ = Matrix<T>(X.rows(), X.cols());
        for (size_t i = 0; i < X.rows(); ++i)
            for (size_t j = 0; j < X.cols(); ++j)
                mask_(i, j) = keep(gen_) ? scale : T(0);

        return hadamard(X, mask_);
    }

    Matrix<T> backward(const Matrix<T>& dY) override {
        if (!this->training() || rate_ == T(0))
            return dY;
        return hadamard(dY, mask_);
    }

private:
    T rate_;
    std::mt19937 gen_;
    Matrix<T> mask_;
};
```

A few things worth noting:

The mask is stored on the layer, exactly like ReLU stores its $x > 0$ mask. The two layers' backward methods are structurally identical — both compute $dY (*) \text{ mask}$. The only difference is how the mask is built.

The random number generator is seeded from `std::random_device` by default, but the constructor accepts an explicit seed, which is what the tests use to get reproducible behaviour. In a real training run you almost certainly want fresh randomness every time, so the default is the right one.

The two early-return paths — eval mode and `rate == 0` — short-circuit to the identity. `rate == 0` is admittedly a degenerate case, but it lets you wire a Dropout layer into a network without committing to using it, and toggle it on later by setting `rate` non-zero. (We don't expose a setter for this yet, but the point stands.)

14.3.1 Testing dropout

We deliberately don't do a numerical gradient check on Dropout, because the forward pass is stochastic — running `forward(X + eps e_ij)` and `forward(X - eps e_ij)` generates *two different masks*, and the gradient check becomes meaningless.

Instead we test the algebraic identity directly. With X all ones, the output Y equals the mask ($Y = X (*) M = M$), so the backward output satisfies $dX(i, j) = M(i, j) * dY(i, j) = Y(i, j) * dY(i, j)$. We check that elementwise. We also verify the scaling-preserves-the-mean property over a 10,000-element matrix, and that `Network::train()` and `Network::eval()` actually propagate the flag down to a Dropout layer embedded in the network.

14.4 Looking ahead

With dropout in place we can sensibly train a network that has many more parameters than the data really supports — which is exactly the situation on MNIST (60k examples, but our networks will have hundreds of thousands of weights). Combined with Adam, this is enough machinery for a respectable MNIST classifier.

The MNIST loader is in the same release as this note; it's a tiny IDX parser that returns a $(784, N)$ matrix of pixels in $[0, 1]$ and a `std::vector<int>` of labels. The actual MNIST classifier example — network architecture, training loop with eval-mode test passes, results — is the next step.

MNIST is the canonical “is your neural network actually working” test — 60,000 training and 10,000 test images of 28×28-pixel handwritten digits, labelled 0-9. It’s small enough to train on a laptop CPU in minutes, and big enough that the regularisation and optimisation choices we made in the last few sessions actually matter.

This is the first weft example that exercises every piece of the library in one go: the IDX loader, an MLP with hidden Dense layers, ReLU, Dropout, Softmax, cross-entropy, Adam, and train/eval mode switching.

15.1 The data pipeline

```
X_train = mnist::load_images<float>(data_dir + "/train-images-idx3-ubyte");
y_train = mnist::load_labels      (data_dir + "/train-labels-idx1-ubyte");
Y_train = one_hot<float>(y_train, 10);
```

`load_images` returns a (784, 60000) matrix — one column per image, pixel values normalised to [0, 1]. This is already a reasonable scale for He-initialised Dense layers, so we skip the Standardizer. (The canonical PyTorch tutorial normalises to mean 0.1307, std 0.3081, but the difference in final accuracy is tiny and the simpler pipeline is worth keeping.)

`load_labels` returns a vector<int> of digit indices. `one_hot<float>` turns that into a (10, 60000) matrix with a single 1 per column.

15.2 The architecture

```
Dense(784, 256) -> ReLU -> Dropout(0.3)
Dense(256, 128) -> ReLU -> Dropout(0.3)
Dense(128, 10)  -> Softmax
```

About 235,000 parameters — large enough relative to 60,000 training examples that overfitting is a real risk, which is exactly the regime where dropout starts paying off.

Why two hidden layers, not one or five? A single hidden layer can in principle approximate any continuous function (universal approximation theorem), but in practice deep narrow networks compose features more efficiently than wide shallow ones. Two hidden layers is enough to get MNIST to ~97-98% without going deep enough that vanishing-gradient issues or training instability start to matter.

Why 256 -> 128, not 256 -> 256 or 128 -> 64? Powers of two are a historical convention (originally cache- and SIMD-friendly), and a shrinking pyramid is a sensible default — early layers extract many low-level features, later layers compress them into class-relevant ones. The exact widths are not load-bearing; $\pm 2\times$ in any of them changes final accuracy by a fraction of a percent.

Why Dropout 0.3 instead of the classic 0.5? 0.5 is what Srivastava’s 2014 paper used on much larger fully-connected networks. With our modest width, 0.3 regularises enough without slowing convergence too much. 0.2 or 0.4 would work nearly as well.

No dropout on the input or the classification head. Dropout right before the softmax would directly zero out class logits, which is nonsensical. Dropout on raw pixels is an option but a weak one — “random crop”-style augmentation is a better fit for the input.

15.3 The optimiser choice

We use Adam, not SGD. We could use SGD with a careful learning rate schedule and probably hit similar accuracy, but Adam is dramatically less fussy to tune. With a single hyperparameter ($\text{lr} = 1\text{e-}3$) and no schedule, it converges to $\sim 97\%$ test accuracy in ten epochs.

Plain SGD at the same learning rate would either diverge or crawl, depending on the value. SGD with momentum at $\text{lr} \approx 0.01$ with a learning rate decay would be competitive, but you’d have two hyperparameters to set well instead of one. The point of Adam for this kind of problem is exactly this: less time tuning, more time understanding.

15.4 The training loop in detail

```
for (int epoch = 1; epoch <= epochs; ++epoch) {
    net.train(); // (a)
    auto idx = shuffled_indices(X.cols(), epoch);

    for (mini-batches) {
        Matrix S = net.forward(X_batch);
        float L = ce.forward(S, Y_batch);
        net.backward(ce.backward());
        net.update(opt);
    }

    net.eval(); // (b)
    auto [test_loss, test_acc] = evaluate(...);
}
```

The two mode switches are the new piece compared to the IRIS example.

(a) puts the network into training mode. Dropout fires, randomly zeroing 30% of each hidden activation. This is the entire point — we deliberately corrupt the activations to force the network to be redundant, knowing the test pass will get the cleaner signal.

(b) puts the network into inference mode. Dropout becomes the identity. The network we evaluate is the full, un-corrupted one.

Without (b), the test accuracy reported each epoch would be measured on a randomly-corrupted network — much lower, and not representative of the model the user actually wants to deploy.

Why mini-batches at this size? Batch 128 is a sweet spot for MNIST. Larger batches (512+) get less stochastic noise per update, which can help final accuracy slightly but means fewer updates per epoch — and modern wisdom is that the noise actually helps generalisation, so the trade-off is roughly even. Smaller batches (16) waste time on per-iteration overhead and don't vectorise well.

15.5 Train vs test accuracy: reading the gap

The reported “train acc” in our table is the running accuracy *during the training pass* — that is, measured with dropout still active. The “test acc” is measured in eval mode.

It's common, especially early in training, for **test accuracy to exceed train accuracy**. This isn't a bug — it's exactly what dropout is designed to do. The network has to make correct predictions despite having 30% of its activations zeroed; the test pass gives it the full network and it does noticeably better.

Later in training, as the network starts to overfit, train accuracy (even with dropout) will catch up and eventually surpass test accuracy. The point at which it crosses over and starts to pull away is the point at which you'd consider stopping training or strengthening regularisation. Watching the gap is a basic diagnostic skill.

15.6 Running it

You'll need the four uncompressed MNIST IDX files. The download script in `data/` fetches them from a reliable mirror:

```
chmod +x data/download_mnist.sh
./data/download_mnist.sh
```

Then from `build/`:

```
make
./bin/examples/mnist_classifier          # uses ../data/
mnist
./bin/examples/mnist_classifier /some/path # or pass a path
```

Expected output: ~97% test accuracy after ten epochs, in a few minutes of CPU time on a modern laptop.

15.7 What’s missing (worth knowing)

A handful of things a “serious” MNIST classifier would do that we deliberately don’t:

- ▶ **Learning rate schedule.** Adam adapts per-parameter, but a global step-decay or cosine schedule on top can squeeze another half-percent of accuracy. Not worth the complexity here.
- ▶ **Data augmentation.** Random shifts of ± 2 pixels and small rotations are the classic MNIST trick — they reliably take an MLP from ~98% to ~99%. Requires an image-aware augmentation layer we don’t have.
- ▶ **Validation set.** We’re using the test set for per-epoch evaluation, which is methodologically lax — in a published result you’d hold out a slice of training for validation and only touch the test set once at the end. For a learning project that’s pedantry.
- ▶ **Model checkpointing.** Save the best-test-accuracy model rather than the last-epoch one. Easy to add when there’s a reason to.
- ▶ **ConvNets.** The big jump from ~98% MLP to ~99.5%+ comes from exploiting the 2D structure of the image with convolutional layers instead of flattening to 784. That’s a roadmap item — needs 4D tensors, Conv2D, and MaxPool.

The accuracy ceiling for a well-tuned MLP on MNIST is around 98-98.5%. The state of the art (deep ConvNets with augmentation and ensembling) sits at 99.8%, where the remaining errors are images even humans disagree about.

We're about to do audio. Step one is being able to look at sound in the frequency domain — what mix of sinusoids is in this signal? — and the tool for that is the discrete Fourier transform. The FFT (fast Fourier transform) is the same transform, computed cleverly enough to be useful in practice.

16.1 The DFT

The discrete Fourier transform of an N -sample complex signal is

$$X[k] = \sum_{n=0}^{N-1} x[n] * \exp(-2\pi i * k * n / N) \\ k = 0, \dots, N-1$$

Read it like this: bin $X[k]$ measures how much of the signal looks like a complex sinusoid at frequency k/N cycles per sample. The magnitude $|X[k]|$ says “how much of that frequency is here”, and the phase $\arg(X[k])$ says “where its peaks line up in time”.

For a real audio signal sampled at 44.1 kHz with $N = 4096$, bin k corresponds to frequency $k * 44100 / 4096 \sim k * 10.77$ Hz. Bin 0 is DC; bin 2048 is Nyquist (22.05 kHz); bins 1..2047 are positive frequencies; bins 2049..4095 are the mirror image (negative frequencies) which carry no extra information for real input. That's why we only keep $N/2 + 1$ bins downstream.

16.2 Why we don't compute the DFT directly

The formula above is an $O(N^2)$ algorithm: N output bins, each a sum over N inputs. For $N = 4096$ that's about 17 million complex multiplies *per FFT*. With 5,000 audio files and tens of frames per file, the cost balloons fast.

The FFT computes the same answer in $O(N \log N)$ operations. For $N = 4096$, that's about 50,000 complex multiplies instead of 17 million — a 350× speedup that's the entire reason audio DSP is practical.

16.3 The Cooley-Tukey trick

The key observation: a length- N DFT can be expressed as two length- $N/2$ DFTs glued together. Specifically, separate x into its even-indexed and odd-indexed samples,

$$E[k] = \text{DFT of } x[0], x[2], x[4], \dots \quad (\text{length } N/2) \\ O[k] = \text{DFT of } x[1], x[3], x[5], \dots \quad (\text{length } N/2)$$

Then for $k = 0 \dots N/2 - 1$:

$$\begin{aligned} X[k] &= E[k] + w^k * O[k] \\ X[k + N/2] &= E[k] - w^k * O[k] \end{aligned}$$

where $w = \exp(-2\pi i / N)$ is the “twiddle factor”. Each step combines two half-sized DFTs into a full one in $O(N)$ extra work, so the total cost satisfies

$$T(N) = 2 * T(N/2) + O(N) \quad \Rightarrow \quad T(N) = O(N \log N)$$

If N is a power of two, we recurse all the way down to $N = 1$ (where the DFT is the identity). That’s the algorithm.

16.4 From recursive to iterative

The recursive version allocates new arrays at every level, which is slow. The iterative version observes that all of those even/odd splits can be done up front: if you reorder the input by reversing the bits of each index, then the recursion’s base cases sit in consecutive memory slots, and you can build up the full transform by pairing adjacent slots and growing the pair size.

That’s what’s in `FFT.h`:

```
1. Bit-reverse permutation:  $O(N)$ 
2. for len = 2, 4, 8, ..., N:
    for each block of ‘len’ samples:
        combine two halves with twiddle factors
```

The outer loop runs $\log_2(N)$ times; the inner loops together touch each sample once per stage. Total: $N/2 * \log_2(N)$ complex multiplies, $N * \log_2(N)$ complex adds, no allocations.

16.5 The inverse

The IDFT differs from the DFT by a sign in the exponent and a $1/N$ factor:

$$x[n] = (1/N) * \sum_{k=0..N-1} X[k] * \exp(+2\pi i * k * n / N)$$

Rather than write a near-duplicate of `fft()` with the sign flipped, we use a small identity:

$$\text{ifft}(x) = (1/N) * \text{conj}(\text{fft}(\text{conj}(x)))$$

Conjugating flips the sign of every imaginary component, which is exactly the sign change in the exponent. So three lines and a reuse of the forward FFT is all we need.

16.6 What we'll actually use it for

In the instrument classifier:

1. Cut each WAV file into overlapping frames (~4096 samples each).
2. Multiply each frame by a window function (Hann) to taper the edges to zero — without this, the frame boundaries look like step discontinuities and smear energy across the spectrum.
3. Lift the windowed frame from real to complex (zero imaginary part) and FFT it.
4. Take magnitudes of the first $N/2 + 1$ bins.
5. Average across frames, weighted by per-frame energy, to get one “spectral envelope” per file.

The classifier then learns instrument identities from those envelopes. Steps 1-5 are the subject of the next note; this one is just the FFT itself, and the tests confirm it correctly returns:

- ▶ the DC magnitude N for a constant input,
- ▶ magnitude peaks at bins k and $N-k$ for a cosine at frequency k/N ,
- ▶ conjugate-symmetric output for real input,
- ▶ the input back when fed through `ifft(fft(.))`.

Anything that depends on FFT correctness — and lots of things will — rests on those.

Loading audio and extracting features

17

Two things are needed to feed audio into the classifier: a way to read sample data from a file, and a way to turn an arbitrary-length sample stream into a fixed-length feature vector that summarises its timbre well enough for a classifier to learn from.

This note covers both, in pipeline order: WAV parsing, then framing and windowing, then the two feature flavours we expose (log-magnitude spectrum and MFCC).

17.1 The WAV format and our reader

WAV is a RIFF (Resource Interchange File Format) container — a file built out of tagged chunks. Every chunk has the same three-part structure:

- ▶ a 4-byte FourCC identifier (e.g. RIFF, fmt, data — yes, with the trailing space on `fmt`)
- ▶ a 4-byte little-endian size of the chunk body
- ▶ the body itself

A minimal valid WAV is a single outer RIFF chunk wrapping the whole file. Its body starts with the 4-byte string `WAVE`, then contains sub-chunks. The two we care about:

- ▶ `fmt` — format metadata: PCM vs float vs compressed, number of channels, sample rate, bits per sample
- ▶ `data` — the actual samples

The naive parser would assume `fmt` is followed by `data` and read both at fixed offsets. In practice, real-world WAVs often interleave metadata chunks: `LIST` carrying ID3-style track info, `JUNK` chunks inserted for alignment, and more. Our reader walks every sub-chunk in order, processes `fmt` and `data` when it sees them, and skips anything else by seeking past its body. This is a few extra lines and makes the parser robust to whatever editor produced the file.

17.1.1 Endianness

RIFF is a Microsoft format that originated on x86, and the spec is explicit: all multi-byte values are little-endian. To keep the parser correct regardless of the host, we assemble multi-byte fields byte by byte rather than reading directly into a struct or integer:

```
std::uint32_t u = std::uint32_t(b[0])
                | (std::uint32_t(b[1]) << 8)
                | (std::uint32_t(b[2]) << 16)
                | (std::uint32_t(b[3]) << 24);
```

99% of the machines anyone runs in 2026 are little-endian and could read directly, but the explicit reconstruction is the same number of lines and works everywhere — including, hypothetically, on a big-endian SPARC, on a PowerPC G5 nobody has touched in fifteen years, or on whatever embedded thing decides to be old-school in the future.

17.1.2 What we actually accept

The `fmt` chunk's first field is a format code that identifies how to interpret the data chunk:

- ▶ 1 — linear PCM (raw signed integer samples)
- ▶ 3 — IEEE float samples
- ▶ 6, 7 — A-law / μ -law (telephony compression)
- ▶ 0xFFFF — `WAVE_FORMAT_EXTENSIBLE` (anything else, disambiguated by a GUID at the end of the `fmt` chunk)
- ▶ many more — various compressed codecs

We accept only code 1 with 16 bits per sample. SOL files are 16-bit PCM, so that covers the use case. Every additional code we'd accept is a dispatch path that needs its own decoding — for float WAVs, scale by the right factor and no conversion from int; for 24-bit, three bytes per sample with sign extension; for `WAVE_FORMAT_EXTENSIBLE`, read and match against the PCM GUID. We'd be writing parsers for cases we don't need. If the file isn't what we expect, the loader throws.

Samples are signed 16-bit integers in `[-32768, +32767]`. We divide by 32768 to land in `[-1, 1]` — the conventional range for downstream DSP.

17.1.3 Stereo to mono

If the file is stereo, the data chunk holds interleaved L, R, L, R, ... samples. We average them to produce a mono signal. Anything more sophisticated (channel selection, mid-side decoding) wouldn't help instrument classification: the spectral envelope of a sustained note is essentially identical in both channels.

The reader returns a `WavData` struct with the mono samples as a `std::vector<float>` and the sample rate. That's everything the feature extractors need.

17.2 The framing pipeline

The FFT gives us a spectrum, but a single FFT of a whole audio file is useless. A 1-second piano note isn't a stationary signal — it has an attack with broadband energy, then a steady tone with harmonics, then a decaying release. Averaging the whole thing into one FFT smears all of that together.

What we want is *one feature vector per file* that summarises the character of the sound well enough to classify. We get there by analysing short frames,

then averaging those analyses with energy weighting so the silent decay doesn't pollute the spectrum of the attack.

The structure is the same regardless of which feature we compute:

```
for each frame at offsets 0, hop, 2*hop, ...:
    multiply frame by Hann window
    compute frame energy = sum of squared windowed samples
    compute frame-level feature (this is what differs)
    accumulate feature, weighted by energy
divide accumulated feature by total energy
```

Three parameters: `frame_size` (typically 4096 samples at 44.1 kHz, around 93 ms), `hop_size` (typically half of `frame_size`, giving 50% overlap), and the per-frame feature function.

17.2.1 Why a window function?

If we just chop the signal into 4096-sample chunks and FFT them, each chunk effectively gets multiplied by a rectangular window. A rectangular window has terrible frequency-domain behaviour: a pure sine at frequency f shows up in the FFT as a wide sinc-shaped smear, not a clean peak. This is **spectral leakage**, and it ruins everything downstream.

The Hann window — $w[i] = 0.5 * (1 - \cos(2\pi i / (N-1)))$ — tapers smoothly to zero at both endpoints. The frequency-domain side-lobes are roughly 30 dB lower than a rectangular window's, which means leakage becomes a small effect instead of a dominant one. There are other windows (Hamming, Blackman, Kaiser) with different tradeoffs between main-lobe width and side-lobe height; Hann is the standard default and fine for our purposes.

17.2.2 Why energy-weighted averaging?

A note's character is in its loud parts. The release of a piano note, where the energy has decayed by 40 dB, isn't carrying much instrument information — it's mostly room reverb and noise. If we did a plain mean across frames, those quiet tails would dilute the signal.

Weighting each frame by its energy (sum of squared windowed samples) puts essentially all the contribution into the loud frames. Frames below an energy threshold are skipped entirely, which also handles leading and trailing silence.

17.3 Log-magnitude spectrum

The simpler of the two features. For each frame:

1. FFT the windowed frame.
2. Take magnitudes of the first $N/2 + 1$ bins.

3. Take $\log(1 + |X[k]|)$ to compress the dynamic range.

The $1 +$ inside the log lets us handle silent bins without $\log(0)$; since we already skip whole silent frames, this is belt-and-braces.

Output dimension: $N/2 + 1$. For $N=4096$ that's 2049 features per file — a high-resolution snapshot of the spectral envelope.

The log compression is the load-bearing trick. A typical instrument's fundamental might be 10^4 times larger than its 10th harmonic, and a neural network whose first layer is fed the raw magnitudes will pay attention to the fundamental and effectively ignore the rest. With log, that 10^4 ratio collapses to a difference of $\log(10^4) \sim 9.2$, and the high harmonics become first-class citizens in the feature vector.

This feature is large but expressive. Good as a baseline.

17.4 MFCC: Mel-Frequency Cepstral Coefficients

A more compact and historically standard feature. Four steps per frame:

1. **FFT and power spectrum.** Square the magnitudes to get power.
2. **Mel filterbank.** Apply a set of triangular filters spaced on the mel scale, summing the power within each filter. Produces a short-dimensional "mel spectrum" (e.g. 40 numbers per frame).
3. **Log.** Take log of each mel-spectrum value.
4. **DCT-II.** Apply a discrete cosine transform and keep only the first 13 coefficients.

We then accumulate (energy-weighted average) across frames to get one 13-dimensional vector per file.

17.4.1 The mel scale

Humans don't perceive pitch linearly. The difference between 100 Hz and 200 Hz sounds like a full octave; the difference between 5000 Hz and 5100 Hz is barely perceptible. The mel scale is a perceptual remapping that approximates "equal mel distance feels like equal pitch distance":

$$\text{mel}(\text{hz}) = 2595 * \log_{10}(1 + \text{hz} / 700)$$

So 700 Hz $\rightarrow$ 781 mel, 1400 Hz $\rightarrow$ 1336 mel (not 1562 — the spacing shrinks as frequency grows).

For audio classification, putting features on the mel scale is a way of saying "we care more about distinguishing 200 Hz from 400 Hz than 8000 Hz from 8200 Hz", which matches what's actually informative about instrument timbre.

17.4.2 The mel filterbank

Concretely: pick a number of filters (40 is standard), space their centre frequencies evenly in mel space across the frequency range, and make each filter a triangle that's 1 at its centre and 0 at the centres of its neighbours. Sum the FFT power within each filter to get that filter's energy.

The output is a 40-dimensional summary of the power spectrum, biased toward lower frequencies (where the filters are narrower and more numerous) — exactly the bias human ears have.

17.4.3 Why the DCT at the end?

The 40 log-mel-energies are strongly correlated — adjacent filters overlap, and slow trends in the spectrum show up across many bins. The DCT changes basis to one where slow trends are captured by the first few coefficients and faster oscillations by later ones. Keeping the first 13 throws away the high-frequency detail (which is mostly noise) and keeps the smooth shape (which is mostly timbre).

It's worth noting that modern audio neural networks often skip the DCT entirely — they feed the log-mel-spectrogram straight to a NN and let the network learn the decorrelation. We do the DCT here because (a) it makes the feature 13-dim instead of 40-dim, which keeps the downstream MLP small, and (b) it's pedagogically interesting: the “cepstrum” (DCT of log spectrum) is the historical foundation of audio analysis going back to speech recognition in the 1960s.

17.5 Averaged vs per-frame output

Both feature functions come in two flavours. The “averaged” forms (`logmag_spectrum`, `mfcc`) collapse the whole signal into a single energy-weighted vector. The “per-frame” forms (`logmag_spectrum_frames`, `mfcc_frames`) keep one vector per non-silent frame and return a list of them.

Same windowing, same FFT, same mel filterbank, same DCT — the only difference is whether the per-frame results get averaged at the end or kept separate.

When to use which:

- **Averaged** — when you want one feature per file, the data is short enough that “what does this sound like on average” is meaningful, and the file count itself is enough training signal. Compact, fast.
- **Per-frame** — when you want to (a) treat each frame as its own training example so the network sees temporal variation, (b) reuse the output for a per-frame application like an autoencoder that reconstructs frame-by-frame, or (c) aggregate downstream predictions per file by averaging softmax outputs across frames at eval time (ensembling).

For SOL specifically, per-frame extraction roughly $20\times$ the number of training rows (single-note files at 44.1 kHz, frame 4096 hop 2048 give ~ 20 frames per file). The downstream classifier needs to (a) split by `file_id` rather than by row to avoid frames-from-the-same-file leaking across train and test, and (b) aggregate per file at eval time. Both are handled by `group_train_test_split` in `Data.h` and per-file aggregation in the classifier.

Per-frame is also what the autoencoder will train on: each frame is one training example, the encoder learns a compact representation of “plausible SOL spectra,” and the decoder inverts.

Note that the per-frame functions skip silent frames (energy below a small threshold) so we don’t train on noise from leading/trailing silence. The averaged version effectively does the same thing via energy weighting.

17.6 Which feature to choose?

For the instrument classifier, both are worth trying:

- ▶ **logmag_spectrum** keeps everything. 2049 features per file. With ~ 5000 SOL files and a `Dense(2049, 256)` first layer, that’s around 500K weights from the input alone. Risk of overfitting; dropout and standardisation help.
- ▶ **mfcc** is a much smaller representation. 13 features per file. The classifier is tiny — `Dense(13, 64) → Dense(64, K)` is enough. Much faster to train. Likely loses some discrimination ability because of the compression.

Both are exposed in the example and you can toggle with a CLI flag. My guess: logmag will be 2-5 percentage points more accurate, but MFCC will train in seconds. Both should comfortably clear chance accuracy by a wide margin. Per-frame mode lifts both by another 5-10 points through file-level ensembling at eval time.

17.7 What’s not in here (yet)

A real audio pipeline usually adds several things we’re skipping:

- ▶ **Pre-emphasis filter** (a simple high-pass that flattens the spectrum before framing). Adds $\sim 1\%$ accuracy on speech tasks; less clearly useful for music.
- ▶ **Delta and delta-delta MFCCs** (time derivatives of MFCC across frames). Captures dynamics; not relevant when we average everything to a single vector anyway.
- ▶ **Constant-Q transform** (logarithmically-spaced frequency bins instead of mel triangles on linear bins). Stronger pitch invariance than mel; more complex to implement.

- ▶ **Spectrogram features for ConvNets** (don't average at all — keep the time axis as a 2D image). The next big architectural milestone for weft.
- ▶ **Float, 24-bit, or compressed WAVs.** Real-world audio libraries add the dispatch logic; for SOL specifically we can get away without it.

For SOL classification, 16-bit PCM in plus energy-weighted log-mag spectrum or MFCC out is enough to get a representative result.

Musical instrument classifier (SOL)

18

Putting the audio pipeline to work. SOL_flat is ~5000 WAV files, each a single note from an orchestral instrument; the filename's first dash-delimited token is the instrument class. The job is to predict the instrument from the audio.

This is the first weft example that builds its own dataset from raw files, and it's split across **two tools** so the slow feature-extraction step doesn't repeat on every classifier iteration.

18.1 Why two tools

Feature extraction is the slow part. Each WAV file gets framed, windowed, FFTed several dozen times, then (for MFCC) run through the mel filterbank and DCT. Across 5000 files this takes a couple of minutes on a laptop. The actual neural-network training on the extracted features takes a few seconds at most.

If we kept everything in one binary, every iteration on the classifier would re-do the slow extraction first. Wasteful.

So the workflow splits:

```
extract_features <data_dir> <output.feats> [logmag|mfcc] [
    averaged|per_frame]
sound_classifier <feature_file>
```

You run the extractor once per (dataset $\times$ feature_type $\times$ mode) combination, then iterate on the model as many times as you like. This is the standard ETL-then-train pattern that essentially every real ML project ends up with.

18.2 Two extraction modes

18.2.1 Averaged (one feature per file)

The original mode: STFT, energy-weighted average across frames, one fixed-length vector per file. Compact, fast, and the baseline result. Network architecture is sized to the feature dimension.

18.2.2 Per-frame (one feature per non-silent frame)

For each WAV file we keep every non-silent STFT frame as its own training example. Same feature dimension as averaged mode (2049 for logmag, 13 for MFCC), but $\sim 20\times$ more rows.

Why it matters:

- ▶ More training data per file $\rightarrow$ the network's parameters are better constrained, less overfitting.
- ▶ The network can learn from the instrument's temporal evolution (attack vs sustain vs release) rather than only seeing the time-averaged spectrum.
- ▶ At inference, predictions are aggregated across all frames of a test file by averaging softmax outputs and taking argmax. This is ensembling and typically buys 5-10 percentage points on its own.
- ▶ The same per-frame infrastructure is what we'll need for the autoencoder / morphing work.

Trade-off: cache file is $\sim 20\times$ larger, training is $\sim 20\times$ more iterations per epoch (though each iteration is the same cost since the architecture is unchanged).

For real SOL the averaged mode plateaus around 75% test accuracy with log-mag features; per-frame typically pushes that into the 85%+ range, with much of the gap coming from per-file ensembling rather than the larger training set.

18.3 The file-leakage problem (and the fix)

Per-frame mode introduces a subtle issue: frames from the same file are not independent. They share most of their spectral character. If you split frames at random into train/test, frames from the same SOL file end up on both sides, and the network can essentially "memorise" specific files at frame granularity. Test accuracy looks inflated.

The fix is to split **by file_id**, not by frame: choose 20% of files to be test, all of their frames go to test, none of their frames go to train. The cache stores a `file_id` per row exactly for this purpose, and `group_train_test_split` in `Data.h` does the file-aware split.

For averaged mode this distinction doesn't matter — each row is its own file by construction — but the same function handles both cases uniformly (in averaged mode `file_ids` are `{0, 1, 2, ...}` and the behaviour collapses to a regular random split).

18.4 Per-file aggregation at eval time

Once the network is trained, evaluating per-frame mode means:

1. Forward-pass all test frames through the network in batches.
2. Group the resulting softmax outputs by `file_id`.
3. Average the softmax outputs across frames of each file.
4. Argmax on the averaged vector to get a per-file prediction.
5. Compare to the file's true label.

This is a textbook example of test-time ensembling. Each frame gives a (slightly noisy) probability distribution over classes; averaging across all frames of a file cancels out frame-level noise and gives a much sharper file-level prediction.

The reported “test acc (per file)” reflects this aggregation.

18.5 Confusion matrix

After training, we print a $K \times K$ confusion matrix over the test set: rows are true class, columns are predicted class. The diagonal shows correct counts; off-diagonal entries show specific confusions.

This is much more informative than a single accuracy number. With ~15 instruments you see, for example:

- ▶ Violin and viola confused with each other at high rate → structural problem with this representation (their overlapping pitch ranges + similar spectral envelopes are nearly indistinguishable in averaged-over-time features)
- ▶ Brass instruments cross-confused but not confused with strings → the network learned the broad timbre family but is fuzzy on individuals
- ▶ Scattered random off-diagonals → mostly model noise

For SOL specifically, the dominant confusions are within instrument families. That's both diagnostic and pedagogically interesting: a human listener would have the same trouble with isolated notes.

18.6 The cache format (v2)

`FeatureCache.h` provides `save_features` and `load_features`. The file format:

```

"WFED" magic (weft feature dataset)
uint32 version = 2
uint32 feature_dim
uint32 n_samples (or n_frames in per_frame mode)
uint32 n_classes
uint32 per_frame (0 or 1)
length-prefixed string feature_type ("mfcc", "logmag", ...)
length-prefixed strings class_names[] (n_classes of them)
float[feature_dim * n_samples] column-major
int32[n_samples] labels
int32[n_samples] file_ids

```

Header integers are little-endian (assembled byte-by-byte, host-endian-agnostic). The bulk payload is native byte order — if you extract on one machine and load on a different-endianness machine, the floats will be scrambled. Don't do that.

Version was bumped from 1 to 2 to add `per_frame` and `file_ids`. v1 caches can't be loaded; just re-run `extract_features`.

18.7 What to expect from the four combinations

feature	mode	typical test accuracy	training time
mfcc	averaged	70-80%	seconds
mfcc	per_frame	80-88%	< 30s
logmag	averaged	75-85%	~30s
logmag	per_frame	85-92%	~3 min

(Per-frame logmag is the slowest because of the 2049-dim Dense first layer $\times 20\times$ more frames. Still very feasible on a laptop.)

The gap from averaged $\rightarrow$ per_frame is mostly per-file ensembling working as intended. The gap from mfcc $\rightarrow$ logmag is the higher-resolution feature giving the network more to work with.

18.8 What's still not in here

- **CQT (Constant-Q Transform)** — logarithmically-spaced bins for better pitch invariance. Genuinely worth adding; would slot in as a third feature type alongside logmag and mfcc.
- **Per-class weighting** — if some instruments have many more samples than others, the loss is dominated by majority classes.
- **Data augmentation** — pitch-shifting, time-stretching, additive noise. Standard moves to lift accuracy further.

- **Spectrogram-based ConvNet** — instead of averaging-or-aggregating frame-level features, feed the full (frames $\times$ bins) spectrogram to a 2D ConvNet. The biggest jump available, and the natural way to handle time in audio. On the roadmap once ConvNets are built.

The current pipeline composes everything else we have — WAV I/O, FFT, windowing, mel filterbank, DCT, Adam, dropout, train/eval mode, standardisation, group-aware splitting, a small binary serialisation format — into a working end-to-end classifier you can iterate on.

Everything so far has been *supervised*: we hand the network an input and a label, and it learns the mapping. An autoencoder throws the labels away. You give it an input and ask it to reproduce that same input on the output — but force the signal through a narrow middle layer first, so it can't just copy. To reconstruct well through a bottleneck, the network has to discover the underlying structure of the data and encode it compactly. That's representation learning, and it's the doorway to generative models.

This note covers the two small primitives the autoencoder needs (MSE loss and the Sigmoid activation), the autoencoder itself, and the demo: reconstructing MNIST digits and morphing one digit into another through the latent space. The demos are written out as BMP images via a small `Bmp.h` writer (uncompressed 24-bit BMP — two headers and raw pixels, the image-format equivalent of the WAV reader), so we can actually look at the results instead of squinting at ASCII art.

19.1 MSE loss

For classification we used cross-entropy, which expects a probability distribution. For an autoencoder reconstructing pixel intensities, the natural objective is squared error: how far is each output pixel from the corresponding input pixel?

$$L = (1 / 2N) * \text{sum over examples, sum over features } (P - T)^2$$

P is the prediction, T the target, N the batch size. We sum over features (the 784 pixels) and average over examples — same convention as cross-entropy, where the per-example averaging factor $1/N$ lives in the loss so the layers never need to know the batch size.

The factor of $1/2$ is a convenience: differentiating the square brings down a 2 that cancels it, leaving a clean gradient:

$$dL/dP = (1/N) (P - T)$$

That's the whole loss. The gradient is literally the (averaged) error itself — push each output toward its target in proportion to how wrong it is. `tests/test_mse.cpp` checks the known value, the gradient formula, and a numerical-gradient agreement.

One thing worth noticing: because we sum over 784 pixels, the loss *value* looks large (tens, early in training) compared to the classifier's cross-entropy (single digits). That's just the summing-over-features convention — what matters is that it decreases. Divide by $2 * 784$ and take the square root to get a per-pixel RMS error in the same $[0, 1]$ units as the pixels.

19.2 Sigmoid activation

MNIST pixels live in $[0, 1]$. If the decoder's last layer were linear, it could output any real number, including negatives and values above 1 — nonsense as pixel intensities. The logistic sigmoid squashes any real input into $(0, 1)$:

$$\text{sigma}(x) = 1 / (1 + e^{-x})$$

so the reconstruction is always a valid intensity. Its derivative has a famously tidy form:

$$\text{sigma}'(x) = \text{sigma}(x) (1 - \text{sigma}(x))$$

Since the forward pass already computed $Y = \text{sigma}(Z)$, the backward pass reuses Y directly: $dZ = dY * Y * (1 - Y)$, element-wise. Like ReLU, it's an element-wise activation, so the Jacobian is diagonal and backward collapses to a Hadamard product. No parameters.

(We'll meet sigmoid again as the gate nonlinearity if we ever build recurrent layers, and the same logistic shape is the binary case of the softmax we already have.)

19.3 What an autoencoder is

The architecture is two halves:

```
encoder:  784 -> 128 -> ReLU -> 32 -> ReLU -> 16      (the
           bottleneck)
decoder:  16 -> 32 -> ReLU -> 128 -> ReLU -> 784 -> Sigmoid
```

The input (784 pixels) is squeezed down to a 16-number *latent code*, then expanded back to 784 pixels. The loss is MSE between the output and the original input. There are no labels anywhere in the training objective — the data supervises itself.

The bottleneck is the whole point. 16 numbers cannot store 784 independent pixels, so the network is forced to find what's redundant. MNIST digits are not random pixel soup; they live on a low-dimensional manifold (strokes, curves, a handful of degrees of freedom). The autoencoder discovers a coordinate system for that manifold. The encoder maps an image to its coordinates; the decoder maps coordinates back to an image.

Note the bottleneck layer itself is **linear** (no activation). We want the latent code to be able to take any real value in any direction; squashing it would distort the geometry we're about to exploit for interpolation.

19.4 Why two Networks instead of one stack

A plain autoencoder is just a deep stack, so we *could* express it as a single Network from 784 to 784. But building it as two separate Networks — an encoder and a decoder — lets us call the halves independently:

- ▶ `enc.forward(x)` → the latent code for an input
- ▶ `dec.forward(z)` → the image for a latent code

That separation is what makes the generative demos possible. With a single stack you'd only ever get input-to-output; you couldn't reach in and grab the latent code or feed the decoder a code of your own choosing.

Training across two Networks needs no new library code — the latent just flows between them:

```
Matrix z      = enc.forward(Xb);           // 784 -> 16
Matrix recon  = dec.forward(z);           // 16  -> 784
float L       = mse.forward(recon, Xb);    // target = the input
            itself

Matrix dRecon = mse.backward();            // dL/d recon
Matrix dz     = dec.backward(dRecon);      // dL/d z, updates
            decoder grads
enc.backward(dz);                          // updates encoder grads
            (return ignored)

dec.update(opt);
enc.update(opt);
```

The chain rule runs straight through the boundary: the decoder's backward returns the gradient with respect to its input (the latent code), which is exactly the gradient the encoder's backward needs as its incoming signal. One Adam instance optimises both networks — it keys its per-parameter state on each weight matrix's address, so it happily tracks parameters from any number of networks at once.

19.5 Demo 1 — reconstruction

After training, we run a few test digits through `enc` then `dec` and write the originals and their reconstructions to a BMP image (`ae_reconstructions.bmp`) using the `Bmp.h` writer: top row the originals, bottom row the reconstructions, each digit scaled up so it's comfortable to look at. A well-trained autoencoder produces a slightly blurry but clearly recognisable copy. The blur is the bottleneck at work — fine detail is exactly the high-dimensional information that 16 numbers can't retain, so the network keeps the broad strokes and drops the rest. A pleasant side effect: because the noise in any single pixel is unpredictable from the rest of the image, it can't survive the bottleneck either, so reconstructions come out *denoised* relative to their inputs.

19.6 Demo 2 — latent interpolation

This is the demo that shows the latent space is *meaningful* rather than just a compression trick. Take two test digits, say a 3 and an 8. Encode both to get codes z_a and z_b . Now walk a straight line between them in latent space and decode each point:

$$z(t) = (1 - t) z_a + t z_b, \quad \text{for } t = 0, 0.2, 0.4, \dots 1.0$$

Decoding each $z(t)$ produces a sequence of images that smoothly morph the 3 into the 8, written side by side into `ae_interpolation.bmp`. The intermediate images are *not* averages of the two pictures (that would just be a double-exposure ghost) — they are plausible digit-like shapes that the decoder generates from points the encoder never actually visited during training. The network has learned a continuous space where “between a 3 and an 8” is a coherent place.

That continuity is the bridge to generation. If decoding arbitrary points in latent space yields plausible outputs, then we can *sample* new points and get new outputs — new digits, or in the audio version, new timbres. The plain autoencoder gets us most of the way there, but with a catch we’ll fix next.

19.7 The catch, and what’s next

A plain autoencoder’s latent space has no enforced shape. The encoder is free to scatter codes wherever is convenient — clusters here, empty voids there, arbitrary scales on each axis. Interpolating between two *real* encoded points usually works because the path tends to stay near the data. But if you sample a random latent code with no real image behind it, you’ll often land in one of the voids and the decoder produces garbage. The space is good for reconstructing and interpolating known points, not for sampling fresh ones.

The **variational autoencoder** (next note) fixes this by forcing the latent space into a known, gap-free shape — a unit Gaussian — so that *any* sample from that distribution decodes to something plausible. That requires two new ideas: encoding a distribution instead of a point (the reparameterisation trick), and a second loss term that pulls the latent distribution toward the standard normal (the KL divergence). Once that’s in place, generation is just: draw z from a normal, decode, done.

And once generation works on MNIST, the same machinery points straight at the audio goal: train the encoder/decoder on per-frame spectra instead of pixels, and the latent space becomes a space of timbres to interpolate through and sample from.

The plain autoencoder learned to compress and reconstruct, and its latent space was continuous enough that interpolating between two *real* encoded points produced sensible blends. But it had a flaw we flagged at the end of note 13: nothing forced the latent space into any particular shape. The encoder scattered codes wherever was convenient, leaving voids between the clusters. Interpolating between known points works because the path stays near the data, but *sample* a random latent code and you usually land in a void and decode garbage. The space is good for reconstructing and interpolating known points, useless for generating new ones.

The variational autoencoder fixes exactly this. The architecture is almost the same; the changes are conceptual, and there are three of them.

20.1 Encode a distribution, not a point

Instead of mapping an image to a single latent code, the VAE's encoder maps it to a whole Gaussian distribution over latent space — a mean μ and a variance σ^2 , one of each per latent dimension. The idea is that an input shouldn't claim a single infinitely-precise point; it should claim a little fuzzy region. During training we draw a sample from that region and decode it, so the decoder is forced to produce something sensible not just at one point but across the whole neighbourhood. That is what makes nearby points in latent space decode to similar images — continuity, by construction.

In code, the encoder simply emits $2 * \text{LATENT}$ numbers instead of LATENT . We slice the output: the first LATENT rows are μ , the next LATENT rows are $\log\text{var}$.

Why $\log\text{var}$ (the log of the variance) and not the variance directly? A variance must be positive, which would mean constraining the network's output. The logarithm is unconstrained — any real number is a valid log-variance — so the network can output it freely, and we recover a guaranteed-positive standard deviation with $\sigma = \exp(0.5 * \log\text{var})$.

20.2 The reparameterisation trick

We need to draw $z \sim N(\mu, \sigma^2)$ and then backpropagate through that draw to train μ and σ . But sampling is random — there's no derivative of "draw a random number" with respect to the distribution's

parameters. If the random node sits directly in the path from parameters to loss, gradients can't pass.

The trick moves the randomness out of the path. Instead of sampling z directly, draw a *standard* normal $\text{eps} \sim \mathcal{N}(0, \mathbf{I})$ — which depends on no parameters at all — and compute

```
z = mu + sigma * eps
```

This z has exactly the distribution we wanted ($\mathcal{N}(\mu, \sigma^2)$), but now eps is just an input, like the image. The path from μ and σ to z is plain differentiable arithmetic:

```
dz/dmu    = 1
dz/dsigma = eps
```

So gradients flow through μ and σ as usual, and the only “random” thing, eps , is a leaf with nothing to learn. This is the single idea that makes VAEs trainable by ordinary backprop.

In the example the trick is written out explicitly between the encoder and decoder rather than hidden in a layer, because seeing the three lines ($\sigma = \exp(0.5 \cdot \log\text{var})$, draw eps , $z = \mu + \sigma \cdot \text{eps}$) is the whole point.

20.3 The KL divergence term

Encoding distributions buys continuity, but on its own it doesn't pull the latent space toward any particular global shape — the encoder could still place its little Gaussians anywhere. So we add a second loss term that measures how far each encoded Gaussian $\mathcal{N}(\mu, \sigma^2)$ is from the standard normal $\mathcal{N}(0, \mathbf{I})$, and penalises the distance. The measure is the Kullback–Leibler divergence, and for two Gaussians it has a closed form:

```
KL = -0.5 * sum_d ( 1 + logvar_d - mu_d^2 - sigma_d^2 )
```

Minimising it pulls every μ toward 0 and every σ^2 toward 1 — that is, it pulls all the per-image Gaussians toward the same unit Gaussian centred at the origin. The encoded clusters get packed together around the origin with no big gaps between them, because every one of them is being tugged toward the same place.

That packing is what makes generation work. After training, the latent space looks (globally) like a unit Gaussian, so if we *draw* a fresh $z \sim \mathcal{N}(0, \mathbf{I})$ it lands where the training data actually mapped, and the decoder turns it into a plausible new image.

The KL gradients we push back into the encoder (per element; the 1/B matches MSE's batch-averaging convention so the two loss terms compose cleanly):


```

dKL/dmu    = (1/B) * mu
dKL/dlogvar = (1/B) * 0.5 * (sigma^2 - 1)

```

20.4 The total loss, and the balance

```
L = reconstruction (MSE) + beta * KL
```

These two terms pull in opposite directions, and that tension is the heart of the VAE. Reconstruction wants the latent to carry as much information about the input as possible (ideally a unique precise code per image — back to the plain AE). KL wants every code to look like generic $N(\theta, \mathbf{I})$ noise (carrying *no* information). The useful solution lives in between: carry enough information to reconstruct, arranged in a smooth gap-free space.

beta sets the balance. With our MSE summed over the 784 pixels, the reconstruction term has the right magnitude for beta = 1 to be the proper variational objective (the ELBO). Turn beta up and the latent space becomes cleaner and more disentangled but reconstructions blur; turn it down and reconstructions sharpen but the space develops the same voids the plain AE had. Two failure modes worth naming:

- **Posterior collapse:** if KL dominates (or the decoder is powerful enough to do without the latent), the encoder gives up and outputs $\mu = \theta$, $\sigma = 1$ for everything — the KL term goes to zero and the latent carries nothing. You'd see KL crash toward 0 while reconstruction stays high. With a tight 2D bottleneck on MNIST this is unlikely; the decoder *needs* the latent.
- **Ignored prior:** if reconstruction dominates, KL stays large, the encoded Gaussians drift away from $N(\theta, \mathbf{I})$, and sampling degrades back toward AE-like voids.

A healthy run shows both terms settling to non-trivial, stable values — recon decreasing then levelling, KL rising from ~0 then plateauing at some modest positive number rather than collapsing or exploding.

20.5 Latent dimension: quality vs. the manifold picture

There's a tension specific to this example. A 2D latent lets us draw the manifold grid (below), which is the clearest demonstration that the space is continuous and gap-free. But two numbers cannot describe a specific handwritten digit — every slant, stroke width, and loop — so 2D reconstructions are necessarily a blurry class-average. Pushing 60,000 varied images through a 2-float bottleneck simply doesn't have the capacity, and you see it directly: a 2D run lands around 0.20 RMS

error per pixel, versus ~ 0.07 for the 16-dimensional plain autoencoder in note 13.

So the example defaults to **LATENT = 16** for genuinely sharp reconstructions and varied samples, and you set it back to 2 when you want the manifold picture. Raising the dimension is the single biggest lever on quality; more epochs help only at the margin once recon stops falling. (A second, smaller lever: MSE itself blurs generative image output, because the MSE-optimal guess under uncertainty is the *average* of all plausible outputs. Per-pixel binary cross-entropy sharpens MNIST, and $\beta < 1$ trades latent regularity for reconstruction sharpness.)

20.6 The demos

Reconstruction uses μ directly (no sampling) — the deterministic “best guess” code for each image. At 16D these are crisp; at 2D they’re the blurry averages described above.

Latent grid (only drawn when **LATENT == 2**) is the demo the plain AE couldn’t produce, and the one that makes the whole thing click. We sweep z over a regular grid covering $[-2.5, 2.5]^2$, decode each grid point, and tile the results into a single image: the entire data manifold laid out as one continuous sheet, each region owning a kind of digit and morphing smoothly into its neighbours with no dead patches. That fully populated sheet is the visual proof the KL term did its job.

(We space the grid linearly for simplicity. The textbook version spaces it by the inverse normal CDF of evenly-spaced probabilities, sampling the prior uniformly *by probability mass* — a slightly nicer manifold, but it needs a probit function we haven’t written.)

Latent interpolation works in any dimension: encode two digits to their means, walk the straight line between the two codes, and decode each step. One digit morphs into another through latent space — the same idea as the AE’s interpolation, now in a space that’s been regularised to be smooth everywhere.

Random samples draw $z \sim N(0, I)$ and decode. These are images the encoder never produced from any real input — pure generation. At 16D most are clean digits; a few are blends from samples landing between clusters.

20.7 What this sets up for audio

Nothing in the VAE machinery is image-specific. Swap the 784-pixel input for a per-frame magnitude spectrum and the same encoder/decoder/reparam/KL apparatus learns a latent space of *spectra* instead of digits. Then:

- interpolating between two instruments’ latent codes is a timbre morph

- ▶ sampling $z \sim N(\theta, I)$ invents a new timbre that never existed
- ▶ decoding back to a spectrum, plus the input phase and an inverse STFT, turns it into audible sound

That's the morphing application. The remaining pieces are the audio plumbing — STFT/iSTFT with overlap-add — not the model, which is done.

So far we've judged networks by their outputs — accuracy, reconstructions, samples. But we can also look directly at what a network *learned* by rendering its weights as images. For the first layer of an MNIST model this is especially vivid, and it needs nothing we haven't already built: the `Bmp.h` writer from note 13 does all the work.

21.1 Weights are images

A Dense layer's weight matrix w has one **row per output neuron** and one **column per input**. For an MNIST model the input is 784 pixels, so each row is 784 numbers — and 784 numbers reshaped to 28x28 is just an image. Render that image and you see, literally, the pattern that neuron responds to. The neuron computes a dot product of its weight row with the input, so the row *is* the template it's matching against.

21.2 The linear classifier: ten templates

The example uses the simplest model that has interpretable weights: a linear classifier, `Dense(784, 10)` followed by `Softmax`, with no hidden layer. Its weight matrix is 10x784 — ten rows, one per digit class — so each row is a per-digit **template**.

A linear classifier's decision is nothing but ten template matches: it dots the input image against each of the ten weight rows (plus a bias) and picks the largest. Softmax then turns those ten scores into probabilities. So the templates aren't a metaphor — they are exactly what the model compares your image to. Visualising them shows the entire "reasoning" of the model in ten pictures.

On real MNIST the templates look like ghostly digit prototypes: the "0" template is a ring of positive weight with a negative hollow in the middle (ink in the centre is evidence *against* a zero); the "1" is a strong vertical bar; and so on. Where a template is positive, ink at that pixel is evidence *for* the class; where negative, ink there is evidence *against*.

(On trivially separable data — like the synthetic shapes used for smoke testing — the templates come out noisy, because a model that hits 100% accuracy in a few epochs never has to commit to a clean template. The interpretable prototypes are a property of *real*, overlapping data forcing the model to find genuinely discriminative pixels.)

21.3 Why a diverging colormap

Weights are signed — positive (evidence for) and negative (evidence against) — and zero is meaningful (this pixel is irrelevant to this class). A plain black-to-white ramp would hide the sign. So we use a **diverging**

colormap: white at zero, reddening toward positive, bluing toward negative, normalised by the largest absolute weight so the scale is shared across all ten templates. This is exactly why `Bmp.h` writes 24-bit RGB rather than greyscale — the colour carries the sign, which is half the information.

21.4 Reaching the weights

Two small things make this possible, both already in the library:

- ▶ `Network::add<L>(...)` returns a *reference* to the layer it just constructed. So we grab the Dense layer at build time — `Dense<float>& fc = net.add<Dense>(784, 10);` — and keep that reference. Because the network owns the layer by `unique_ptr`, the object's address is stable; the reference stays valid as more layers are added.
- ▶ `Dense::W()` exposes the weight matrix (const) for reading. After training, `fc.W()` is the 10x784 matrix we render.

Each row is reshaped to 28x28, mapped through the diverging colormap, scaled up, and tiled into a 5x2 grid written to `mnist_weights.bmp`.

21.5 Deeper networks

For a network *with* hidden layers, the same trick applies to the **first** layer: each of its rows is still a 784-vector, still an image, and they tend to look like localised stroke and edge detectors — little oriented strokes, loops, and blobs that later layers combine. The deeper layers are not directly visualisable this way, because their inputs are activations, not pixels; seeing what *they* respond to needs fancier techniques (e.g. optimising an input to maximally excite a chosen neuron). First-layer weights are the honest, zero-machinery version of interpretability, and a linear model is the purest case of all — there *are* no hidden layers to hide behind, so the weights are the entire model.

This is also a nice closing demonstration of why we built `Bmp.h` when we did: the same writer renders autoencoder reconstructions, VAE manifolds, and now the learned weights themselves.

The audio VAE: training, serialization, and remapping

22

We have a working VAE (note 14) and the audio plumbing — WAV I/O and a transparent STFT/iSTFT (the round-trip reproduces a signal to the quantisation floor). This note connects them: train the VAE on the spectra of orchestral sounds, save it, and use it to “remap” an arbitrary input sound onto the manifold of orchestral timbres.

Three pieces: model serialization (so we train once), the log-magnitude pipeline (turning audio into something the VAE eats and back again), and the remap application itself.

22.1 Serialization — train once, apply many times

Training on the SOL frames is slow (a 2049-wide first layer over hundreds of thousands of frames). We don’t want to retrain every time we remap a different song, so the network can now save and load its weights — exactly the train-once / iterate-fast split we already use for features.

The design is polymorphic, mirroring how `update(Optimizer&)` works. Layer gained two virtuals with no-op defaults:

```
virtual void save_params(std::ostream&) const {}  
virtual void load_params(std::istream&) {}
```

Activations inherit the no-ops (they have no parameters); Dense overrides them to write and read its weight matrix and bias. `Network::save` writes a small header (a WNET tag, a version, and the layer count) and then asks each layer to serialise itself; `load` does the reverse. Crucially, `load` does **not** build the graph — it fills an already-constructed network of the *same architecture*. Loading into the wrong shape throws (the layer count and each matrix’s dimensions are checked). That’s why the application tools rebuild the identical encoder/decoder before calling `load`.

No `dynamic_cast`, no central registry of layer types: a future Conv layer just implements the two methods and serialises for free.

22.2 The log-magnitude pipeline

Audio is a waveform; the VAE works on fixed-length vectors. The STFT bridges them: slice into overlapping frames, window, FFT. Each frame becomes a complex spectrum, from which we take the **magnitude** (how much energy at each frequency) and the **phase** (the alignment of each frequency). We feed the VAE magnitude only — phase is unstructured and notoriously hard for a network to model, so the standard move is to model magnitude and handle phase separately (see §4).

We don’t feed raw magnitude, but **log-magnitude**, $\log(1 + |x|)$. Spectral magnitudes span a huge dynamic range — a few loud harmonics over a

sea of near-silent bins — and raw MSE would obsess over the loud bins. The logarithm compresses that range into roughly $[0, 6]$, so the network spends its capacity sensibly across the spectrum. This is the same `logmag` feature the classifier used, so the existing `extract_features ... logmag_per_frame` produces exactly the training set, and the `remap` computes the identical quantity from its own STFT (same Hann window, same FFT, same $\log(1 + \text{mag})$) — no train/inference mismatch.

The VAE itself is the MNIST one with bigger layers and a 32-D latent (we don't need a 2-D latent here — there's no manifold picture to draw, just quality). One change: the decoder's output layer is **linear**, not sigmoid. Pixels were bounded to $[0, 1]$; log-magnitudes are non-negative and unbounded, so there's nothing to squash. The apps clamp the decoder output to ≥ 0 and invert with `expm1` to recover magnitude.

22.3 Training (`train_audio_vae`)

`train_audio_vae` reads the per-frame `logmag` cache, ignores the labels (this is unsupervised), and runs the exact VAE training loop from note 14 — encoder emits `mu` and `logvar`, reparameterise $z = \mu + \sigma \cdot \epsilon$, decode, $\text{loss} = \text{MSE} + \beta \cdot \text{KL}$, backprop the reconstruction and KL gradients into the encoder. At the end it saves `<prefix>.enc` and `<prefix>.dec`.

This is also the first training run heavy enough to need performance care: the frames are subsampled (adjacent STFT frames are highly redundant) and the model is checkpointed periodically so a long run is never all-or-nothing. Those tricks, and the parallelised `matmul` underneath them, are the subject of note 17.

A balance note: because MSE is summed over 2049 bins, the reconstruction term dwarfs the KL at $\beta = 1$, so the latent isn't pulled tightly onto the unit Gaussian. That's the "ignored prior" regime from note 14 — perfectly fine for the `remap` (which encodes real frames and uses `mu`, never sampling the prior), but for clean *sampling* and *interpolation* we'll want a larger β . It's a constant in the file.

22.4 Remapping (`vae_remap`) — projection onto the manifold

This is the application the whole audio arc was built for. Take any input — a song, a voice — and pass each of its magnitude frames through the VAE. The decoder has only ever learned to produce SOL-like spectra, so an out-of-distribution frame (a drum hit, a vocal formant) is reconstructed as its *nearest orchestral interpretation*. The result is the input's rhythm and structure rendered in orchestral timbre. Conceptually it's autoencoding out-of-distribution input through a decoder constrained to one manifold — projecting the song onto the space of orchestral sounds.

The pipeline:

1. STFT the input $\rightarrow$ complex frames.
2. For every frame, compute $\log(1 + \text{magnitude})$ and stash the phase.

3. Stack all frames into one $2049 \times n_frames$ matrix and run a single batched encode $\rightarrow \mu \rightarrow$ decode. (One big matmul beats thousands of one-column forward passes — and the Network is already built for batches, columns = examples.)
4. For each frame, invert the decoder's log-magnitude (`expm1`, `clamp >= 0`) and recombine it with that frame's **original phase**.
5. iSTFT and peak-normalise.

The phase reuse in step 4 is the key to why this application is robust. The VAE changed the magnitude — the *what frequencies* — but we keep the input's phase — the *when and how aligned*. We're not inventing phase from nothing, so there are no phase-reconstruction artifacts. That's why remapping sounds clean while pure generation (next) is harder.

22.5 What to expect from the remap

This is a dense network on magnitude frames with reused phase: expect “input rendered through the orchestra,” recognisable and demo-quality, not a commercial timbre-transfer plugin. Each frame is treated independently (the model has no sense of time), so sustained tones fare better than sharp transients, and the orchestral “colour” comes from the decoder being unable to represent anything but SOL spectra.

22.6 Generating (`vae_generate`): interpolation and sampling

The remap had the input's phase to lean on. The two generative applications produce a magnitude with no obvious phase, so each has to borrow one. Both avoid iterative phase estimation (Griffin-Lim), trading some fidelity for simplicity.

Interpolation morphs between two real sounds. Encode both to latent means, then for each output frame blend the codes — $z = (1-t)*z_A + t*z_B$, with t sweeping linearly from 0 at the first output frame to 1 at the last — and decode. The phase is reused from sound A, frame by frame. Because interpolation has a genuine source to borrow phase from, it sounds nearly as clean as the remap on sustained material; coherence degrades as t approaches 1 and A's phase no longer matches B's magnitude, worst on transients.

Sampling is the pure generative case: draw count anchor latents $z \sim N(0, 1)$ and walk linearly through them across the donor's duration. Frame 0 lands on anchor 0, the last frame lands on anchor count-1, and interior frames interpolate between adjacent anchors — so you hear a journey through several invented timbres in one go. This also exposes the latent space's continuity, which is one of the VAE's signature properties: nearby latents decode to similar sounds. The trade-off comes from phase: there is no source at all, so we borrow the phase of the donor sound frame by frame. This is the crudest path, and honestly so: the donor's phase only “fits” the frequency bins where the donor had energy, and the sampled magnitude generally peaks elsewhere, so the mismatch produces artifacts.

It is exactly the case a real phase-reconstruction method (Griffin-Lim, or a neural vocoder) would most improve. We include it because hearing a timbre drawn straight from the prior — however rough — is the point of having a VAE rather than a plain autoencoder.

A subtlety worth stating plainly: the phase problem is **not** a limitation of the VAE. Sampling produces a complete, valid magnitude spectrum; the difficulty is purely that our chosen representation (magnitude only) discards phase, so turning that spectrum back into a waveform needs phase from somewhere. On images there is no such tax — a sampled image is the finished output. It is the audio representation, not the generative model, that charges the toll. That realisation is the historical motivation for the entire neural-vocoder line of work; here we simply borrow phase and move on.

22.7 Bigger picture

A weak beta keeps the latent loosely organised, which barely matters for the remap but does for sampling and interpolation, where a tidier latent would morph and sample more smoothly. And the per-frame model's blindness to time is the deepest limitation: timbre that should evolve coherently is reconstructed frame by independent frame. Both point the same way — toward a convolutional VAE that sees the 2D time-frequency structure of a spectrogram, and a sequence model on top of its latents. Those are the next milestones.

Making training practical

Up to the audio VAE we never had to think about speed. IRIS trains in a blink; the MNIST classifier in a minute or two. You could change a hyperparameter, rerun, and see the result before your coffee cooled.

The audio VAE broke that. Its first layer is 2049 wide, and the SOL corpus is hundreds of thousands of frames — so a single epoch does on the order of 10^{12} multiply-adds, and a naive run can take many minutes per epoch. And it only gets heavier from here: convolutional layers, the ConvVAE, and the GAN are all more expensive than what we've built so far. This is the point where a few optimization tricks stop being optional.

The good news is that we don't need anything exotic, and nothing here changes what the library *is*. Three kinds of move: compute the same work faster, do less work, and make long runs safe.

23.1 Where the time goes

Almost all of it is in one place: the matrix multiply in Dense, $Z = W \cdot X$. Everything else — activations, the loss, the Adam update — is linear in the data and cheap by comparison. The first layer dominates because it's the widest: W is 256×2049 , so each batch is a 256×2049 by $2049 \times B$ multiply, far larger than any later layer's. Optimising the matmul optimises training. So that single function is where every trick below is aimed.

23.2 Compute the same work faster

Vectorization (free). Compiling at `-O3` lets the compiler turn the inner loop of the matmul into SIMD instructions — one machine instruction multiplying several numbers at once (NEON on Apple Silicon, AVX on x86). Our matmul is written in `i-k-j` order precisely so the innermost loop walks contiguously through memory, which is both cache-friendly and easy for the compiler to vectorise. This costs one flag and typically buys a few times speedup.

Threads (cheap, big). The matmul is *embarrassingly parallel*: each output row of Z depends only on one row of W and all of X , and different output rows never touch the same memory. So we split the result rows into contiguous blocks and compute the blocks on separate `std::threads` — no locks, no shared writes, no synchronisation beyond a join at the end. With as many threads as cores, this scales nearly linearly: an eight-core machine does the big matmuls roughly eight times faster.

Two details matter. First, threads have a launch cost, so it's only worth parallelising when there's enough arithmetic to amortise it — the matmul runs serially below a work threshold (small matmuls, like the ones over the 32-D latent, just run on one thread). Second, we use `std::thread` rather than OpenMP deliberately: `std::thread` is part of the C++ standard

library, so it needs no compiler extension and no external runtime, and it compiles unchanged under Apple Clang, GNU g++, and anything else. That keeps the library's "standard library only, no dependencies" rule intact — arguably more so than OpenMP, which is a compiler add-on that Apple's toolchain doesn't even ship.

23.3 Do less work

The cheapest computation is the one you skip. Audio is extremely redundant frame to frame: with a 4096-sample window and a 2048 hop, neighbouring frames overlap by half and, on a sustained note, look almost identical. Training on every single frame spends most of its effort re-learning the same spectrum. So `train_audio_vae` can **subsample** — train on every Nth frame. With hundreds of thousands of frames, taking every third or fourth costs almost nothing in quality and cuts epoch time proportionally.

One implementation note worth its own line: we subsample the *index list*, not the data matrix. The cached feature matrix is already several gigabytes; copying a subsampled version would briefly double that. Instead we build a list of the column indices we want and select mini-batches from the original matrix through that list — no large copy, same memory footprint.

(Subsampling and a smaller dataset also argue for slightly more epochs, since each epoch sees fewer distinct examples — but given how alike the dropped frames are, it's nearly free in practice.)

23.4 Make long runs safe

This one isn't about speed; it's about not losing work. A long training run that only saves at the very end is fragile — a crash, a closed laptop, or a `Ctrl-C` at minute 90 throws everything away. So `train_audio_vae` **checkpoints**: every few epochs it writes the current encoder and decoder to the same `<prefix>.enc/.dec` files (serialization, note 16, makes this a one-liner). Kill the run whenever you like and the latest weights are already on disk. It also lets you *listen* to an intermediate model — load the epoch-10 checkpoint in `vae_remap`, decide whether more training is actually improving things, and stop early if not.

23.5 An optional bigger hammer: BLAS

There is one larger lever we make available but leave off by default. A tuned **BLAS** library — OpenBLAS, Intel MKL, or Apple's Accelerate — implements matrix multiply with cache blocking and hand-written assembly we are not going to reproduce, and for large matmuls it beats our loop by a wide margin (and on Apple Silicon, Accelerate uses the chip's dedicated matrix units). So `Matrix::operator*` has an optional BLAS path, enabled at compile time with `make BLAS=1`.

The design keeps it from contaminating the rest of the library:

- ▶ All BLAS use is in *one place* — `operator*` — guarded by `#ifdef WEFT_USE_BLAS`. Nothing else in the codebase knows it exists, and the default build neither includes nor links any BLAS, so the “standard library only” promise holds for anyone who doesn’t ask for it.
- ▶ Because `operator*` is a *member* of `Matrix`, it accesses the row-major `data_` buffer directly and passes it straight to `cblas_sgemm` / `cblas_dgemm` — the BLAS path needs nothing from outside the class.
- ▶ It dispatches with `if constexpr` on the scalar type: `float` and `double` go to BLAS, any other type falls through to the portable threaded loop. So the feature is type-safe and degrades gracefully.
- ▶ The build is OS-aware: on macOS it links Apple’s Accelerate framework, which ships with the system (nothing to install); elsewhere it links OpenBLAS.

This is the intended shape of an optional dependency in a library like this: a single, well-isolated seam, off by default, that a user who wants raw speed can switch on without the rest of the code — or the rest of the users — ever paying for it. The point of writing our own `matmul` was to *understand* it; once understood, delegating it to a tuned kernel for production-scale runs is a reasonable, deliberate choice rather than a hidden default.

23.6 The hammer we still leave in the drawer: GPU

A GPU would be faster still, by one to two orders of magnitude. But it is a different kind of project, not a flag: the data has to live in device memory, every operation needs a kernel, and the whole mental model shifts from “read the loop” to “manage a device.” That’s the *how to make neural nets fast on accelerators* course, which is genuinely separate from the *how do neural nets work* course this library is. We leave it out by choice.

The honest summary: the tricks here are engineering, not magic. They don’t change the number of multiply-adds the math requires — vectorization and threads just do them faster, an optional BLAS does them with a better kernel, subsampling does fewer of them, and checkpointing protects the run. Stacked together they’re enough to keep CPU training of everything in this library, up through the GAN, bearable on a laptop. Knowing which lever to pull, and when reaching for a BLAS or a GPU is worth the cost, is itself part of doing machine learning.

For seventeen notes we have used a single data structure, `Matrix`, for all of the numerical work in this library. The MLPs, the autoencoders, the variational autoencoders — every layer in every network has taken a matrix in and produced a matrix out. That has been sufficient because all of those networks work on *flat* feature vectors: an MNIST image is 784 numbers, an audio frame is 2049 numbers, and a batch of them is a two-dimensional arrangement of (features down, examples across). One row index, one column index. The matrix fit the job.

Convolutional layers do not fit that shape. A convolutional layer's input is a *batch of images*, and an image is fundamentally 2-D in space (height and width). It also has a *channel* dimension — one for grayscale, three for RGB, sixteen or thirty-two for the intermediate feature maps deeper in a network. So the natural shape is **four-dimensional**: batch $\times$ channels $\times$ height $\times$ width. We need a data structure that holds that shape.

This note is about what such a data structure is, what layout we chose for it, and — because it is a question worth being honest about — why we built `Tensor4D` as its own class rather than generalising `Matrix` to N dimensions.

24.1 What a tensor is

In mathematics a *tensor* is a multi-dimensional array of numbers, a straightforward generalisation of objects we already know:

- ▶ A **scalar** is a 0-D tensor: a single number.
- ▶ A **vector** is a 1-D tensor: an array indexed by one integer.
- ▶ A **matrix** is a 2-D tensor: a grid indexed by two integers.
- ▶ An **image batch** is a 4-D tensor: indexed by (batch, channel, height, width).

(The word “tensor” carries some deeper mathematical baggage from physics and differential geometry that machine learning borrowed lightly. In ML, *tensor* mostly just means “ N -dimensional array of numbers.”)

So `Matrix` is mathematically a 2-tensor with shape (rows, cols), and our new `Tensor4D` is a 4-tensor with shape (N, C, H, W). The natural question follows immediately: if `Matrix` is a special case of tensor, why do we have two classes? Couldn't a single unified `Tensor` cover both? That is the design question this note is really about, and the answer is a real tradeoff rather than an obvious one. We come back to it after settling the layout choice.

24.2 NCHW: the order of the axes

A 4-D tensor with shape (N, C, H, W) could be stored in memory in any of 24 different orderings. Two are common in practice:

- **NCHW** — batch, channels, height, width (outermost first).
- **NHWC** — batch, height, width, channels.

The choice is about which dimension is *innermost in memory* — i.e., which adjacent indices correspond to adjacent bytes. With NCHW (what we use), walking the flat storage one element at a time walks W fastest (within one row), then H (within one channel of one image), then C (within one image), then N (between images). Concretely, in our `Tensor4D` the element at index (n, c, h, w) lives at flat offset

```
((n * C + c) * H + h) * W + w
```

So pixels in the same row of the same channel of the same image sit next to each other in memory.

That is exactly what a 2-D spatial convolution wants: it slides a kernel across the H - W plane of one channel at a time. Having that plane contiguous in memory means the inner loop walks contiguous bytes, which is friendly to the CPU cache and to BLAS. NHWC interleaves channels — pixels of different channels at the same (h, w) position are adjacent — which is better for some quantisation tricks and for memory-bound *channel*-wise operations, but less natural for spatial convolution on CPU.

PyTorch and cuDNN default to NCHW; we follow the same convention so the code reads as expected to anyone arriving from that ecosystem.

24.3 One class or two?

Now the design question. There are two reasonable approaches:

Path A — one unified Tensor. A single class that holds an N -dimensional shape and a flat data buffer, with operations that work for any rank. This is what every production library does: PyTorch’s `torch.Tensor`, NumPy’s `ndarray`, TensorFlow’s `tf.Tensor`, JAX’s `arrays`. There is exactly one object, and a 2-D matrix is a tensor with rank 2.

Path B — separate Matrix and Tensor4D. What we did. `Matrix` is its own class with its own operations (`operator*`, `transpose`, `sumColumns`). `Tensor4D` is its own class. They are bridged by explicit free functions like `flatten(Tensor4D) -> Matrix` when data needs to move between them.

The pros and cons cut both ways, and which one is “right” depends on what the library is for.

24.4 The case for Path A (unified Tensor)

- **Mathematical cleanliness.** A matrix really is a tensor of rank 2; having one class matches the math.
- **One API to learn.** Users construct one kind of object regardless of dimensionality.
- **flatten becomes reshape.** No type conversion is needed; you just reinterpret the same storage with a different shape. The data buffer never moves.

- **Future-proof.** Whatever rank comes up next — 3-D for attention sequences, 5-D for video — the class already handles it. No new type to invent each time.
- **Matches what students will use professionally.** Anyone arriving here from PyTorch already thinks in those terms; anyone going to PyTorch later won't have to unlearn anything.

24.5 The case for Path B (what we have)

- **Type safety.** Our `Dense::forward(Matrix)` literally cannot accept a 4-D tensor. The compiler refuses. Passing the wrong-rank object to a layer is a *compile* error, not a runtime exception somewhere deep inside the function. For a learning context — where building mental models of the data flow is the whole point — this matters; mistakes are caught at the boundary, not buried.
- **Operation specificity.** Many of `Matrix`'s operations only make sense in 2-D. What is “transpose” on a 4-D tensor? You have to say *which two axes* you're swapping. What is “matrix multiply” on two 4-D tensors? A contraction over which pair of axes? In a unified `Tensor` each of these needs either restriction to rank 2 (special cases sprinkled throughout) or generalisation to indexed-axis operations (a different and more complex API). The two-class design lets each class have exactly the operations that fit its dimensionality, with no special-casing.
- **Pedagogical clarity.** When you read

```
Z = W * X + b
dW = dZ * X.transpose() / N
```

in our `Dense` layer, the code reads literally like the matrix-form backprop equations from note 02. No `.view()`, no `.contract(...)`, no axis names. The math and the code are the same string of symbols. In a unified `Tensor` every operation must specify which axes are involved, which moves the code one step away from its mathematical form.

- **Small, focused files.** `Matrix.h` is about 250 lines and can be read in one sitting; `Tensor4D.h` is 160. A unified `Tensor` with proper N-D indexing, broadcasting, reshape, and generalised contractions is easily a thousand lines, and the reader can no longer hold all of it in their head while reading conv code.
- **The friction itself is the lesson.** The explicit `flatten` step between the conv stack and the dense classifier *shows* where the architecture switches from “operating on spatial structure” to “operating on flattened feature vectors.” In PyTorch that line is `x = x.view(N, -1)` and the change is invisible. In our code the type changes — `Tensor4D` becomes `Matrix` — and the reader cannot miss that something architectural has happened.

24.6 Why production libraries chose Path A

The libraries that ML engineers use every day chose Path A, and they are not wrong to. The reasons map almost exactly to where their goals differ from ours:

- ▶ They are libraries for working practitioners, not learners. The type system isn't being asked to teach; it is being asked to stay out of the way.
- ▶ They serve every possible rank — 1-D vectors, 3-D sequences, 5-D video, dynamically-shaped data — and cannot afford one class per rank.
- ▶ They optimise broadcasting and reshape heavily, so a `.view()` call is essentially free and idiomatic.
- ▶ They have automatic differentiation, which records computational graphs over generic tensors and would be enormously complicated by per-rank types.

For those goals, unified is the correct choice. For ours — teaching — separate is correct. Same problem, different priorities, different answers.

24.7 The principle, and a look ahead

The principle to carry forward is this: **each data structure earns its own type when it has its own conceptual identity**. Matrix has an identity — it is the object on which the backprop math is written. Tensor4D has an identity — it is what a batch of feature maps looks like, and the operations on it (convolution, pooling) are different in character from matrix operations. If a third structure came along that was *only* “matrix with a bigger shape” we would not necessarily need a new class for it.

When attention arrives later in the book, the data shape will be 3-D — batch $\times$ time $\times$ features. The question will repeat: a new Tensor3D class, or a reshape of Matrix? The honest answer will depend on whether the operations are meaningfully new. If they are — and attention has its own characteristic operations (scaled dot product, masking, positional encoding) — a new type may earn its keep. If they are not, reuse.

If this library ever grew into a more general framework — N-D tensors, broadcasting, autograd, GPU support — refactoring toward a unified Tensor would be a natural step. The current design is not a mistake to be undone; it is a deliberate choice for an educational stage, and a more general library would eventually outgrow it. That is a fair thing to say in a book: the design *is* a design, with reasons.

What comes next is the layer that finally uses the Tensor4D — Conv2D, which turns a Tensor4D(N , in_C , H , W) into a Tensor4D(N , out_C , H' , W') via the `im2col-and-matmul` approach. That is the next note, and where the shape we just chose pays off in code.

For the first eighteen notes we worked exclusively with dense (fully connected) layers. Dense layers handle the MNIST classifier, the Iris classifier, the audio VAE on spectrograms — anything where the input is naturally a flat vector of features and you don't care which feature sits next to which. They are limited in a specific, important way: they have no notion of *spatial structure*. A dense layer treats every input neuron as exchangeable with every other one. Permute the columns of a batch of MNIST images and a dense MLP, retrained on the permuted data, will reach exactly the same accuracy. That is the giveaway that dense layers are not using anything about the *2-D arrangement* of pixels. For images, that's leaving most of the structure on the table.

This chapter is about the layer that does use it: **convolution**. The goal is to motivate it honestly, derive its forward and backward math, explain the engineering trick (`im2col + matmul`) that makes it run fast on a CPU, and then train a small ConvNet on CIFAR-10 to see the payoff — not just the accuracy number, but the *learned first-layer kernels*, which are interpretable as images in their own right and tell you what the network has actually decided “an edge” looks like.

25.1 Why dense MLPs are wrong for images

Three reasons, each separately damning:

Parameter explosion. A 32×32 RGB CIFAR-10 image has $32 \cdot 32 \cdot 3 = 3072$ input pixels. A single dense layer with 1024 hidden units would have $3072 \times 1024 \approx 3.1$ million weights in the *first layer alone*. Most of those weights are answering questions like “what is the joint relevance of pixel (3, 7, red) and pixel (28, 14, blue)?” — questions that for natural images simply don't have meaningful answers. The weights are mostly wasted.

No translation invariance. Suppose we have a perfectly trained dense MLP that classifies cats correctly. We take a cat image, shift every pixel one column to the right (and wrap the rightmost column to the left, say). To a human, it's the same cat. To the dense MLP, every single input has changed: the pixel that *was* at index 5 of the input vector is now at index 6, and the weight matrix has no idea those two positions are spatially adjacent. The MLP has to learn the concept of “cat” *separately* at every possible translation. That's vastly inefficient and, in practice, doesn't generalise.

No exploitation of locality. Natural images have a strong property: pixels close together in space are far more likely to be related than pixels far apart. Edges, textures, object parts — all local. A dense layer cannot represent this prior; it has to discover locality from data, using millions of weights that mostly should be zero. The right thing is to *build the locality into the architecture*.

25.2 The convolution operator

A 2-D convolution at one output position computes

$$y[h, w] = \sum_{k[h, w]} x[h + kh, w + kw] \cdot k[kh, kw]$$

where k is a small kernel (3×3 or 5×5 are typical) and the same kernel k is applied at every output position (h, w) . Three properties fall out for free:

- ▶ **Local connectivity.** Each output value $y[h, w]$ depends on a small neighbourhood of the input — only the K^2 pixels in the kernel's footprint — not on the whole image. A 3×3 kernel uses 9 input pixels per output, regardless of how big the image is. The parameter explosion goes away.
- ▶ **Weight sharing.** The same kernel k slides across the whole image. There are K^2 weights total *for the entire layer*, not per position. A 3×3 kernel has 9 numbers in it, period. (More precisely: $K^2 \times C_{in} \times C_{out}$, but the *spatial* part is K^2 regardless of image size.)
- ▶ **Translation equivariance.** Shift the input one pixel right; the output shifts one pixel right. The kernel didn't change, the computation didn't change. The network has the right structural prior for natural images baked in.

The third property is the deepest. Equivariance is not invariance — the network's *output* shifts when the input shifts, it doesn't stay the same. But that turns out to be what you want for the early layers, because the *final* layers (after pooling and flattening) can become invariant on top of equivariant features. Build up locality first, then collapse it.

Multi-channel extension. Real images have channels (R, G, B), and intermediate feature maps have many channels (32, 64, sometimes hundreds). So the kernel is actually a 3-D object — (C_{in}, K, K) — that's applied to a C_{in} -channel patch and produces *one* scalar output. To produce C_{out} output channels, we use C_{out} independent 3-D kernels. So a conv layer with $C_{in}=3, C_{out}=32, K=5$ has

$$32 \times 3 \times 5 \times 5 = 2400 \text{ weights}$$

plus 32 bias values (one per output channel). Tiny compared to a dense layer's millions, and that's the point.

25.3 Padding and stride

Two knobs let convolution work with images of any size.

Padding adds zero-pixels around the input border so the output can have the same spatial dimensions as the input. A 3×3 kernel with no padding ($p = 0$) takes a 28×28 input to a 26×26 output (it can't be centred on the boundary pixels). With $p = 1$, you get a 28×28 input to 28×28 output — “same” padding. Whether you want that depends on the architecture; it's

common to use it in early layers so the spatial dimensions only shrink at the pooling steps, not inside the conv itself.

Stride controls how far the kernel moves between output positions. Stride 1 (the default) puts an output at every input pixel. Stride 2 puts one at every other pixel, which halves the output spatial size.

The combined output formula is

$$H_{\text{out}} = (H_{\text{in}} + 2p - K) / s + 1$$

and the same for W . A 32×32 input with $K=5$, $p=2$, $s=1$ gives $H_{\text{out}} = (32 + 4 - 5) / 1 + 1 = 32$ — preserved. $K=3$, $p=1$, $s=1$ also preserves. $K=3$, $p=1$, $s=2$ halves: $H_{\text{out}} = (32 + 2 - 3) / 2 + 1 = 16$.

25.4 im2col: making convolution fast on a CPU

The math above describes the *operation*. Writing it as a literal implementation — six nested loops over $(n, \text{out_C}, h_{\text{out}}, w_{\text{out}}, \text{in_C}, kh, kw)$ — works, but is slow. The CPU’s cache doesn’t enjoy seven-deep loop nests, and we’ve already invested significant work into making `Matrix::operator*` fast (parallel rows, optional BLAS dispatch). It would be nice to *reuse* the `matmul` for convolution.

We can. The trick is called **im2col**, short for image-to-column. The idea: rearrange the input so that the convolution becomes a single matrix multiplication.

Specifically: for every output position $(n, h_{\text{out}}, w_{\text{out}})$, the input patch under the kernel is a 3-D chunk of shape (C_{in}, K, K) . Flatten that chunk into a column of length $C_{\text{in}} * K * K$. Stack the columns side by side across all output positions:

```
im2col matrix: (C_in \ensuremath{\cdot} K \ensuremath{\cdot} K)
               \ensuremath{\times} (N \ensuremath{\cdot} H_out \ensuremath{\cdot} W_out)
```

Now reshape the kernel from $(C_{\text{out}}, C_{\text{in}}, K, K)$ into a matrix of shape $(C_{\text{out}}, C_{\text{in}} * K * K)$ — one row per output channel, the row being that channel’s flattened kernel. Then

```
Z = W \ensuremath{\cdot} im2col           shape: (C_out, N \ensuremath{\cdot} H_out \ensuremath{\cdot} W_out)
```

and the entire convolution is *one* matrix multiply. We get all our prior `matmul` optimisations (`std::thread` parallelisation, optional BLAS) for free. This is exactly what cuDNN does on GPUs.

The cost is the memory of the temporary `im2col` matrix, which is K^2 times larger than the input. For our scales — CIFAR-10 with small batches — this is comfortably within memory. For very large images or deep networks, more sophisticated schemes (Winograd, FFT-convolution) become competitive; we don’t need them.

Backward via col2im. The backward pass reverses the trick. Given the gradient dY of shape $(N, C_{out}, H_{out}, W_{out})$:

1. Reshape dY into dZ of shape $(C_{out}, N * H_{out} * W_{out})$.
2. Parameter gradients are straightforward matmuls: $dW = dZ * \text{im2col}^T(\text{kernel gradient})$, $db = \text{sum-along-columns of } dZ$ (bias gradient).
3. For the gradient w.r.t. the input, compute $dCol = W^T * dZ$. This has the shape of the original `im2col` matrix.
4. **col2im** folds $dCol$ back into the input shape, *accumulating* the contributions at every input pixel that was touched by more than one output position. When stride is less than kernel size, that's most pixels.

The accumulation is the subtle part. If you write `=` instead of `+=` in the `col2im` loop, the gradient computation is silently wrong for every conv layer with `stride < K`. The numerical gradient check in `tests/test_conv2d.cpp` is exactly what catches this kind of bug.

25.5 Pooling

After every conv layer (or every couple) we want to *reduce* the spatial size — both because we want to build invariance to small shifts on top of the equivariant features, and because it cuts the computational cost of subsequent layers. The standard tool is **max-pooling**: take a window (typically 2×2) and emit the maximum value.

Forward is obvious. Backward is the part worth understanding: the output depended on *one* input pixel per window (the one with the max value). Every other pixel in the window contributed nothing, so the gradient dY for that output cell flows entirely to the winning input pixel and nowhere else. We store the winning index during forward and look it up during backward.

With non-overlapping windows (`stride = pool size`, the typical “halve everything” case), each input pixel wins at most once. With overlapping pooling (`stride < pool size`), an input pixel can win in multiple output windows, and the gradients accumulate. Same `+=` issue as conv backward; the same test pattern catches it.

Max-pool has zero learnable parameters and runs in $O(N \cdot C \cdot H \cdot W)$. Cheap, robust, and the workhorse of every conv classifier.

25.6 The conv-to-dense bridge

Putting convolution and pooling together: a small conv stack reduces a $32 \times 32 \times 3$ image to a (C, H', W') feature map — say $(64, 8, 8)$. To predict a class probability vector, we need a *vector* output, not a 3-D feature map. The standard move is **flatten**: turn the 3-D feature map into a 1-D vector and feed it to a dense classifier.

In our library, this is the moment where the data structure changes — `Tensor4D` becomes `Matrix`. We do this with an explicit free function `flatten` (`Tensor4D`) -> `Matrix`, *not* a hidden reshape inside a unified tensor type. The conv stack is one `ConvNetwork`; the dense head is another `Network`; the example wires them together. Why this is the right choice for a teaching library is discussed at length in [note 18](#).

The classic AlexNet shape, in our library, looks exactly like this:

```
ConvNetwork<float> conv;
conv.add<Conv2D>( 3, 32, 5, 1, 2); conv.add<ReLU4D>(); conv.
  add<MaxPool2D>(2);
conv.add<Conv2D>( 32, 64, 3, 1, 1); conv.add<ReLU4D>(); conv.
  add<MaxPool2D>(2);

Network<float> dense;
dense.add<Dense>(64 * 8 * 8, 10);
dense.add<Softmax>();
```

The forward pass at every training step:

```
Tensor4D<float> feat = conv.forward(X);           // (N, 64,
  8, 8)
Matrix<float> flat = flatten(feat);               // (4096, N)
Matrix<float> probs = dense.forward(flat);         // (10, N)
```

And the backward mirrors it exactly, with `unflatten` doing the reverse reshape on the gradient.

25.7 What the library provides

The conv work in this chapter added five new headers:

- **Tensor4D.h** — the 4-D dense tensor in NCHW layout, with element access, `apply`, `fill`, randomisation, and a `hadamard` free function for element-wise products in backward computations. Covered in [note 18](#).
- **ConvLayer.h** — the abstract base class for conv-stack layers, parallel to `Layer` for the dense world.
- **ConvNetwork.h** — the container, parallel to `Network`. Supports `add<L>(args...)`, `forward/backward/update`, `summary()`, `save/load` (with a distinct `WCNV` magic so a conv file cannot be loaded into a dense network by mistake).
- **Conv2D.h** — the convolution itself, with He initialisation, the `im2col` forward and `col2im` backward, configurable kernel size, stride, and padding. Verified by numerical gradient check on `dX`, `dW`, `db` in two stride configurations.
- **ReLU4D.h**, **MaxPool2D.h**, **Flatten.h** — the supporting cast. `MaxPool2D` carries a numerical gradient check too, because its backward (routing through the cached `argmax`) is exactly the kind of indexing logic that's easy to get wrong.

The whole conv library lives behind tests that pass at full numerical precision (double-precision gradient checks, $1e-6$ relative tolerance). If

those pass, the math is right; if they fail, no architectural experiment will rescue it.

25.8 The CIFAR-10 classifier

CIFAR-10 is the standard “step up from MNIST” image dataset: 60,000 32×32 colour images split into 50,000 training and 10,000 test, with 10 classes (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck). It’s exactly the right size to be challenging without needing a GPU: a small ConvNet trained on CPU can reach ~75-80% test accuracy in something like an hour.

We deliberately *skipped* MNIST for the conv classifier. The dense MLP chapter already classified MNIST at ~98% test accuracy; a conv version would just be making the same point in a more complicated way. Going straight to CIFAR-10 shows the genuine win of convolution — on data where a dense MLP struggles to get above ~50% accuracy, the small ConvNet reaches well above 70%.

The architecture we use is `cifar_conv.cpp`:

```
input (N, 3, 32, 32)
  Conv2D(3 -> 32, k=5, p=2)    -> (N, 32, 32, 32)    <-- 5x5
    first layer
  ReLU4D
  MaxPool2D(2)                  -> (N, 32, 16, 16)
  Conv2D(32 -> 64, k=3, p=1)    -> (N, 64, 16, 16)
  ReLU4D
  MaxPool2D(2)                  -> (N, 64, 8, 8)
---- flatten ----              -> (4096, N)
  Dense(4096, 10)
  Softmax
```

Two design choices worth pointing at:

- **5×5 kernels in the first layer**, 3×3 in the second. The first layer is the one we visualise, and 5×5 RGB tiles are much more legible than 3×3 ones. The deeper layer can use the smaller kernel because it operates on richer features (32 channels rather than 3). This is the same logic that led Krizhevsky’s AlexNet to use 11×11 at the first layer (working on 224×224 images), 5×5 next, then 3×3.
- **Two conv blocks**, not three. A third block would push accuracy slightly higher (78-82% range) but adds another hour of CPU training for a marginal improvement. The architecture above is the smallest one that demonstrates the full conv arc — locality, hierarchy, pooling, dense head — without the iteration time becoming frustrating. It’s a pedagogical choice; a production CIFAR-10 model would use ResNet or something larger.

Training. Adam optimizer ($\text{lr} = 1\text{e-}3$, defaults), batch size 64, 15 epochs, He-initialised conv kernels, zero-initialised biases, no data augmentation. Run it with

```
./bin/examples/cifar_conv data/cifar-10-batches-bin 15
cifar_model
```


and on an Apple M3 with `make BLAS=1` it should take roughly 60-90 minutes and produce a model that reaches around 72-78% test accuracy. The training prints loss, train accuracy, and test accuracy each epoch, and saves a checkpoint after each one — Ctrl-C is safe.

25.9 The payoff: visualising what the first layer learned

The whole point of going to CIFAR rather than MNIST was the visualisation. The companion example `cifar_filters.cpp` loads the trained model, extracts the first conv layer's $32 \times 5 \times 5 \times 3$ filters, and arranges them in a 4×8 grid BMP. Each filter is normalised independently to fill $[0, 255]$ so weak and strong filters are both visible.

What you should see, after a successful training run:

- ▶ **Oriented edge detectors** at various angles (horizontal, diagonal, vertical). These look like dark/light gradients across the kernel — high values on one side, low on the other. The kernel is effectively a small derivative filter in some direction.
- ▶ **Color-opponent blobs**. Red-on-green, blue-on-yellow. These are the network discovering colour structure on its own — nothing in the training told it that RGB pixels have any particular meaning, but with enough natural images the colour-opponency falls out of what's discriminative.
- ▶ **Low-frequency centre-surround patterns**. Some filters look like blurry blobs, picking up overall brightness or colour in a region.
- ▶ **Maybe a few dead filters** — uniform grey tiles that never activated and never learned. This is normal; on a small budget of filters, a few often end up at random init.

This is exactly the picture Krizhevsky's AlexNet paper (2012) and Zeiler & Fergus's *Visualizing and Understanding Convolutional Networks* (2014) made famous. Our version, trained on CPU on a much smaller architecture and dataset, shows the same qualitative pattern. That's not a coincidence — the structure falls out of the operator and the data, not the depth or scale.

The deeper layer is harder to visualise as raw weights, because its filters operate on the 32-channel output of the first layer rather than on RGB. Each second-layer kernel is $(32, 3, 3)$ — not directly an image. Techniques for visualising deeper layers (activation maximisation, deconvolutional inversion) exist but are out of scope here. For now, the point is made: the first layer learns *features*, and those features look like edges and colour patches because that's what is locally informative in natural images.

25.10 What's next

The conv arc continues into audio. The same Conv2D layer, applied to log-magnitude spectrograms instead of CIFAR images, gives us a **ConvVAE for sound** — the same generative architecture as the dense

audio VAE from [note 16](#), but with the magnitude-blur problem of the dense version largely fixed by local spatial structure in the encoder and decoder. That's [note 20](#) (still to come): the architecture, the training, the morphs and interpolations, and an honest comparison against the dense version. We should hear the difference.

After ConvVAE the architecture path goes one more direction — **attention** — and then the capstone, the matching-pursuit + attention generative audio system. The work to get to that point is now mostly mechanical, because the foundation is in place.

The audio VAE of [note 16](#) works on individual STFT frames. Each column of the magnitude spectrogram becomes an isolated 2049-dimensional vector that the dense encoder squeezes to 32 latent dimensions and the dense decoder reconstructs. That gives us a learned dictionary of *instantaneous spectra*, but throws away time. The encoder never sees that the slice taken 50 ms into a violin attack looks very different from the slice taken 500 ms into a violin sustain, even though both are “violin.” Anything we want to do about how a sound *changes* over time has to come from outside the model.

The natural fix is to encode short time-frequency *patches* instead of single frames. A patch of 16 STFT frames $\times$ 64 frequency bins is a small 2-D image whose vertical axis is *which frequencies are present* and horizontal axis is *how those frequencies move in time*. Once it’s a 2-D image we have everything we need: Conv2D, MaxPool2D, ReLU4D, Upsample2D, all from the convnet work of [note 19](#). Stack them into a conv encoder and a mirrored conv decoder with a small dense bottleneck in between, and we have a VAE that models *local time-frequency texture*, not just static timbre.

This note builds that ConvVAE and three applications of it: interpolating between two timbres in latent space, sampling from the prior to invent new timbres, and remapping arbitrary input audio onto the manifold the model has learned. We’ll also be honest about where the result is good and where it isn’t.

26.1 Patch shape

A first design choice is the patch geometry. With our existing STFT settings (FRAME_SIZE = 4096, HOP_SIZE = 2048, 44.1 kHz sample rate) each frame has 2049 magnitude bins spanning 0 to 22 kHz at ~10.77 Hz per bin, and consecutive frames are 46 ms apart. We need:

- **Time dimension.** Sixteen frames is ~0.8 seconds. Long enough to capture attack-to-sustain transitions, short enough that almost any TinySOL note produces at least one full patch. (Thirty-two frames = ~1.5 s would carry more context but would exclude all the shorter notes from training.)
- **Frequency dimension.** Sixty-four bins is small enough for a tractable conv stack but only *if we pick the right 64 bins*. That last clause is where most of the trouble lives; §3 below.

So the patch is (1 channel, 64 freq, 16 time). The single channel is the “this is one magnitude image, not RGB” placeholder.

26.2 Architecture

The encoder is a conv stack with three downsampling stages, followed by a dense bottleneck that emits μ and $\log \sigma^2$. The decoder mirrors it.

```
Encoder:
  Conv2D(1->16, 3x3) -> ReLU -> MaxPool(2)    (1,64,16) ->
    (16,32,8)
  Conv2D(16->32, 3x3) -> ReLU -> MaxPool(2)    ->
    (32,16,4)
  Conv2D(32->64, 3x3) -> ReLU -> MaxPool(2)    -> (64,
    8,2)
  Flatten -> 1024
  Dense(1024, 256) -> ReLU -> Dense(256, 2*LATENT)

z = mu + sigma * eps, eps ~ N(0, I)

Decoder:
  Dense(LATENT, 256) -> ReLU -> Dense(256, 1024) -> ReLU
  Unflatten -> (64,
    8, 2)
  Upsample(2) -> Conv2D(64->32, 3x3) -> ReLU ->
    (32,16, 4)
  Upsample(2) -> Conv2D(32->16, 3x3) -> ReLU ->
    (16,32, 8)
  Upsample(2) -> Conv2D(16->1, 3x3) ->
    (1,64,16)
```

A few small decisions worth flagging:

Linear output, no sigmoid. Log-magnitudes are non-negative but unbounded, not $[0, 1]$ like pixel intensities. A sigmoid would saturate them. The generation code clamps to 0! before exponentiating back to linearmagnitude.

Reparameterisation inline. Same trick as [note 14](#): the encoder's last dense layer emits $2 \times \text{LATENT}$ numbers; the first half is μ and the second half is $\log \sigma^2$. We sample $z = \mu + \sigma * \text{eps}$ with $\text{eps} \sim N(0, I)$ and pass z to the decoder. Gradients flow through μ and σ cleanly because the stochastic part (eps) has no parameters.

Four objects in code. The training loop pipes data through *four* sub-networks — `enc_conv`, `enc_dense`, `dec_dense`, `dec_conv` — because we have to convert between Tensor4D and Matrix at the bottleneck:

```
Tensor4D feat = enc_conv.forward(Xb);
Matrix flat = flatten(feat);
Matrix h = enc_dense.forward(flat); // [mu;
    logvar]
// ... sample z ...
Matrix df = dec_dense.forward(z);
Tensor4D dfeat = unflatten(df, B, 64, 8, 2);
Tensor4D recon = dec_conv.forward(dfeat);
```

Backward is the same chain in reverse. Each of the four sub-networks saves to its own file, with suffixes `.enc_conv`, `.enc_dense`, `.dec_dense`,

.dec_conv. Explicit but unambiguous, and the load-side rebuilds the architecture and reads the four files back.

26.3 The frequency-axis problem

Sixty-four bins cannot represent 2049 linear-frequency bins without losing something. The question is *what*.

The naïve choice — block-average groups of 32 consecutive bins — destroys harmonic structure. A 440 Hz tone produces a sharp peak at bin 41 in the original spectrum, with harmonics at bins 82, 122, 163, and so on. Averaging bins 32–63 mixes that one peak with 31 neighbouring valley bins, producing a low plateau value that no longer *looks* like a peak. The VAE learns to reproduce those blurry plateaus. When we upsample back to 2049 bins by repeating each output value 32 times, we get a step-function spectrum with no sharp peaks anywhere. Resynthesised, that sounds like *broadband noise modulated by the right energy envelope* — because the envelope (overall energy distribution) survives the averaging, but the harmonic peaks that define pitched sound do not.

The reconstruction quality with block-averaging is bad enough that a plain phase vocoder doing cross-synthesis on the raw 2049-bin spectrogram beats the ConvVAE handily. That’s a problem: the whole point of the conv VAE is that it should *add* something a vocoder can’t do, but if the underlying audio quality is worse, none of the latent-space arguments matter.

Mel-frequency scaling solves the harmonic-smearing problem. The triangular mel filterbank concentrates resolution where the ear cares — logarithmic spacing, narrow filters at low frequencies, wide at high. With 64 mel bins covering 0–22 kHz at 44.1 kHz sample rate, roughly 40 of those bins sit below 4 kHz where most of the harmonic action lives. A 440 Hz tone now contributes to several adjacent mel bins whose triangles overlap the peak; the harmonic structure survives the forward transform.

Going back from mel to linear uses the *transpose* of the filterbank matrix as a pseudo-inverse. Each linear bin receives the sum, over mel bins, of the mel bin’s value weighted by the triangle that originally fed it. This is lossy — $M^T M$ is not the identity, the filters overlap — but it’s the standard “good enough” inversion. The spectrogram comes back blurry but with the peaks roughly where they should be.

One stability gotcha learned the hard way: mel binning concentrates energy, so raw log-mel-magnitude values are about 9× larger than the corresponding log-mag values for the same audio. Feeding those larger values into the encoder’s randomly-initialised final dense layer produces correspondingly larger $\log \sigma^2$ outputs at initialisation, and the $\exp(\log \sigma^2)$ term in the KL loss explodes — we see KL values in the hundreds of thousands instead of the expected tens, and training oscillates wildly. The fix is a constant scale factor: divide log-mel values by 8 on the way into the network, multiply back by 8 on the way out. The constant lives in `MelTransform`’s state so train and inference always agree.

26.4 Training

The loss is identical to the dense audio VAE: MSE between the reconstructed patch and the input patch, plus β times the KL between the encoder's $N(\mu, \sigma^2)$ posterior and the standard normal prior, with $\beta = 1$. Adam at learning rate $1e-3$, batch size 32, twenty to thirty epochs.

What's different from the dense case is the data pipeline. We read the per-frame logmag cache (the same one [note 16](#) consumes), then for each contiguous run of frames belonging to one source WAV, slide a 16-frame window with a hop of 8 frames and extract patches. Each patch's 16 columns get mel-transformed from 2049 to 64 bins inside the loop, and the resulting $(1, 64, 16)$ tile becomes one training example. On TinySOL we typically get 50–80k patches, which fits comfortably in memory.

Healthy training shows reconstruction loss monotonically decreasing, KL holding stable in single digits, no oscillation. A representative TinySOL run hits recon ≈ 13 at epoch 1 and recon ≈ 6 by epoch 10, with KL hovering around 2–3 throughout. If KL spikes by factors of 100 or recon bounces, the mel scale factor is wrong and the encoder is producing pathological $\log \sigma^2$ at init.

26.5 Three applications

convae_generate interp — encode the middle 16-frame patch of sound A to z_A , the middle patch of sound B to z_B . Sweep $z_k = (1-t) z_A + t z_B$ for t from 0 to 1 across N output patches. Concatenate the $N \times 16$ decoded mel-frames into a single mel-spectrogram, mel-invert to linear magnitudes, pair with phases borrowed (and tiled) from A, inverse-STFT. The result is a single ~ 5 –6 second audio clip whose magnitude evolves smoothly from A's timbre to B's, with A's rhythmic phase character throughout.

convae_generate sample — draw count anchor latents from the prior $N(0, I)$ and walk linearly through them across N output patches. A donor WAV supplies the phase. With count = 1 you sustain a single invented timbre; count = 4 walks a trajectory through four points of the learned timbre manifold.

convae_remap — slide 16-frame patches across an arbitrary input audio file with 50% overlap. Encode each patch to its latent *mean* (no sampling — we want determinism for this application). Decode each back. Overlap-add the reconstructions in the mel-spectrogram domain, invert mel to linear magnitude, pair with the input's *own* STFT phase frame-by-frame, inverse-STFT. The output audio has the same duration as the input; the rhythm and articulation are the input's, the timbre is whatever the VAE projects.

26.6 What this is and is not

A successful ConvVAE on TinySOL plus a remap pass over a song will *not* sound like a clean phase vocoder doing cross-synthesis. The vocoder

works on the full 2049-bin spectrogram and leaves phase untouched; the VAE has compressed each 1024-pixel patch through a 32-dimensional bottleneck and back. That compression is the whole point — it's what gives us a latent space to manipulate — but it costs audible fidelity that a vocoder doesn't have to pay.

What the VAE does that a vocoder cannot is *smooth manipulation in a learned space*. The vocoder has no latent code; pitch-shifting two sounds toward each other or interpolating their formants are different operations, each requiring its own DSP pipeline. The ConvVAE makes a manifold of timbres concrete and walkable: the same architecture gives us interp, sample, and remap with no per-operation engineering.

Honest limitations from this build, in order of how much they hurt:

- ▶ **Mel inversion is lossy.** The transpose-as-pseudo-inverse blurs the spectrogram on the way back. Better mel inverters exist — learned vocoders like HiFi-GAN, iterative methods, Griffin-Lim starting from a good initialisation — but none are in the library.
- ▶ **Phase mismatch.** In remap, the output pairs the input's phase with the VAE's reconstructed magnitude. The phase encodes interference patterns specific to the *original* magnitude, so the pairing causes partial cancellations and an audible coloration. Griffin-Lim iteration would reconstruct phase from magnitude alone and close this gap (~100 lines we don't have yet).
- ▶ **The latent bottleneck is severe.** 1024 pixels → 32 latents → 1024 pixels is a 32× compression. Sustained tones round-trip well; rapid spectral changes (a piano attack, a percussive transient) blur into the surrounding sustain.

The thing to take from this note isn't a finished audio system — for that you'd reach for a vocoder. What we have is the *architectural pattern*: encoder as ConvNetwork → Flatten → Network, decoder as Network → Unflatten → ConvNetwork, latent in the middle, reparameterise to sample. Once that pattern is in place the same machinery applies to any 2-D data with local spatial correlations — spectrogram patches today, image tiles tomorrow, raw waveforms in 1-D the day after.

The next note picks up the other major architectural family we haven't touched: **attention**. After that, the matching-pursuit + attention generative audio system that ties Part II together.

The library so far has two ways to mix information across positions in an input. A *dense* layer mixes them all-to-all with a fixed weight matrix that has to be learned exhaustively for every pair, which is parameter-greedy and translation-blind. A *convolution* mixes only neighbours within a fixed receptive field, which exploits locality brilliantly when locality is the right prior, and not at all when it isn't. Both share a hidden assumption: which positions interact, and how much, is set at *layer construction time*, baked into the weight shapes. Once you fix the network, the relationships are fixed.

Attention drops that assumption. Each position decides at *forward time*, from the data, which other positions it should listen to. The mechanism is small: a softmax over similarity scores. The consequences are large enough to have replaced almost everything else for language and a growing share of music and audio.

This note builds the attention operator from the ground up: scaled-dot-product attention as a pure function of (Q, K, V) , then multi-head attention as a `Layer<T>` that learns the projections. Note 22 will add the surrounding pieces (positional encoding, LayerNorm, a transformer encoder block).

27.1 Queries, keys, values

The unit of attention is a *retrieval*. Think of a dictionary lookup where the comparison is fuzzy. You have a *query* vector and a set of key/value pairs. The query is compared against every key; the closer the match the more that key's *value* is included in the result. With hard equality this is just a dictionary; with a soft, differentiable comparison it becomes a generalisation of one.

Concretely, for one query q in $\mathbb{R}^{d_k}$ and a set of n_k keys $\{k_j\}$ and values $\{v_j\}$:

<code>score_j = <q, k_j></code>	(a scalar
<code>similarity per key)</code>	
<code>weight_j = softmax(score_j over all keys)</code>	(a probability per
<code>key)</code>	
<code>out = sum_j weight_j * v_j</code>	(weighted
<code>combination)</code>	

That's it. Two operations: a similarity (a dot product), and a weighted sum. The softmax in between is what makes it differentiable and what guarantees that the weights sum to 1 — the result is a *convex combination* of the values, so the output stays in the value space.

For a *batch* of queries (in our case, every token in a sequence is its own query), we stack them as columns of a matrix $Q \in \mathbb{R}^{d_k \times n_q}$. Keys and values stack as columns of $K \in \mathbb{R}^{d_k \times n_k}$ and $V \in \mathbb{R}^{d_v \times n_v}$.

n_k }. The whole batched operation collapses to two matrix multiplications with a per-column softmax between them:

$S = K^T Q$	$(n_k \times n_q)$	every column =
scores for one query		
$A = \text{softmax}(S, \text{per column})$	$(n_k \times n_q)$	every column =
attention weights for one query		
$\text{out} = V A$	$(d_v \times n_q)$	every column =
weighted value combination		

In our column-as-example convention this lands cleanly: each column of the output is one query's retrieval.

27.2 The scaling

The formula above is the “raw” attention. Scaled-dot-product attention divides the scores by $\sqrt{d_k}$ before the softmax:

```
A = softmax( (K^T Q) / sqrt(d_k) )
```

The reason is statistical. If q and k are independent vectors with unit-variance components, then $\langle q, k \rangle$ has mean 0 and variance d_k . For large d_k the dot product spreads over a wide range, and the softmax becomes nearly one-hot: a tiny perturbation in the score flips the weight by a lot, and the gradient through softmax collapses to almost zero almost everywhere. The $\sqrt{d_k}$ rescaling keeps the score variance at 1 regardless of dimension, so the softmax stays in its “soft” regime where gradients flow. The Vaswani paper introduced the scaling for exactly this reason, and it's the difference between training that works and training that doesn't past $d_k \sim 64$.

27.3 Backward

The forward pass has two matmuls and a softmax sandwiched between them. The backward is the same in reverse: matmul backward (which we already had in spirit since $Z = W X$ plus its gradient is just $dW = dZ X^T$, $dX = W^T dZ$) and softmax backward (which `Softmax<T>` implements column-wise, exactly the form we need).

For $S = (K^T Q) / \sqrt{d_k}$, $A = \text{softmax}(S)$, $\text{out} = V A$, given $d\text{out} := dL/d\text{out}$:

```
dV = dout A^T                                (d_v x n_k)
dA = V^T dout                                (n_k x n_q)
dS = softmax_backward(dA, A) per column,
    dS[k, i] = A[k, i] * (dA[k, i] - sum_j A[j, i] dA[j, i])
dQ = (K @ dS) / sqrt(d_k)                    (d_k x n_q)
dK = (Q @ dS^T) / sqrt(d_k)                  (d_k x n_k)
```

We pack this in `ScaledDotProductAttention<T>` as a plain struct with its own `forward(Q, K, V)` and `backward(dOut)` returning a tuple `(dQ, dK, dV)`. It owns a `Softmax<T>` instance internally, which caches `A` for its own backward — that’s how we get the softmax-backward step for free.

The numerical-gradient check passes at the $\sim 1e-11$ level in double precision for all three gradients, which is the bar the library uses to declare a backward correct.

27.4 Why three inputs, why not a Layer

You may have noticed that `ScaledDotProductAttention` is *not* a `Layer<T>`. The library’s layer abstraction promises a single matrix input and a single matrix output. Attention has three inputs and one output, and although you could stack `Q, K, V` vertically into one big matrix and have the layer split it internally, doing so muddies the gradient routing and forces a specific layout choice on every caller. Instead we leave the bare operator as a non-Layer helper and wrap it in a Layer one level up.

27.5 Multi-head attention

A single attention operation produces one weighted combination per query. With one set of `(Q, K, V)` you get one “view” of the relationships between positions. Multi-head attention runs `H` such operations in parallel on disjoint slices of the feature axis, then concatenates and re-mixes the result. Each head can attend to something different — one might learn syntactic adjacency, another long-distance reference, another a position-independent pattern — and the model gets to use all of them.

The wiring, for input `X` in $\mathbb{R}^{d_model \times seq_len}$ (one token per column):

```
Q = Wq X                                (d_model x seq_len)
K = Wk X
V = Wv X
split feature axis into H heads (each of size d_head = d_model
    / H)
for h in 0..H-1:
    Q_h = rows [h*d_head, (h+1)*d_head) of Q    (d_head x
        seq_len)
    K_h, V_h similarly
    Y_h = ScaledDotProductAttention(Q_h, K_h, V_h)
    concat the Y_h back along the feature axis (d_model x seq_len
        )
Y = Wo (concat)
```

The four projections `Wq, Wk, Wv, Wo` are the only learnable parameters; everything else is the deterministic split-attend-concat. We implement them as four full `Dense<T>` layers because the column-as-example convention means each token in the sequence is naturally one “example” — so `Dense(d_model, d_model)` does the right thing without modification, and we inherit `He` initialisation, biases, `save/load` and the optimizer hook from `Dense`. Each head gets its own `ScaledDotProductAttention` instance,

which is what isolates the softmax caches so per-head backward works independently.

Gradient routing through MHA needs one care: in self-attention, x is the *common* input to W_q , W_k , and W_v , so the gradient returning from the three projection backwards must be summed:

```
Matrix<T> dXq = Wq_.backward(dQ);
Matrix<T> dXk = Wk_.backward(dK);
Matrix<T> dXv = Wv_.backward(dV);
return dXq + dXk + dXv;
```

It's a one-line detail that's easy to miss, and a perfect example of why finite-difference checks matter: a missing summand here would pass every shape test, run without warnings, and quietly train a broken model.

27.6 Two implementation traps we hit

These aren't deep ideas; they're the kind of thing the test suite catches and that you'd otherwise spend an afternoon debugging.

SIGFPE in the constructor. Our first version of `MultiHeadAttention(d_model, n_heads)` had `d_head_(d_model / n_heads)` in the member-initializer list, with the validation `if (n_heads == 0) throw ...` in the body. Member initializers run *before* the body. Integer division by zero on Linux is SIGFPE (a hardware trap), not a C++ exception, so passing `n_heads = 0` crashed the test executable before any of our error messages could fire. The fix is to do the validation in a static helper called from the initializer list, so the throw lands first.

One softmax cache per head, not one shared. Each head needs to remember *its own* attention weights for its backward pass, because softmax backward formulas depend on the forward output. If multiple heads shared a single `Softmax<T>` instance, the second head's forward would overwrite the first's cache, and the first head's backward would silently use the wrong weights. We give each head its own `ScaledDotProductAttention` (which in turn owns its own `Softmax<T>`); the per-head independence falls out of object ownership.

27.7 What's not here yet

Attention as written above is *bidirectional* — every query attends to every key in the sequence, no constraints. Three useful variants need a small extension to support:

- **Causal masking.** For language models that predict the next token, position i must not attend to positions $> i$. The fix is to set `S[j, i] = -inf` for $j > i$ *before* the softmax (so those weights become zero). One mask matrix, one entrywise add.

- ▶ **Padding masking.** Variable-length sequences in a batch get padded to a common length; the padded positions shouldn't contribute to anyone's attention. Same trick: `-inf` in the score matrix at padded positions.
- ▶ **Encoder-decoder cross-attention.** The query comes from the decoder's previous layer; the keys and values come from the encoder's output. Same operator, different sourcing of (Q, K, V) . We've already separated Q from K, V in the API, so this is just a usage pattern, not new code.

These all fit on top of `ScaledDotProductAttention` without changing its interface. Note 22 builds the surrounding pieces — sinusoidal positional encoding, `LayerNorm`, residual connections — and assembles a complete transformer encoder block, with all of those features in position to be added when we need them.

Positional encoding, LayerNorm, and the transformer block

28

Note 21 built attention as a pure operator on (Q, K, V) and wrapped it in MultiHeadAttention with learnable projections. By itself attention can't quite stand alone yet: it has no notion of *where* a token sits in the sequence, no normalisation to keep activations in a workable range across many layers, and no internal feed-forward to give each position its own learned transformation in between attention passes.

This note adds the three remaining pieces and assembles them into a **transformer encoder block** — the basic repeating unit of a transformer. None of the three ideas is deep, but skipping any one of them in a real transformer training run produces something that either won't learn or won't generalise. We'll build each one and end with a single composable Layer<T> that wraps them all.

28.1 Why position matters

Attention is *permutation-equivariant* on the token axis. If you shuffle the columns of the input, attention shuffles the columns of the output the same way — but each individual output column is unchanged. The output at position 3 doesn't know it's at position 3 versus position 7; it only knows what it asked for and what it found.

For images that's fine (a CNN treats translated patches the same), but for sequences it's a problem. "The cat sat on the mat" and "Mat the on sat cat the" should not produce the same set of token representations. We need to inject the position into each token's representation before attention sees it.

28.2 Sinusoidal positional encoding

The "Attention is All You Need" paper picked a beautifully simple solution: add a deterministic, fixed pattern to each input token's embedding, where the pattern depends only on the position.

For position *pos* and feature dimension *i*:

```
PE(pos, 2i)      = sin( pos / 10000^(2i / d_model) )
PE(pos, 2i+1)    = cos( pos / 10000^(2i / d_model) )
```

Two pieces of intuition:

Each position is unique. The frequency progression $10000^{(2i/d_model)}$ covers a geometric sweep — at low *i* (early features) the wavelength is short and the sine oscillates rapidly with *pos*; at high *i* the wavelength is enormous and the sine is nearly constant across the whole sequence. The combination of all these frequencies is unique for every position, like a fingerprint.

Relative offsets are linear. Because $\sin(a + b) = \sin a \cos b + \cos a \sin b$, the encoding $\text{PE}(\text{pos} + k)$ is a (position-independent) linear function of $\text{PE}(\text{pos})$. In principle the network can learn to attend by *offset* — “look 3 positions back” — just from the encoding, without needing to learn an explicit indexing scheme.

`SinusoidalPositionalEncoding<T>` precomputes the full $(d_{\text{model}} \times \text{max_seq_len})$ PE matrix at construction and just adds it column-wise in forward. Backward is the identity: $d(X + \text{PE})/dX = I$, so $dX = dY$. No parameters, no training cost.

A learned alternative (a $(d_{\text{model}} \times \text{max_seq_len})$ matrix of free parameters, no sinusoids) is also common in modern transformers. We’ll add that variant if a downstream chapter needs it; for now the sinusoidal version has the advantage of *extrapolating* to lengths beyond the longest one seen in training (the formulas keep producing distinct fingerprints).

28.3 LayerNorm

Stacking many attention blocks back to back has a tendency to drift the activation magnitudes — each block adds and rescales things, small errors accumulate. The original transformer paper used **LayerNorm** to keep them in check; modern transformer training without it is essentially never seen.

The formula is, per token (per column j of X):

```
mu_j      = (1/D) sum_i X[i, j]
var_j     = (1/D) sum_i (X[i, j] - mu_j)^2
x_hat[i,j] = (X[i, j] - mu_j) / sqrt(var_j + eps)
Y[i, j]   = gamma[i] * x_hat[i, j] + beta[i]
```

D is the feature dimension; γ and β are learnable D -vectors initialised to 1 and 0 respectively (so the layer starts as an exact standardisation and can drift to anything). The $+ \text{eps}$ (we use $1e-5$) keeps the divide safe when the variance is near zero.

LayerNorm differs from BatchNorm in *which* axis the statistics come from: BatchNorm uses the batch (column) axis, LayerNorm uses the feature (row) axis. Transformers prefer LayerNorm because (i) it works with any batch size including 1, (ii) it behaves identically at train and inference time without needing running averages, and (iii) the per-token statistics make sense for sequences of arbitrary length where the “batch” might be a single sentence.

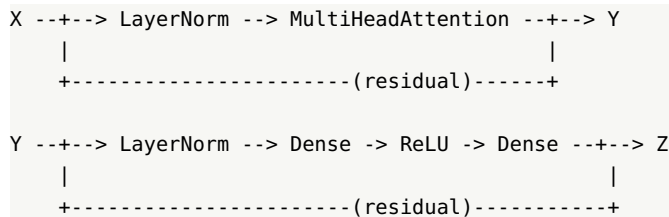
The backward is where LayerNorm earns its reputation for being finicky. $X[k, j]$ influences $x_{\text{hat}}[i, j]$ through three paths: directly (numerator), via the column mean μ_j , and via the column variance var_j . After careful chain-ruling, with $dx_{\text{hat}}[i, j] = dY[i, j] * \gamma[i]$, $S1_j = \sum_i dx_{\text{hat}}[i, j]$, and $S2_j = \sum_i dx_{\text{hat}}[i, j] * x_{\text{hat}}[i, j]$, the input gradient is:

```
dx[i, j] = (1 / (D * sqrt(var_j + eps))) *
           ( D * dx_hat[i, j] - S1_j - x_hat[i, j] * S2_j )
```


`dgamma[i]` and `dbeta[i]` accumulate as `sum_j dY[i, j] * x_hat[i, j]` and `sum_j dY[i, j]` respectively (the parameters are shared across the batch, so their gradients sum across it). The finite-difference test verifies all three to better than $1e-10$ in double precision, which is plenty to trust the implementation.

28.4 The transformer encoder block

With attention, positional encoding, and LayerNorm in place we can assemble the standard encoder block. The composition has been the subject of small but consequential variations; we use the **pre-LN** variant, which is now the default in modern transformer implementations because it trains more stably:



Two normalised sub-stacks (attention, feed-forward), each wrapped in a residual connection. The feed-forward sub-stack is a standard two-Dense bottleneck — `d_model -> d_ff -> d_model`, with `d_ff` usually $4 * d_{\text{model}}$. The $4\times$ expansion gives the per-position transformation enough capacity to do interesting work; the ReLU is the only nonlinearity in the sub-stack.

The **residual connections** are the load-bearing detail. Without them, training a stack of more than a handful of these blocks fails — gradients vanish through the long chain of matmuls and softmaxes. With them, the gradient gets an identity shortcut that keeps signal flowing, and the network can train hundreds of layers deep. It's the same trick as ResNet, in the same role.

In code, each block is one `Layer<T>` that owns six sub-layers in fixed order:

```

LayerNorm<T>      ln1_;
MultiHeadAttention<T> mha_;
LayerNorm<T>      ln2_;
Dense<T>          ff1_;
ReLU<T>           relu_;
Dense<T>          ff2_;

```

Forward chains them with the two residual sums. Backward chains them in reverse, with the residuals contributing identity gradients that sum with the sub-stack gradients at the residual junctions:

```

Matrix<T> backward(const Matrix<T>& dZ) {
    // FFN sub-stack: Z = Y + FFN(LN2(Y))
    Matrix<T> df = dZ;
    Matrix<T> de = ff2_.backward(df);
}

```

```

Matrix<T> dd = relu_.backward(de);
Matrix<T> dc = ffl_.backward(dd);
Matrix<T> dY_from_ffn = ln2_.backward(dc);
Matrix<T> dY = dZ + dY_from_ffn;

// Attention sub-stack: Y = X + Attn(LN1(X))
Matrix<T> db = dY;
Matrix<T> da = mha_.backward(db);
Matrix<T> dX_from_attn = ln1_.backward(da);
return dY + dX_from_attn;
}

```

The finite-difference check on dx agrees with the analytical gradient to about $1e-8$ in double precision — looser than the individual sub-layer checks (which were near $1e-11$) because numerical error accumulates across the six-layer chain, but well within the range that catches real bugs.

28.5 What we have

Counting the moving parts of one encoder block at $d_model = 512, n_heads = 8, d_ff = 2048$:

- ▶ Two LayerNorms: $2 * (512 + 512) = 2,048$ parameters
- ▶ One MultiHeadAttention: four $(512, 512)$ projections with bias = $4 * (512 * 512 + 512) = 1,050,624$ parameters
- ▶ One feed-forward: $(2048, 512) + (512, 2048)$ projections with bias = $2048*512 + 2048 + 512*2048 + 512 = 2,099,712$ parameters
- ▶ **Total per block: ~3.15 million parameters**

For comparison, our ConvVAE for audio (note 20) had ~2 million total across the whole encoder + decoder. A single encoder block at typical transformer sizes already outweighs the entire ConvVAE. This is the order-of-magnitude scaling that defines modern deep learning: not because the *operations* are any deeper, but because the *architectures* duplicate small blocks dozens of times. Twelve of these blocks stacked is a small transformer (~38M parameters); ninety-six of them stacked is GPT-2 large.

28.6 What's next

Two directions from here.

The decoder side of a full encoder–decoder transformer needs *causal* masked self-attention (each position attends only to earlier ones), and a *cross-attention* layer (the decoder queries the encoder output). Both are minor extensions of what we have: the score matrix gains an additive $-\infty$ mask before softmax, and the cross-attention just takes Q from one source and K, V from another. We've already separated those inputs in the SDPA interface, so it's a usage pattern, not new code.

The applied direction is what closes Part II: a generative audio system that combines matching pursuit (note 3) with attention. The form is up

for design — attention selecting MP atoms from a learned dictionary, attention over MP residuals, attention as a re-ranking step after greedy MP. Whatever the choice, it brings the two generative families we've built (dictionary-based and representation-based) into one place and points toward the chapters of Part III, which will be about creative applications rather than mechanism.

CREATION

APPENDICES

